

**Universidade do Minho**  
Escola de Engenharia  
Licenciatura em Engenharia Informática

## **Unidade Curricular de Laboratórios de Informática IV**

Ano Letivo de 2025/2026

### **Sistema para Cadeia de Lojas de Conveniência - Grupo 9**

**Afonso  
Martins**  
a106931

**Gonçalo  
Castro**  
a107337

**Francisco  
Lage**  
a106813

**Ricardo  
Neves**  
a106850

**Guilherme  
Costa**  
a107326

Maio, 2026

|                 |  |
|-----------------|--|
| Data da Receção |  |
| Responsável     |  |
| Avaliação       |  |
| Observações     |  |

## Sistema para Cadeia de Lojas de Conveniência - Grupo 9

**Afonso Martins**

a106931

**Mar-**

**Gonçalo Castro**

a107337

**Francisco Lage**

a106813

**Ricardo Neves**

a106850

**Guilherme**

**Costa**

a107326

Maio, 2026

## Resumo

A crescente integração de modelos de linguagem de grande escala (*Large Language Models*, LLM) no processo de Engenharia de *Software* abre novas perspetivas para aumentar a produtividade das equipas, melhorar a qualidade dos artefactos produzidos e reduzir o tempo de entrega de sistemas complexos. É neste contexto que se insere o presente trabalho, desenvolvido no âmbito da Unidade Curricular de Laboratórios de Informática IV da Licenciatura em Engenharia Informática da Universidade do Minho, cujo objetivo central é explorar de forma sistemática o uso de LLM ao longo de todas as fases do ciclo de vida do desenvolvimento de *software*.

O sistema desenvolvido — denominado *BelaVista* — consiste numa plataforma de gestão integrada para uma cadeia de pequenas lojas de conveniência. A arquitetura adotada é híbrida e distribuída: cada loja opera com o seu próprio *backend* local e base de dados independente, garantindo autonomia operacional total, enquanto os dados são consolidados diariamente num sistema central responsável pela geração de relatórios, estatísticas e governação da cadeia. O sistema cobre as principais necessidades operacionais identificadas nos requisitos (e no tema do projeto): gestão de inventário e stock, registo de vendas e faturação, gestão de fornecedores e funcionários, e geração de relatórios por loja, por período e por categoria de produto.

O desenvolvimento seguiu uma metodologia sequencial estruturada em quatro etapas — Engenharia de Requisitos, Arquitetura e Design, Implementação e Desenvolvimento, e Verificação e Validação — com recurso a LLM como agentes de apoio em cada uma delas. Os modelos foram utilizados na recolha e refinamento de requisitos, na produção de diagramas UML e documentação arquitetural, na geração e revisão de código, e na criação de casos de teste. Em cada secção do relatório é apresentada uma avaliação crítica do contributo do LLM, incluindo o *prompt* utilizado, o modelo e versão e uma análise dos resultados obtidos, tudo isto num template uniforme para que seja notória o apoio prestado pelos modelos de IA em todos os momentos em que foi usada (tenha sido útil ao trabalho ou não).

O sistema encontra-se funcional e cobre o âmbito definido no enunciado, com os dois *backends* implementados em *Kotlin/Spring Boot*, as bases de dados *SQLite* persistidas e a API REST documentada. A integração de LLM revelou-se particularmente impactante na fase de implementação, onde acelerou significativamente a produção de código a partir de especificações bem definidas.

**Área de Aplicação:** Engenharia de *Software* Assistida por LLM; Sistemas de Informação Distribuídos; Gestão de Cadeias de Retalho.

**Palavras-Chave:** *Large Language Models*, Inteligência Artificial, Engenharia de Requisitos, Arquitetura Distribuída, Kotlin, Spring Boot, Sincronização de Dados, Gestão de Inventário, *Waterfall*, API REST.

# Índice

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>1. Definição e Caracterização do Sistema</b>                              | <b>1</b>  |
| 1.1. Introdução                                                              | 1         |
| 1.2. Contextualização do Problema                                            | 2         |
| 1.3. Fundamentação                                                           | 3         |
| 1.4. Motivação e Objetivos                                                   | 4         |
| 1.5. Viabilidade                                                             | 5         |
| 1.6. Equipa de Trabalho                                                      | 6         |
| 1.7. Recursos                                                                | 7         |
| 1.8. Planeamento                                                             | 8         |
| <b>2. Levantamento e Análise de Requisitos</b>                               | <b>11</b> |
| 2.1. Identificação dos <i>Stakeholders</i>                                   | 11        |
| 2.1.1. <i>Stakeholders</i> Internos                                          | 11        |
| 2.1.2. <i>Stakeholders</i> Externos                                          | 11        |
| 2.1.3. <i>Stakeholders</i> Técnicos                                          | 12        |
| 2.2. Técnicas de Recolha de Requisitos                                       | 12        |
| 2.2.1. Questionários/Formulários                                             | 12        |
| 2.2.2. Atas de Reuniões                                                      | 14        |
| 2.2.2.1. <b>Ata de Reunião - Reunião de <i>Kickoff</i> com Direção Geral</b> | 14        |
| 2.2.2.2. <b>Ata de Reunião - Workshop com Gestores de Loja</b>               | 16        |
| 2.2.3. Entrevistas                                                           | 17        |
| 2.2.4. Observação Direta                                                     | 20        |
| 2.3. Documentos Analisados                                                   | 21        |
| 2.3.1. Fatura de Fornecedor                                                  | 21        |
| 2.3.2. Relatório de Vendas                                                   | 22        |
| 2.3.2.1. Produtos Mais Vendidos                                              | 22        |
| 2.4. Procedimentos Internos                                                  | 23        |
| 2.4.1. Política de Devoluções                                                | 23        |
| 2.4.2. Gestão de Stock                                                       | 23        |
| 2.5. Catálogo de Produtos                                                    | 25        |
| 2.6. Síntese das Necessidades Identificadas                                  | 25        |
| 2.7. <i>User Stories</i>                                                     | 26        |
| 2.7.1. Módulo de Gestão de Produtos                                          | 26        |
| 2.7.2. Módulo de Vendas e Faturação                                          | 27        |
| 2.7.3. Módulo de Gestão de Stocks                                            | 27        |
| 2.7.4. Módulo de Fornecedores                                                | 28        |
| 2.7.5. Módulo de Relatórios                                                  | 28        |
| 2.7.6. Módulo de Consolidação                                                | 29        |
| <b>3. Requisitos do Sistema</b>                                              | <b>32</b> |
| 3.1. Requisitos Funcionais                                                   | 32        |
| 3.1.1. RF1 — Gestão de Utilizadores                                          | 38        |
| 3.1.2. RF2 — Gestão de Lojas                                                 | 38        |
| 3.1.3. RF3 — Gestão de Produtos                                              | 39        |



|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| 3.1.4. RF4 — Gestão de Inventário e <i>Stock</i> .....              | 39        |
| 3.1.5. RF5 — Gestão de Vendas e Faturação .....                     | 39        |
| 3.1.6. RF6 — Estatísticas e Relatórios .....                        | 40        |
| 3.1.7. RF7 — Gestão de Fornecedores .....                           | 40        |
| 3.1.8. RF8 — Gestão de Clientes .....                               | 40        |
| 3.1.9. RF9 — Consolidação de Dados .....                            | 40        |
| 3.2. Requisitos Não Funcionais .....                                | 41        |
| 3.2.1. RNF1 — Integridade de Dados .....                            | 41        |
| 3.2.2. RNF2 — Performance .....                                     | 41        |
| 3.2.3. RNF3 — Segurança .....                                       | 42        |
| 3.2.4. RNF4 — Consistência e Consolidação .....                     | 42        |
| 3.2.5. RNF5 — Disponibilidade Operacional .....                     | 42        |
| <b>4. Modelação Conceptual .....</b>                                | <b>44</b> |
| <b>5. Diagrama de Casos de Uso .....</b>                            | <b>46</b> |
| 5.1. Descrição dos Casos de Uso .....                               | 47        |
| 5.1.1. Use Case: Autenticar .....                                   | 47        |
| 5.1.2. Use Case: Registar venda .....                               | 47        |
| 5.1.3. Use Case: Emitir fatura do cliente .....                     | 48        |
| 5.1.4. Use Case: Adicionar nova loja à cadeia .....                 | 48        |
| 5.1.5. Use Case: Calcular estatísticas da loja .....                | 49        |
| 5.1.6. Use Case: Calcular estatísticas da cadeia .....              | 49        |
| 5.1.7. Use Case: Consultar histórico de fornecimento de produto ... | 50        |
| 5.1.8. Use Case: Registar entrada de stock de produto .....         | 50        |
| 5.1.9. Use Case: Criar produto .....                                | 51        |
| 5.1.10. Use Case: Editar produto .....                              | 51        |
| 5.1.11. Use Case: Criar utilizador .....                            | 52        |
| 5.1.12. Use Case: Editar utilizador .....                           | 52        |
| 5.1.13. Use Case: Listar produtos .....                             | 53        |
| 5.1.14. Use Case: Associar Funcionário à loja .....                 | 53        |
| 5.1.15. Use Case: Criar perfil de cliente .....                     | 54        |
| 5.1.16. Use Case: Editar perfil de cliente .....                    | 54        |
| <b>6. Arquitetura Geral .....</b>                                   | <b>56</b> |
| 6.1. Comparação de Alternativas .....                               | 59        |
| <b>7. Modelação Arquitetural .....</b>                              | <b>61</b> |
| 7.1. Lógica Local (Loja) - Diagrama de Classes .....                | 61        |
| 7.2. Lógica Central (Cadeia) - Diagrama de Classes .....            | 62        |
| 7.3. Diagrama de Componentes .....                                  | 65        |
| 7.4. Diagramas de Sequência .....                                   | 66        |
| 7.4.1. Consolidação Diária .....                                    | 66        |
| 7.4.2. Gerar Relatório de Estatísticas por Loja .....               | 67        |
| 7.4.3. Autenticar .....                                             | 68        |
| 7.4.4. Registar Nova Loja .....                                     | 69        |
| 7.4.5. Listar Produtos .....                                        | 70        |
| 7.4.6. Registar Entrada de Stock .....                              | 70        |
| 7.4.7. Exportar Eventos Pendentes .....                             | 71        |
| 7.4.8. Listar Produtos .....                                        | 71        |
| <b>8. Principais Decisões .....</b>                                 | <b>72</b> |
| 8.1. Camada Cliente/Loja .....                                      | 72        |

|                                                                     |            |
|---------------------------------------------------------------------|------------|
| 8.2. Sistema Local (por Loja) .....                                 | 72         |
| 8.3. Sistema Central (Cadeia) .....                                 | 72         |
| 8.4. Mecanismo de Consolidação Diária .....                         | 73         |
| 8.5. Categoria como Entidade Escalável em vez de Enumeração .....   | 74         |
| 8.6. MovimentoStock Unificado como Operação de Inventário .....     | 74         |
| 8.7. Entidade Inventário .....                                      | 75         |
| 8.8. Utilizador como Classe Base com Herança para Funcionario ..... | 75         |
| 8.9. Cliente com NIF como Chave Primária Natural .....              | 75         |
| 8.10. Devolucao como Entidade separada de LinhaVenda .....          | 75         |
| 8.11. ConfigLoja como <i>Singleton</i> .....                        | 76         |
| 8.12. Bases de Dados Separadas por Loja .....                       | 76         |
| 8.13. Diagrama Lógico de Persistência .....                         | 76         |
| 8.13.1. Base de Dados Central .....                                 | 76         |
| 8.13.2. Base de Dados Local .....                                   | 77         |
| <b>9. Stack Tecnológica .....</b>                                   | <b>79</b>  |
| 9.1. <i>Frontend</i> .....                                          | 79         |
| 9.1.1. Escolha: Next.js + TypeScript + Tailwind CSS .....           | 79         |
| 9.2. <i>Backend</i> .....                                           | 79         |
| 9.2.1. Escolha: Kotlin + Spring Boot .....                          | 80         |
| 9.3. Base de Dados .....                                            | 80         |
| 9.3.1. Escolha: SQLite .....                                        | 80         |
| 9.4. Resumo .....                                                   | 80         |
| <b>10. Esboço das Interfaces do Sistema .....</b>                   | <b>82</b>  |
| 10.1. Interface e Prototipagem .....                                | 82         |
| 10.1.1. Vantagens do <i>Stitch</i> .....                            | 82         |
| 10.1.2. Desvantagens do <i>Stitch</i> .....                         | 83         |
| 10.1.3. Vantagens do Figma .....                                    | 83         |
| 10.1.4. Desvantagens do Figma .....                                 | 83         |
| 10.2. Descrição das Interfaces Prototipadas .....                   | 83         |
| <b>11. Especificação da API .....</b>                               | <b>91</b>  |
| 11.1. API Central .....                                             | 91         |
| 11.2. API Local .....                                               | 93         |
| 11.3. Contratos de Dados Relevantes .....                           | 94         |
| <b>12. Guia de Utilização .....</b>                                 | <b>96</b>  |
| <b>13. Planeamento Inicial VS Execução Final .....</b>              | <b>99</b>  |
| <b>14. Conclusões e Trabalho Futuro .....</b>                       | <b>101</b> |
| 14.1. Conclusões .....                                              | 101        |
| 14.2. Trabalho Futuro .....                                         | 101        |
| <b>Referências .....</b>                                            | <b>103</b> |
| <b>Lista de Siglas e Acrónimos .....</b>                            | <b>104</b> |
| <b>Anexos .....</b>                                                 | <b>105</b> |
| Anexo 1 Guia Completo do Sistema .....                              | 105        |
| Anexo 2 Ata da Reunião de Kick-off .....                            | 121        |
| Anexo 3 Ata de Workshop de Requisitos .....                         | 122        |
| Anexo 4 Logo da Universidade do Minho .....                         | 123        |

## Lista de Figuras

|           |                                                                                                                                                                                                                                            |    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 1  | Modelo Waterfall (modelo em cascata) aplicado ao ciclo de vida de desenvolvimento de software, evidenciando as fases sequenciais de requisitos, desenho, implementação, verificação e manutenção. <i>Adaptado de Mota (2015)</i> . . . . . | 8  |
| Figura 2  | Modelo em V aplicado à gestão e desenvolvimento de projetos, ilustrando a relação entre as fases de concepção, implementação e testes/validação ao longo do ciclo de vida do sistema. <i>Adaptado de Manfrini (2023)</i> . . . . .         | 9  |
| Figura 3  | Diagrama de Gantt com o planeamento temporal . . . . .                                                                                                                                                                                     | 9  |
| Figura 4  | Formulário de Recolha de informações gerais junto dos <i>Stakeholders</i> . . . . .                                                                                                                                                        | 13 |
| Figura 5  | Ata da reunião inicial do projeto com a Direção Geral da Cadeia de Lojas BelaVista . . . . .                                                                                                                                               | 15 |
| Figura 6  | Ata Workshop . . . . .                                                                                                                                                                                                                     | 16 |
| Figura 7  | Diagrama de Domínio do Problema das LOjas de Conveniência BelaVista . . . . .                                                                                                                                                              | 44 |
| Figura 8  | Diagrama de Use Cases . . . . .                                                                                                                                                                                                            | 46 |
| Figura 9  | Diagrama que esquematiza a Arquitetura de todo o Sistema BelaVista . . . . .                                                                                                                                                               | 57 |
| Figura 10 | Diagrama de Classes da Lógica de Negócio Local (Loja) . . . . .                                                                                                                                                                            | 62 |
| Figura 11 | Diagrama de Classes da Lógica de Negócio Central (Cadeia) . . . . .                                                                                                                                                                        | 63 |
| Figura 12 | Diagrama de Componentes de todo o Sistema BelaVista . . . . .                                                                                                                                                                              | 66 |
| Figura 13 | Diagrama de Sequência — Consolidação Diária . . . . .                                                                                                                                                                                      | 67 |
| Figura 14 | Diagrama de Sequência — Gerar Relatório de Estatísticas por Loja . . . . .                                                                                                                                                                 | 68 |
| Figura 15 | Diagrama de Sequência — Autenticar . . . . .                                                                                                                                                                                               | 69 |
| Figura 16 | Diagrama de Sequência — Registar nova loja . . . . .                                                                                                                                                                                       | 69 |
| Figura 17 | Diagrama de Sequência — Registar venda . . . . .                                                                                                                                                                                           | 70 |
| Figura 18 | Diagrama de Sequência — Registar entrada de stock . . . . .                                                                                                                                                                                | 70 |
| Figura 19 | Diagrama de Sequência — Exportar eventos pendentes . . . . .                                                                                                                                                                               | 71 |
| Figura 20 | Diagrama de Sequência — Exportar eventos pendentes . . . . .                                                                                                                                                                               | 71 |
| Figura 21 | Esquema Lógico da Base de Dados Cadeia . . . . .                                                                                                                                                                                           | 77 |
| Figura 22 | Esquema Lógico da Base de Dados Local (Loja) . . . . .                                                                                                                                                                                     | 78 |
| Figura 23 | Interface da plataforma Stitch . . . . .                                                                                                                                                                                                   | 82 |
| Figura 24 | Ecrã de <i>Dashboard</i> — resumo executivo para o diretor . . . . .                                                                                                                                                                       | 84 |
| Figura 25 | Ecrã de Vendas — ponto de venda com encomenda atual . . . . .                                                                                                                                                                              | 85 |
| Figura 26 | Ecrã de Lojas — visão geral da rede de lojas . . . . .                                                                                                                                                                                     | 86 |
| Figura 27 | Ecrã de Produtos — catálogo global em grelha . . . . .                                                                                                                                                                                     | 87 |
| Figura 28 | Ecrã de Inventário — gestão de <i>stock</i> e alertas de reposição . . . . .                                                                                                                                                               | 88 |
| Figura 29 | Ecrã de Funcionários — diretoria e distribuição por papel . . . . .                                                                                                                                                                        | 89 |
| Figura 30 | Ecrã de Relatórios — análise estratégica e <i>insights</i> automáticos . . . . .                                                                                                                                                           | 90 |
| Figura 31 | Diagrama de Gantt com o planeamento temporal . . . . .                                                                                                                                                                                     | 99 |
| Figura 32 | Diagrama de Gantt com a execução final do projeto . . . . .                                                                                                                                                                                | 99 |

## Lista de Tabelas

|           |                                                                                 |    |
|-----------|---------------------------------------------------------------------------------|----|
| Tabela 1  | <i>Stakeholders</i> Internos da cadeia de lojas .....                           | 11 |
| Tabela 2  | <i>Stakeholders</i> Externos da cadeia de lojas .....                           | 12 |
| Tabela 3  | <i>Stakeholders</i> Técnicos do projeto académico LI4 .....                     | 12 |
| Tabela 4  | Informação da reunião de <i>Kickoff</i> com a Direção Geral .....               | 15 |
| Tabela 5  | Informação do <i>Workshop</i> com os Gestores de Loja .....                     | 16 |
| Tabela 6  | Entrevista 1 — Gestora de Loja (Maria Santos) .....                             | 17 |
| Tabela 7  | Entrevista 2 — Caixa (João Ferreira) .....                                      | 18 |
| Tabela 8  | Entrevista 3 — Gestora Central (Ana Ribeiro) .....                              | 19 |
| Tabela 9  | Entrevista 4 — Fornecedor (Carlos Mendes) .....                                 | 20 |
| Tabela 10 | Atividades observadas nas lojas durante a observação direta .....               | 21 |
| Tabela 11 | Fatura de Fornecedor — Distribuidora Alimentar Norte Lda (FA-2025-01458) .....  | 22 |
| Tabela 12 | Relatório de Vendas Diárias — Janeiro 2025 .....                                | 22 |
| Tabela 13 | Produtos com maior rotatividade mensal .....                                    | 23 |
| Tabela 14 | Stock mínimo obrigatório por categoria de produto .....                         | 24 |
| Tabela 15 | Catálogo de produtos da cadeia de lojas com códigos EAN .....                   | 25 |
| Tabela 16 | Síntese das necessidades identificadas e respetivas origens de elicitação ..... | 26 |
| Tabela 17 | <i>User Stories</i> — Módulo de Gestão de Produtos .....                        | 27 |
| Tabela 18 | <i>User Stories</i> — Módulo de Vendas e Faturação .....                        | 27 |
| Tabela 19 | <i>User Stories</i> — Módulo de Gestão de Stocks .....                          | 28 |
| Tabela 20 | <i>User Stories</i> — Módulo de Fornecedores .....                              | 28 |
| Tabela 21 | <i>User Stories</i> — Módulo de Relatórios .....                                | 28 |
| Tabela 22 | <i>User Stories</i> — Módulo de Consolidação .....                              | 29 |
| Tabela 23 | Use Case: Autenticar .....                                                      | 47 |
| Tabela 24 | Use Case: Registar venda .....                                                  | 47 |
| Tabela 25 | Use Case: Emitir fatura do cliente .....                                        | 48 |
| Tabela 26 | Use Case: Adicionar nova loja à cadeia .....                                    | 48 |
| Tabela 27 | Use Case: Calcular estatísticas da loja .....                                   | 49 |
| Tabela 28 | Use Case: Calcular estatísticas da cadeia .....                                 | 49 |
| Tabela 29 | Use Case: Consultar histórico de fornecimento de produto .....                  | 50 |
| Tabela 30 | Use Case: Registar entrada de stock de produto .....                            | 50 |
| Tabela 31 | Use Case: Criar produto .....                                                   | 51 |
| Tabela 32 | Use Case: Editar produto .....                                                  | 51 |
| Tabela 33 | Use Case: Criar utilizador .....                                                | 52 |
| Tabela 34 | Use Case: Editar utilizador .....                                               | 52 |
| Tabela 35 | Use Case: Listar produtos .....                                                 | 53 |
| Tabela 36 | Use Case: Associar Funcionário à loja .....                                     | 53 |
| Tabela 37 | Use Case: Criar perfil de cliente .....                                         | 54 |
| Tabela 38 | Use Case: Editar perfil de cliente .....                                        | 54 |
| Tabela 39 | Comparação entre modelo unificado e modelos separados por contexto .....        | 60 |
| Tabela 40 | Subsistemas do sistema central, requisitos cobertos e interfaces expostas ..... | 73 |
| Tabela 41 | API Central — Módulo de Autenticação .....                                      | 91 |
| Tabela 42 | API Central — Módulo de Gestão da Cadeia e Lojas .....                          | 91 |
| Tabela 43 | API Central — Módulo de Gestão de Funcionários .....                            | 92 |
| Tabela 44 | API Central — Módulo de Estatísticas .....                                      | 92 |
| Tabela 45 | API Central — Módulo de Sincronização .....                                     | 92 |

|           |                                                                   |    |
|-----------|-------------------------------------------------------------------|----|
| Tabela 46 | API Central — Módulo de Fornecedores .....                        | 92 |
| Tabela 47 | API Local — Módulo de Autenticação .....                          | 93 |
| Tabela 48 | API Local — Módulo de Vendas e Faturação .....                    | 93 |
| Tabela 49 | API Local — Módulo de Produtos .....                              | 93 |
| Tabela 50 | API Local — Módulo de Stock e Fornecedores .....                  | 94 |
| Tabela 51 | API Local — Módulo de Clientes .....                              | 94 |
| Tabela 52 | API Local — Módulo de Sincronização .....                         | 94 |
| Tabela 53 | Principais <i>schemas</i> de dados das APIs Central e Local ..... | 95 |

# 1. Definição e Caracterização do Sistema

## 1.1. Introdução

Nos últimos anos, a Engenharia de *Software* tem vindo a sofrer uma transformação significativa, impulsionada pelo aumento da complexidade dos sistemas, pela crescente dependência de dados e pela necessidade de desenvolver soluções cada vez mais adaptáveis e eficientes aos olhos de utilizadores cada vez mais exigentes. Paralelamente, a evolução dos modelos de Inteligência Artificial, em particular os *Large Language Models (LLM)*, tem introduzido novas abordagens e oportunidades no processo de desenvolvimento de *software*, permitindo automatizar tarefas, apoiar a tomada de decisão e aumentar a produtividade das equipas de desenvolvimento.

Neste contexto, o presente documento, desenvolvido no âmbito da unidade curricular de Laboratórios de Informática IV, tem como objetivo a conceção, especificação, desenvolvimento e validação de um sistema de *software* aplicado a um problema real, seguindo uma abordagem estruturada e assistida por LLM ao longo de todo o ciclo de vida do desenvolvimento. O tema selecionado consiste na criação de um sistema de gestão integrada para uma cadeia de pequenas lojas de conveniência, cujo propósito é apoiar a gestão operacional e estratégica das lojas através do registo, processamento e análise centralizada de dados de vendas, produtos, *stocks*, fornecedores, entre outros.

O sistema desenvolvido deverá permitir que cada loja opere de forma autónoma no seu dia-a-dia, assegurando simultaneamente a consolidação periódica da informação num sistema central. Esta abordagem possibilita não só a gestão eficiente das operações locais, como também o suporte a processos de análise de dados e tomada de decisão ao nível global da cadeia. Entre as principais funcionalidades do sistema destacam-se a gestão de inventário, a reposição de produtos, a faturação e a geração de pequenos relatórios de gestão, estruturados por loja, período temporal e categoria de produto.

Para além do desenvolvimento do sistema em si, este projeto assume como objetivo central a avaliação da integração de LLM enquanto agentes de apoio ao processo de Engenharia de *Software*. Ao longo das diferentes etapas — desde a engenharia de requisitos, passando pela definição da arquitetura e design, até à implementação e validação — os LLM são utilizados como ferramentas de suporte cognitivo, contribuindo para a elicitação e refinamento de requisitos, geração de documentação, apoio ao desenvolvimento de código, identificação de melhorias e criação de casos de teste. Deste modo no final de cada secção apresentaremos informações sobre este apoio num formato simples incluindo o texto e documentos base (*prompt*), o modelo e versão utilizados e por fim uma análise crítica dos resultados obtidos, comentários sobre iterações ou refinações à instrução inicial.

Por fim, este trabalho pretende não só demonstrar a capacidade de desenvolver um sistema de *software* completo e funcional, mas também refletir criticamente sobre o papel dos LLM na Engenharia de *Software*, avaliando os seus benefícios, limitações e impacto na forma como os sistemas são concebidos, desenvolvidos e validados.

### Avaliação de Resposta LLM

#### PROMPT

Tendo em conta o tema abaixo, cria uma introdução para o meu relatório, incluindo os pontos esperados no enunciado do trabalho em anexo

#### CONTEXTO

*Enunciado do trabalho prático de LI4*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AVALIAÇÃO CRÍTICA

Dado que a introdução pode ser uma secção mais generalizada e baseada no enunciado, só foi preciso ajustarmos alguns pontos específicos no que toca "ao português".

## 1.2. Contextualização do Problema

O setor do comércio de retalho, em particular o segmento das pequenas lojas de conveniência, caracteriza-se por um elevado volume de transações diárias, grande diversidade de produtos e uma necessidade constante de reposição de *stock*. Este tipo de negócio exige rapidez operacional, controlo rigoroso de inventário e capacidade de adaptação às preferências dos consumidores.

Considere-se o caso da cadeia fictícia “BelaVista”, uma rede de pequenas lojas de conveniência distribuídas por diferentes localizações, onde cada unidade opera de forma autónoma. Cada loja é responsável pelo registo das suas vendas, controlo de inventário e gestão local de produtos e fornecedores, permitindo uma adaptação eficiente às necessidades específicas do seu contexto.

No entanto, esta autonomia resulta numa gestão descentralizada da informação, em que os dados são armazenados de forma isolada em cada loja. Esta fragmentação dificulta a obtenção de uma visão global do negócio, limitando a capacidade de análise e acompanhamento do desempenho da cadeia como um todo.

Adicionalmente, a ausência de integração entre os sistemas locais impede a consolidação eficiente dos dados operacionais, tornando difícil o acesso a informação estruturada e atualizada sobre vendas, *stocks* e fornecedores. Esta limitação compromete a capacidade de monitorização e controlo das operações, bem como a identificação de padrões relevantes para a gestão do negócio.

### Avaliação de Resposta LLM

#### PROMPT

Cria uma história de contextualização tendo em conta este tema inventando uma narrativa de um contexto fictício, enunciando objetivamente o contexto do problema com os principais conceitos associados e os problemas principais que se pretende resolver

|                      |                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CONTEXTO             | Descrição do Tema 1 fornecido no enunciado do trabalho                                                                                                                                                                                                                                                                                                                                       |
| MODELO               | GPT-5.3 (OpenAI)                                                                                                                                                                                                                                                                                                                                                                             |
| AValiação<br>CRÍTICA | A primeira resposta a esta instrução era demasiado generalizada e incluía aspetos que queríamos referir apenas nas secções seguintes, por isso reformulamos o comando inicial para reforçar mais a ideia de uma narrativa apropriada, e até obtermos um contexto que nos satisfizesse em termos de nome, negócio e história como um todo, sem estravar os objetivos de uma contextualização. |

### 1.3. Fundamentação

A gestão eficiente de cadeias de pequenas lojas de conveniência apresenta desafios significativos, resultantes da combinação entre operações distribuídas, elevada rotatividade de produtos e necessidade constante de atualização da informação. À medida que o número de lojas aumenta, torna-se progressivamente mais difícil garantir consistência, controlo e visibilidade sobre os dados gerados por cada unidade.

No contexto da cadeia BelaVista, a operação descentralizada das lojas, embora vantajosa do ponto de vista da flexibilidade local, origina uma fragmentação da informação que dificulta a gestão global do negócio. A inexistência de um sistema integrado impede a consolidação eficiente dos dados de vendas, *stocks* e fornecedores, limitando a capacidade de análise e dificultando a identificação de padrões relevantes para a operação.

Esta limitação tem impacto direto em vários aspetos críticos do negócio. Por um lado, a falta de visibilidade sobre os níveis de *stock* pode conduzir tanto a ruturas de produtos, comprometendo a satisfação dos clientes, como a situações de excesso de inventário, resultando em custos desnecessários. Por outro lado, a ausência de dados consolidados dificulta a comparação de desempenho entre lojas e a avaliação da eficiência operacional de cada unidade.

Adicionalmente, a gestão manual ou parcialmente automatizada dos processos de reposição e controlo de inventário aumenta a probabilidade de erro humano, reduzindo a fiabilidade da informação disponível. A inexistência de mecanismos sistemáticos de análise e de geração de relatórios limita ainda a capacidade da gestão em tomar decisões fundamentadas e atempadas.

Neste contexto, torna-se necessário desenvolver uma solução que permita integrar a informação proveniente das diferentes lojas, assegurando a sua consolidação, consistência e disponibilidade. Um sistema de gestão integrada possibilita não só o suporte às operações diárias de cada loja, como também a centralização dos dados num sistema único, permitindo uma visão global do negócio.

Desta forma, a implementação de um sistema desta natureza constitui um elemento fundamental para melhorar o controlo operacional, otimizar a gestão de inventário, reduzir ineficiências e apoiar os processos de tomada de decisão, contribuindo para uma gestão mais eficaz e sustentável da cadeia de lojas.



## Avaliação de Resposta LLM

### PROMPT

Seguindo o mesmo contexto, cria uma secção de Fundamentação que descreva a solução atual deste tipo de negócio, e o porquê da necessidade de um novo sistema, indicando claramente os pontos que melhorariam na introdução do novo sistema

### CONTEXTO

*Contextualização do Problema (secção acima)*

### MODELO

*Claude Sonnet 4.6 (Antropic)*

### AVALIAÇÃO CRÍTICA

Neste tópico a ideia tivemos de iterar uma vez para aprofundar mais os pontos no que toca principalmente à justificação da nova implementação, pois achamos a primeira resposta demasiado generalizada. Fizemo-lo pedindo mais detalhes no prompt seguinte.

## 1.4. Motivação e Objetivos

O presente projeto tem como objetivo geral o desenvolvimento de um sistema de gestão integrada para a cadeia de lojas de conveniência “BelaVista”, capaz de suportar de forma eficiente as operações diárias das lojas e, simultaneamente, permitir a centralização e análise da informação ao nível global da organização.

De forma mais específica, o sistema a desenvolver deverá cumprir os seguintes objetivos:

- Permitir o registo estruturado e consistente das vendas realizadas em cada loja;
- Assegurar o controlo eficiente de *stocks* e inventário, incluindo a monitorização dos níveis de produtos;
- Suportar a gestão de utilizadores, vendas, categorias e fornecedores;
- Automatizar os processos de registo de reposição de produtos nas diferentes lojas;
- Garantir a consolidação diária dos dados operacionais num sistema central;
- Disponibilizar mecanismos de faturação e devolução associados às vendas realizadas;
- Permitir a geração de relatórios de gestão por loja, período temporal e categoria de produto;
- Proporcionar uma visão global do desempenho da cadeia, facilitando a análise de dados e o apoio à tomada de decisão.

Desta forma, o sistema deverá contribuir para a melhoria da eficiência operacional das lojas, para a redução de erros associados à gestão manual e para o aumento da capacidade de controlo e planeamento por parte da gestão da cadeia.

## Avaliação de Resposta LLM

### PROMPT

Cria uma secção sobre a "motivação e objetivos" que diga os principais pontos a ter no sistema para responder aos problemas apresentados, e o porquê da necessidade de um novo sistema, indicando claramente os pontos fracos do sistema antigo e os pontos que o novo sistema é esperado culmar

### CONTEXTO

*Contextualização do Problema e Fundamentação (secções acima)*

### MODELO

*Claude Sonnet 4.6 (Antropic)*

### AVALIAÇÃO CRÍTICA

Neste tópico a ideia foi mais usar alguns outputs com pouca ou nenhuma alteração para ver que ideias diferentes de tópicos/funcionalidades surgiriam, até que selecionamos os presentes nesta secção. Os critérios de exclusão de outputs foram baseados na nossa expectativa para o trabalho, pelos requisitos principais diretamente retirados do parágrafo do Tema que nos foi atribuído ou simplesmente se achamos demasiado desadequado ao tema em questão.

## 1.5. Viabilidade

A viabilidade do desenvolvimento do sistema proposto para a cadeia “BelaVista” pode ser analisada sob diferentes perspetivas, nomeadamente técnica, operacional e temporal.

Do ponto de vista técnico, o sistema enquadra-se no domínio dos sistemas de informação tradicionais, envolvendo funcionalidades comuns como gestão de dados, controlo de inventário, processamento de transações e geração de relatórios. Estas funcionalidades podem ser implementadas recorrendo a tecnologias amplamente utilizadas e estáveis, nomeadamente bases de dados relacionais, arquiteturas em camadas e interfaces gráficas para interação com o utilizador. Assim, não se identificam limitações tecnológicas significativas que impeçam o desenvolvimento do sistema.

Relativamente à viabilidade operacional, o sistema proposto responde diretamente às necessidades identificadas no contexto da cadeia de lojas, permitindo melhorar a organização da informação, aumentar o controlo sobre as operações e apoiar os processos de gestão. A sua adoção não implica alterações profundas nos processos de negócio existentes, mas sim a sua digitalização e integração, o que facilita a sua aceitação e utilização por parte dos utilizadores. Ainda assim assumimos alguma possível carga adicional de adaptação aos novos processos mais digitalizados, por parte dos utilizadores finais, neste que seria um novo contexto face ao processo atual de trabalho (dependendo do contexto atual de cada negócio individual).

No que diz respeito à viabilidade temporal, o desenvolvimento do sistema encontra-se estruturado em várias etapas sequenciais, desde a análise de requisitos até à implementação e testes, permitindo uma evolução controlada e progressiva do projeto. Esta abordagem facilita o acompanhamento do progresso e a identificação atempada de eventuais problemas, contribuindo para o cumprimento dos prazos definidos. É ainda espetável que o processo tenha flexibilidade para sofrer alterações em fases terminais do projeto, para que se ajuste secções iniciais consoante o aprofundamento de estudo do tema, por exemplo: na fase de Desenvolvimento da Solução

dar-se conta de que um Use Case não cobre a totalidade do âmbito pedido, e fazer a alteração em conformidade (isto dá-se de forma direta já que o contexto do problema é fictício - não mapeia diretamente para um contexto real com um cliente real). Todas as principais alterações ao contexto inicial depois de validação primária, serão oportunamente validadas com a equipa docente (que faz o papel de interessado pelo projeto - “cliente”)

De forma global, conclui-se que o projeto apresenta condições favoráveis à sua realização, sendo tecnicamente exequível, operacionalmente relevante e ajustado ao período de desenvolvimento estabelecido.

### Avaliação de Resposta LLM

#### PROMPT

Tendo em conta o contexto do projeto BelaVista até agora e as secções anteriores (Contextualização, Fundamentação e Objetivos), cria uma secção de Viabilidade analisando as dimensões técnica, operacional e temporal, justificando a execução do sistema

#### CONTEXTO

*Contextualização do Problema, Fundamentação e Motivação e Objetivos*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AValiação CRÍTICA

Nesta secção a geração foi relativamente direta, uma vez que a estrutura da análise de viabilidade (técnica, operacional e temporal) é bastante standard. Ainda assim, foi necessário ajustar ligeiramente o texto para manter consistência com a terminologia usada nas secções anteriores e evitar repetições excessivas. Não foram necessárias iterações significativas, apenas pequenas reformulações para melhorar a clareza e adequação ao contexto específico, e as questões mais "humanas" de validação com os docentes e, no geral, contexto da UC.

## 1.6. Equipa de Trabalho

Para a implementação do *software* para a *BelaVista* ficou delineada uma equipa que consiste nos seguintes envolvidos:

- Pessoal Externo – Equipa de Engenheiros Informáticos Júniores nomeadamente: Gonçalo Castro, Afonso Martins, Ricardo Neves, Francisco Lage e Guilherme Costa são responsáveis pelo presente relatório e por todo o desenvolvimento técnico e modelação do Sistema que lhe é subjacente.
- Pessoal Interno – Funcionários da Cadeia (apresentados numa secção posterior), incluindo gestores da cadeia de mais alto nível mas também os cruciais trabalhadores de algumas das lojas locais da cadeia. Estas pessoas são a fonte principal de informação e requisitos, pois são os intervenientes e finais utilizadores do sistema que pretendemos desenvolver.

## 1.7. Recursos

Para além dos recursos humanos envolvidos no desenvolvimento do projeto, identificam-se de seguida os principais recursos necessários para garantir o seu bom progresso e execução:

- Computadores pessoais com capacidade suficiente para suportar o ambiente de desenvolvimento, incluindo ferramentas de modelação, sistemas de gestão de bases de dados e outros ambientes de programação inerentes às tecnologias escolhidas.
- Acesso a ferramentas de *software* adequadas para a modelação e desenvolvimento do sistema, nomeadamente ferramentas de desenho de diagramas UML (no nosso caso *Lucid Chart*, *Visual Paradigm* e *draw.io*), bem como ambientes de desenvolvimento integrados (*IDE*).
- Utilização de sistemas de controlo de versões, como o *Git*, para gestão do código-fonte, controlo de alterações e colaboração entre os elementos da equipa.
- Acesso a plataformas de apoio ao desenvolvimento assistido, nomeadamente ferramentas baseadas em modelos de linguagem (**Large Language Models**), utilizadas para apoio à engenharia de requisitos, geração de código, revisão e validação.
- Infraestrutura para alojamento do sistema, podendo incluir servidores locais ou soluções em *cloud*. No caso de infraestrutura local, poderão ser necessários equipamentos como servidores físicos, sistemas de armazenamento e mecanismos de proteção elétrica; em alternativa, poderão ser utilizados serviços de *cloud computing* (como AWS, Azure ou Google Cloud), garantindo maior escalabilidade e disponibilidade.

**Nota:** Ainda que o presente projeto não inclua explicitamente a fase de *deployment* do sistema, considera-se relevante a identificação destes recursos, tendo em vista uma futura implementação real do sistema de gestão integrada para a cadeia “BelaVista”.

### Avaliação de Resposta LLM

#### PROMPT

Identifica e descreve os principais recursos necessários à sua realização deste projeto, incluindo recursos tecnológicos, ferramentas de desenvolvimento, controlo de versões e infraestrutura de suporte

#### CONTEXTO

*Contextualização do Problema, Fundamentação e Motivação e Objetivos; Viabilidade*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AVALIAÇÃO CRÍTICA

A identificação dos recursos foi relativamente direta, uma vez que se trata de um tópico mais objetivo e baseado em práticas comuns de desenvolvimento de software. O output gerado foi utilizado quase na totalidade, tendo sido apenas ajustado ao nível da linguagem e da especificação de algumas ferramentas concretas utilizadas pelo grupo (como Lucidchart, Visual Paradigm, ...), de forma a alinhar o conteúdo com a realidade do projeto e das competências pretendidas para a UC.

## 1.8. Planeamento

O diagrama de Gantt apresentado ilustra o planeamento temporal das diferentes atividades do projeto, estruturadas de acordo com as várias fases do ciclo de desenvolvimento de *software*. O projeto encontra-se organizado em quatro etapas principais: engenharia de requisitos, arquitetura e *design*, implementação e, por fim, verificação e validação.

Importa ainda destacar que a atividade de escrita, revisão e validação do relatório decorre de forma transversal a todas as fases do projeto, assim como o acompanhamento e validação de processos com os docentes da Unidade Curricular. Estas tarefas acompanham continuamente o desenvolvimento, assegurando o registo das decisões tomadas, dos entregáveis produzidos e da evolução do sistema, contribuindo para a consistência e qualidade da documentação final.

O planeamento adotado segue uma abordagem inspirada em modelos clássicos de desenvolvimento de *software*, nomeadamente o modelo sequencial (**Waterfall**) e o modelo em V (**V-Model**), nos quais cada fase depende dos resultados da anterior e contribui para uma progressão estruturada do projeto. Esta metodologia permite garantir maior controlo sobre o processo de desenvolvimento, bem como uma validação sistemática dos resultados obtidos em cada etapa.

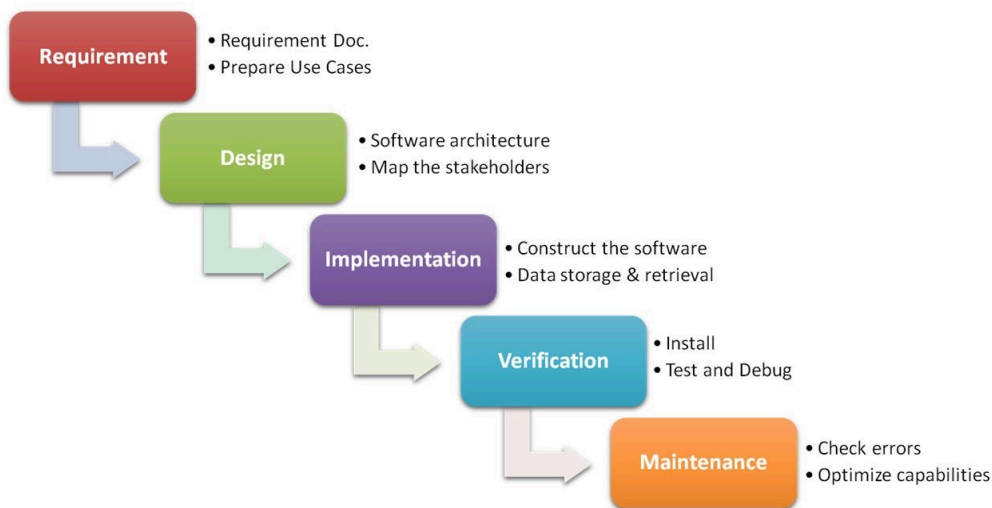


Figura 1: Modelo Waterfall (modelo em cascata) aplicado ao ciclo de vida de desenvolvimento de software, evidenciando as fases sequenciais de requisitos, desenho, implementação, verificação e manutenção. *Adaptado de Mota (2015).*

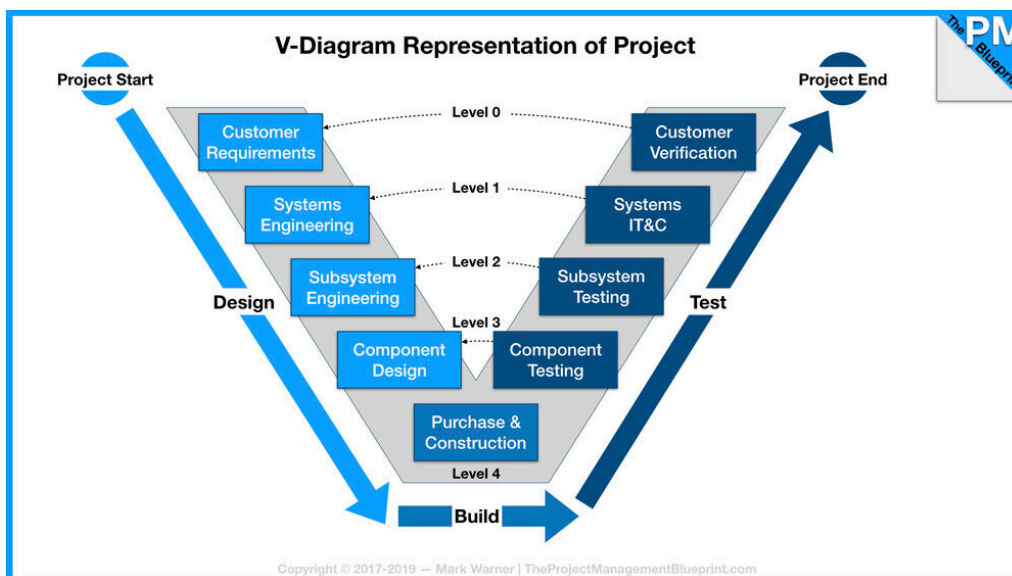


Figura 2: Modelo em V aplicado à gestão e desenvolvimento de projetos, ilustrando a relação entre as fases de concepção, implementação e testes/validação ao longo do ciclo de vida do sistema. *Adaptado de Manfrini (2023).*

Desta forma, o diagrama de Gantt não só representa a distribuição temporal das tarefas, como também reflete uma abordagem metodológica rigorosa, alinhada com boas práticas de Engenharia de *Software*, contribuindo para o cumprimento dos prazos e para a qualidade do sistema desenvolvido.

Sistema de Gestão Integrada para Cadeia de Lojas de Conveniência  
LM - Grupo 9

Project start: **sex, 2/20/2026**  
Display week: **1**

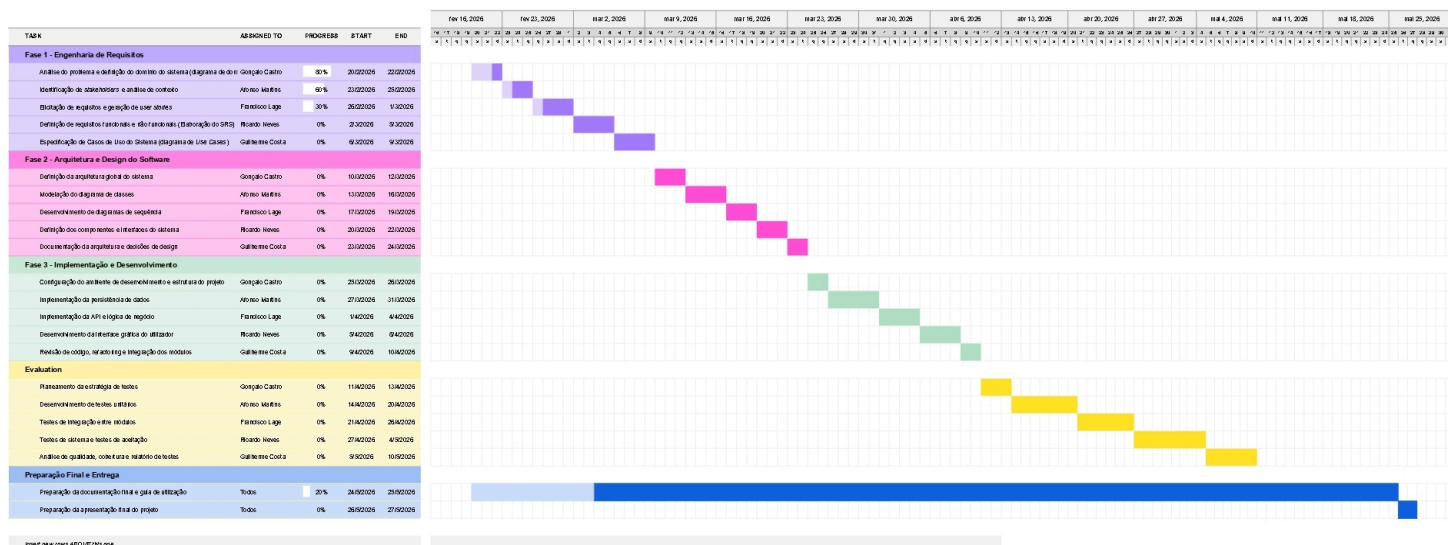


Figura 3: Diagrama de Gantt com o planejamento temporal

## Avaliação de Resposta LLM

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PROMPT            | Tendo em conta as datas previstas no enunciado do trabalho para cada etapa (mas tomando-as apenas como guia), dá-me os intervalos de execução, a partir do dia de hoje.                                                                                                                                                                                                                                                                                                                                                                        |
| CONTEXTO          | <i>Datas oficiais das etapas do projeto fornecidas no enunciado (FEV–MAI 2026)</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| MODELO            | <i>GPT-5.3 (OpenAI)</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| AVALIAÇÃO CRÍTICA | O modelo ajudou a refletir sobre a distribuição das tarefas ao longo das diferentes etapas, permitindo confirmar a coerência global e identificar pequenos ajustes na duração de algumas atividades. Foram feitas ligeiras adaptações para garantir consistência com o diagrama final, tendo em conta a nossa experiência na execução de trabalhos semelhantes até ao momento (principalmente alargando um pouco o tempo para a Engenharia de Requisitos e para a fase de Arquitetura que esperamos que demorem mais tempo do que o esperado). |

|                                  |                                                                                                                                                                                                                                                                                                                                                                                  |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Avaliação de Resposta LLM</b> |                                                                                                                                                                                                                                                                                                                                                                                  |
| PROMPT                           | Dá-me um pequeno texto de apoio a este diagrama de gantt para colocar na secção de "Planeamento" do meu relatório.                                                                                                                                                                                                                                                               |
| CONTEXTO                         | <i>Diagrama de GANTT da figura 1</i>                                                                                                                                                                                                                                                                                                                                             |
| MODELO                           | <i>GPT-5.3 (OpenAI)</i>                                                                                                                                                                                                                                                                                                                                                          |
| AVALIAÇÃO CRÍTICA                | No geral a primeira resposta foi satisfatória para o objetivo de acompanhar a figura 1, salientando a ajuda com a conformalização a duas metodologias conhecidas de Desenvolvimento de Software definidas pelo grupo. As únicas adições à versão final foi alguns aspetos mais específicos como a tarefa que dura a totalidade da timeline prevista que achamos melhor destacar. |

## 2. Levantamento e Análise de Requisitos

### 2.1. Identificação dos *Stakeholders*

Um *stakeholder* é qualquer indivíduo, grupo ou organização que possa afetar ou seja afetado pela implementação do sistema. A identificação sistemática dos *stakeholders* é fundamental para o sucesso do projeto.

#### 2.1.1. *Stakeholders* Internos

Nesta secção apresenta-se a caracterização dos *stakeholders* internos da cadeia de lojas, identificando, para cada um, a sua descrição e as principais responsabilidades e interesses face ao sistema a desenvolver. Esta análise é essencial para garantir que os requisitos levantados refletem as necessidades reais de todos os intervenientes, contribuindo para o sucesso da solução final.

| <i>Stakeholder</i> | Descrição                                            | Responsabilidades/Interesses                            |
|--------------------|------------------------------------------------------|---------------------------------------------------------|
| Direção Geral      | Órgão de gestão estratégica da cadeia de lojas       | Rentabilidade, crescimento, apoio à decisão estratégica |
| Gestor Central     | Responsável pela gestão global da cadeia             | Consolidação de dados, relatórios analíticos            |
| Gestor de Loja     | Responsável pela gestão operacional de cada loja     | Gestão diária, stocks, relatórios locais                |
| Caixa/Atendente    | Colaborador no processo de vendas                    | Registo de vendas, atendimento ao cliente               |
| Repositor          | Colaborador responsável pela organização de produtos | Alertas de reposição, gestão de exposição               |
| Administrador      | Responsável pela gestão técnica                      | Configuração, manutenção, segurança                     |

Tabela 1: *Stakeholders* Internos da cadeia de lojas

#### 2.1.2. *Stakeholders* Externos

Para além dos intervenientes internos, o sistema relaciona-se igualmente com diversas entidades externas à organização, cuja atuação tem impacto direto no funcionamento das lojas. Estes *stakeholders*, embora não pertençam à estrutura interna da cadeia, desempenham um papel relevante no fluxo de operações, seja no abastecimento de produtos, na concretização das vendas ou no cumprimento das obrigações administrativas e fiscais. A sua identificação permite assegurar que o sistema contempla as interações necessárias com o ambiente exterior, garantindo uma solução integrada e adequada à realidade do negócio.



| <b>Stakeholder</b> | <b>Descrição</b>                        | <b>Responsabilidades/Interesses</b>        |
|--------------------|-----------------------------------------|--------------------------------------------|
| Fornecedores       | Empresas que fornecem produtos às lojas | Encomendas eletrónicas, gestão de entregas |
| Clientes           | Compradores finais das lojas            | Rapidez no atendimento, disponibilidade    |
| Contabilidade      | Serviços de contabilidade               | Dados financeiros, faturação               |

Tabela 2: *Stakeholders* Externos da cadeia de lojas

### 2.1.3. *Stakeholders* Técnicos

No nosso caso académico necessitamos deste tipo de *stakeholders*, uma vez que o projeto se enquadra no contexto da unidade curricular de LI4, sendo desenvolvido por uma equipa de alunos sob orientação docente. Embora não façam parte da estrutura empresarial da cadeia de lojas, estes intervenientes assumem um papel determinante no ciclo de vida do sistema, sendo responsáveis pela sua conceção, desenvolvimento e validação. A sua identificação permite enquadrar adequadamente as responsabilidades técnicas e pedagógicas associadas ao projeto, garantindo o alinhamento entre os objetivos académicos e a qualidade da solução a entregar.

| <b>Stakeholder</b>        | <b>Descrição</b>                              | <b>Responsabilidades/Interesses</b>  |
|---------------------------|-----------------------------------------------|--------------------------------------|
| Equipa de Desenvolvimento | Alunos de LI4 responsáveis pela implementação | Requisitos claros, qualidade         |
| Docente Orientador        | Docente responsável pela orientação técnica   | Cumprimento de objetivos pedagógicos |

Tabela 3: *Stakeholders* Técnicos do projeto académico LI4

## 2.2. Técnicas de Recolha de Requisitos

A elicitação de requisitos constitui uma das fases mais críticas no desenvolvimento de qualquer sistema de informação, uma vez que dela depende a correta compreensão das necessidades dos utilizadores e do negócio. Para garantir uma recolha rigorosa, abrangente e fiável, optou-se pela aplicação combinada de várias técnicas complementares, permitindo cruzar diferentes perspetivas e obter uma visão integrada do problema. As técnicas selecionadas foram os questionários, as atas de reuniões e a análise documental, escolhidas tendo em conta o perfil dos *stakeholders* envolvidos, o tempo disponível e a natureza do projeto.

### 2.2.1. Questionários/Formulários

Os questionários foram a primeira técnica aplicada, por permitirem recolher informação de forma estruturada junto de um número alargado de colaboradores num curto espaço de tempo. Esta abordagem revelou-se particularmente útil para captar a perceção dos utilizadores finais sobre os processos atuais, identificar pontos de melhoria e estabelecer prioridades funcionais para o sistema a desenvolver. As perguntas foram desenhadas para combinar respostas quantitativas (escalas e *rankings*) com respostas qualitativas (opções múltiplas e campos abertos), de modo a obter dados que pudessem ser analisados estatisticamente, mas também enriquecidos com contributos individuais.

**Aplicação:** Questionário estruturado aplicado a 20 colaboradores das lojas (gestores, caixas, repositores).

## Sistema de Avaliação de Uso e Dificuldades

Avalie o uso de sistemas e as prioridades na sua loja

Com que frequência utiliza sistemas informáticos no seu trabalho? \*

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5

Nunca Sempre

Quais são as principais dificuldades que encontra no processo de vendas? \*

- ☐ Lentidão  
☐ Erros  
☐ Stock incorreto  
☐ Outro

Indique a ordem de prioridade das seguintes funcionalidades:

1: Stock  
2: Vendas  
3: Relatórios  
4: Faturação

Qual o seu método preferido para receber alertas? \*

- ☐ Notificação no sistema  
☐ Email  
☐ Outro

Qual o número aproximado de produtos por loja? \*

e.g., 23

Figura 4: Formulário de Recolha de informações gerais junto dos *Stakeholders*

A amostra foi cuidadosamente selecionada de forma a representar os diferentes perfis de utilizador que irão interagir com o sistema, assegurando que as conclusões refletem a diversidade de funções e responsabilidades existentes nas lojas. Os resultados obtidos encontram-se sumariados na tabela seguinte:

| Questão/Tema                                  | Tipo             | Opções/Resultados                                      |
|-----------------------------------------------|------------------|--------------------------------------------------------|
| Frequência de uso de sistemas informáticos    | Escala 1-5       | Média: 4.2 - Uso elevado                               |
| Principais dificuldades no processo de vendas | Múltipla escolha | Lentidão (80%), Erros (45%), Stock incorreto (60%)     |
| Funcionalidades prioritárias                  | Ranking          | 1º Stock, 2º Vendas, 3º Relatórios, 4º Faturação       |
| Método preferido de alertas                   | Escolha          | Notificação no sistema (65%), Email (25%), Outros(10%) |
| Número aproximado de produtos por loja        | Número           | 1000-2000 produtos                                     |

A análise dos dados recolhidos permitiu extrair várias ilações relevantes para o desenho da solução. Em particular, ficou evidente que existe uma forte familiaridade dos colaboradores com

ferramentas digitais, o que reduz o risco de resistência à mudança e indica que o sistema poderá adotar interfaces mais ricas sem comprometer a usabilidade. Por outro lado, os problemas identificados nos processos atuais (lentidão, erros e inconsistências de stock) constituem oportunidades claras de melhoria que o novo sistema deverá endereçar de forma prioritária.

#### **Conclusões dos Questionários:**

- 85% dos colaboradores utilizam sistemas informáticos diariamente
- A lentidão no processo de vendas é o principal problema identificado (80%)
- A gestão de stock é considerada a funcionalidade mais importante (70%)
- 65% preferem alertas via notificação no sistema
- Necessidade de interface intuitiva para utilizadores não técnicos

Estes resultados serviram de base para a definição das funcionalidades prioritárias do sistema, sendo posteriormente cruzados com a informação recolhida através das restantes técnicas, de forma a validar as conclusões e garantir a consistência dos requisitos identificados.

#### **2.2.2. Atas de Reuniões**

As reuniões com os diferentes *stakeholders* representaram um complemento essencial aos questionários, permitindo aprofundar tópicos específicos, esclarecer dúvidas e promover a discussão entre as várias partes interessadas. Ao contrário dos questionários, que oferecem uma visão mais ampla mas superficial, as reuniões possibilitaram explorar em detalhe as motivações, expectativas e restrições associadas ao projeto. Foram realizadas duas reuniões principais, cada uma com objetivos distintos e participantes específicos, devidamente registadas em atas para garantir a rastreabilidade das decisões tomadas.

##### **2.2.2.1. Ata de Reunião - Reunião de *Kickoff* com Direção Geral**

A reunião de *kickoff* marcou o arranque formal do projeto e teve como propósito alinhar a equipa de desenvolvimento com a visão estratégica da Direção Geral. Este momento foi determinante para estabelecer os objetivos de alto nível, as expectativas de negócio e os principais constrangimentos a respeitar ao longo de todo o ciclo de desenvolvimento.

### Ata da Reunião de Kickoff com a Direção Geral

A reunião de kickoff realizou-se no dia 27 de fevereiro de 2026 e contou com a presença de membros da Direção Geral da empresa e da equipa de desenvolvimento responsável pelo projeto. O principal objetivo deste encontro foi apresentar a visão global do sistema a desenvolver, alinhar expectativas entre as partes envolvidas e estabelecer os princípios orientadores que irão guiar o desenvolvimento da solução ao longo do semestre.

Durante a reunião, a Direção Geral destacou a necessidade de implementar um sistema de gestão integrada que permita às diferentes lojas da rede operar de forma autónoma no seu funcionamento diário, sem dependência constante de um sistema central. No entanto, foi também salientada a importância de garantir a existência de um mecanismo de consolidação automática dos dados gerados por cada loja, de forma a permitir que a informação relevante seja agregada e sincronizada com o sistema central no final de cada dia de operação.

Outro ponto considerado essencial foi o cumprimento rigoroso das obrigações legais associadas ao processo de faturação. A Direção Geral reforçou que o sistema deverá suportar faturação certificada, em conformidade com a legislação portuguesa em vigor, assegurando que todas as transações comerciais realizadas nas lojas possam ser registadas, armazenadas e auditadas de forma adequada.

Além disso, foi manifestado um forte interesse na capacidade de o sistema gerar relatórios analíticos detalhados que permitam à gestão central acompanhar o desempenho das diferentes lojas da rede. Estes relatórios deverão possibilitar análises comparativas entre lojas, identificação de tendências de vendas e avaliação do desempenho de categorias de produtos, constituindo assim uma ferramenta de apoio para processos de tomada de decisão estratégica.

No final da reunião, ficou acordado que a equipa de desenvolvimento deverá trabalhar numa primeira versão funcional do sistema que inclua as funcionalidades essenciais identificadas nesta fase inicial. Esta versão deverá estar disponível até ao final do semestre, permitindo assim a sua demonstração e avaliação por parte da Direção Geral e dos restantes stakeholders envolvidos no projeto.

Figura 5: Ata da reunião inicial do projeto com a Direção Geral da Cadeia de Lojas BelaVista

|                      |                                              |
|----------------------|----------------------------------------------|
| <b>Data</b>          | 27 de fevereiro de 2025                      |
| <b>Participantes</b> | Direção Geral, Equipa de Desenvolvimento LI4 |
| <b>Objetivo</b>      | Apresentar visão do projeto e expectativas   |

Tabela 4: Informação da reunião de *Kickoff* com a Direção Geral

Durante a reunião foram discutidos diversos aspetos estruturantes do projeto, com particular ênfase nos requisitos de conformidade legal, na arquitetura distribuída do sistema e nos prazos de entrega. A Direção Geral salientou a importância da autonomia operacional de cada loja, evitando dependências críticas de conectividade permanente, ao mesmo tempo que reforçou a necessidade de consolidação centralizada da informação para fins de gestão estratégica.

#### Resumo das Decisões:

- Sistema deve permitir gestão autónoma de cada loja
- Consolidação obrigatória ao final de cada dia
- Faturação certificada é requisito obrigatório
- Relatórios analíticos são prioritários para a gestão central
- Prazo para primeira versão funcional: fim do semestre

### 2.2.2.2. Ata de Reunião - Workshop com Gestores de Loja

Posteriormente, foi organizado um *workshop* com os gestores de loja, com o objetivo de descer ao nível operacional e recolher requisitos funcionais detalhados. Esta sessão revelou-se especialmente produtiva, uma vez que os gestores possuem um conhecimento profundo dos processos diários, das dificuldades sentidas pelas equipas e das oportunidades de otimização que não são facilmente percetíveis ao nível da Direção Geral.

#### Ata do Workshop com Gestores de Loja

O workshop com os gestores de loja teve lugar no dia 3 de março de 2026 e contou com a participação de três gestores responsáveis por diferentes lojas da rede, bem como com os membros da equipa de desenvolvimento do projeto. O principal objetivo desta sessão foi recolher informações detalhadas sobre as necessidades operacionais das lojas, identificar requisitos funcionais relevantes e compreender os principais desafios enfrentados no dia a dia da gestão comercial.

Durante a discussão, os gestores destacaram a importância de o sistema incluir mecanismos automáticos de monitorização do stock disponível em cada loja. Em particular, foi referida a necessidade de implementar alertas automáticos que notifiquem os utilizadores sempre que a quantidade de determinado produto atinja um nível mínimo previamente definido. Este tipo de funcionalidade é considerado essencial para evitar rupturas de stock e garantir a disponibilidade contínua dos produtos mais procurados pelos clientes.

Outro aspecto amplamente discutido foi a eficiência da interface utilizada no processo de venda. Os gestores enfatizaram que o sistema deverá proporcionar uma experiência de utilização simples, rápida e intuitiva para os operadores de caixa. Idealmente, cada transação de venda deverá poder ser concluída num período máximo de aproximadamente trinta segundos, de forma a evitar filas prolongadas e melhorar o atendimento ao cliente.

No que diz respeito à gestão de produtos, foi sugerido que o sistema permite organizar os artigos comercializados de acordo com diferentes categorias, facilitando assim a sua pesquisa e identificação durante o processo de venda. Para além disso, foi considerada importante a possibilidade de associar cada produto a um fornecedor específico, permitindo uma gestão mais eficiente dos processos de reposição e encomendas.

Os gestores manifestaram ainda interesse na disponibilização de relatórios de vendas diários, que permitam acompanhar o desempenho da loja ao longo do dia e identificar rapidamente padrões de consumo. Estes relatórios deverão incluir informações como volume de vendas, produtos mais vendidos, métodos de pagamento utilizados e períodos de maior movimento.

Por fim, foi discutida a necessidade de o sistema suportar múltiplas formas de pagamento, incluindo pagamentos em numerário, cartão bancário e outros métodos digitais. Esta funcionalidade é considerada fundamental para garantir flexibilidade no processo de venda e responder às diferentes preferências dos clientes.

Figura 6: Ata Workshop

|                      |                                                      |
|----------------------|------------------------------------------------------|
| <b>Data</b>          | 3 de março de 2026                                   |
| <b>Participantes</b> | 3 Gestores de Loja, Equipa de Desenvolvimento<br>LI4 |
| <b>Objetivo</b>      | Levantamento de requisitos funcionais                |

Tabela 5: Informação do *Workshop* com os Gestores de Loja

A discussão centrou-se sobretudo nas funcionalidades do dia a dia, com destaque para o processo de vendas, a gestão de *stock* e a comunicação com fornecedores. Os participantes contribuíram com exemplos concretos de situações problemáticas vivenciadas nas lojas, permitindo à equipa de desenvolvimento compreender com maior profundidade os requisitos não funcionais associados, nomeadamente em termos de desempenho, usabilidade e fiabilidade.

#### **Resumo das Decisões:**

- Necessidade de alertas automáticos de stock baixo
- Interface de venda deve ser rápida (máximo 30 segundos por transação)
- Gestão de produtos por categoria e fornecedor
- Relatórios de vendas diários são essenciais
- Suporte a múltiplas formas de pagamento

A combinação das duas reuniões permitiu obter uma visão coerente do sistema, articulando os requisitos estratégicos definidos pela Direção Geral com os requisitos operacionais identificados pelos gestores de loja. Esta articulação é fundamental para garantir que a solução final responde simultaneamente às necessidades de gestão e às exigências do trabalho diário nas lojas.

#### **Conclusões das Atas de Reuniões:**

- Autonomia das lojas durante o horário de funcionamento
- Consolidação diária automática é obrigatória
- Faturação certificada conforme legislação portuguesa
- Relatórios por loja, período e categoria de produto
- Alertas de reposição de stock são fundamentais

### **2.2.3. Entrevistas**

#### **Entrevista 1 - Gestora de Loja (Maria Santos)**

| <b>Campo</b>            | <b>Conteúdo</b>                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------|
| Data                    | 27 de fevereiro de 2026                                                                              |
| Perfil                  | Gestora de Loja há 5 anos                                                                            |
| Principais Necessidades | Sistema de gestão de stock em tempo real, alertas de produtos em falta, relatórios diários de vendas |

Tabela 6: Entrevista 1 — Gestora de Loja (Maria Santos)

A primeira entrevista foi realizada no dia 27 de fevereiro de 2026 com Maria Santos, gestora de uma das lojas da cadeia há aproximadamente cinco anos. O objetivo desta entrevista foi compreender as principais dificuldades enfrentadas na gestão diária da loja e identificar funcionalidades que poderiam melhorar a eficiência operacional.

Durante a entrevista, Maria Santos destacou que um dos maiores desafios na gestão da loja está relacionado com a falta de visibilidade imediata sobre o estado do stock. Atualmente, muitas verificações ainda são feitas manualmente ou com informação que nem sempre está atualizada em tempo real, o que pode levar a situações em que determinados produtos ficam indisponíveis sem que os funcionários se apercebam atempadamente. Esta situação pode resultar em perdas de vendas e insatisfação por parte dos clientes.

Nesse sentido, a gestora enfatizou a importância de um sistema que permita acompanhar o stock em tempo real, possibilitando a visualização imediata da quantidade disponível de cada produto. Para além disso, sugeriu a implementação de alertas automáticos sempre que o nível de stock

de um produto atingir um valor mínimo previamente definido. Estes alertas permitiriam agir rapidamente e iniciar o processo de reposição antes que ocorra uma rutura de stock.

Outro aspeto considerado fundamental pela entrevistada foi a existência de relatórios de vendas detalhados. Segundo Maria Santos, relatórios que apresentem o desempenho da loja em diferentes períodos — nomeadamente diário, semanal e mensal — seriam extremamente úteis para analisar tendências de consumo e ajustar estratégias de reposição e promoção de produtos.

Por fim, a gestora reforçou que a consolidação automática dos dados no final de cada dia seria uma funcionalidade essencial, permitindo garantir que todas as informações relativas às vendas e ao stock são corretamente sincronizadas com o sistema central da empresa.

#### **Conclusões:**

- Maior problema: falta de visibilidade sobre o stock em tempo real
- Necessita de alertas automáticos quando produto está abaixo do limiar
- Deseja relatórios de vendas por período (diário, semanal, mensal)
- Consolidação automática ao final do dia é essencial

#### **Entrevista 2 - Caixa (João Ferreira)**

| <b>Campo</b>            | <b>Conteúdo</b>                                                                            |
|-------------------------|--------------------------------------------------------------------------------------------|
| Data                    | 28 de fevereiro de 2026                                                                    |
| Perfil                  | Caixa há 3 anos                                                                            |
| Principais Necessidades | Interface simples e rápida, scanning de código de barras, suporte a devoluções e descontos |

Tabela 7: Entrevista 2 — Caixa (João Ferreira)

A segunda entrevista foi realizada no dia 28 de fevereiro de 2026 com João Ferreira, que exerce funções como operador de caixa há cerca de três anos. O principal objetivo desta entrevista foi compreender as necessidades específicas relacionadas com o processo de venda e com a utilização do sistema no ponto de atendimento ao cliente.

Durante a conversa, João Ferreira destacou que a rapidez e simplicidade do sistema são fatores extremamente importantes no contexto do atendimento ao público. Segundo o operador, durante períodos de maior movimento na loja é essencial que o processo de registo de produtos e finalização da venda seja o mais rápido possível, de forma a evitar filas prolongadas e reduzir o tempo de espera dos clientes.

Uma das funcionalidades consideradas indispensáveis é o suporte para leitura de códigos de barras através de scanners. Esta funcionalidade permite identificar automaticamente os produtos durante o processo de venda, reduzindo significativamente o tempo necessário para introduzir informações manualmente e diminuindo a probabilidade de erros.

Além disso, o entrevistado referiu a necessidade de o sistema suportar operações adicionais como devoluções de produtos e aplicação de descontos. Estas operações fazem parte do funcionamento normal da loja e devem ser realizadas de forma simples e rápida, sem exigir procedimentos complexos ou demorados.

João Ferreira destacou ainda que a interface do sistema deve ser clara e intuitiva, permitindo que os operadores de caixa executem as tarefas necessárias com o mínimo de cliques possível. Uma interface bem organizada pode reduzir o tempo de formação de novos funcionários e melhorar significativamente a eficiência no atendimento.

**Conclusões:**

- Processo de venda deve ser rápido (máximo 30 segundos)
- Scanning por código de barras é essencial
- Necessita de suporte para devoluções e descontos
- Interface deve ser intuitiva e fácil de usar

**Entrevista 3 - Gestora Central (Ana Ribeiro)**

| Campo                   | Conteúdo                                                                                 |
|-------------------------|------------------------------------------------------------------------------------------|
| Data                    | 1 de março de 2026                                                                       |
| Perfil                  | Gestora Central da cadeia                                                                |
| Principais Necessidades | Visão consolidada de todas as lojas, relatórios analíticos, comunicação com fornecedores |

Tabela 8: Entrevista 3 — Gestora Central (Ana Ribeiro)

A terceira entrevista foi conduzida no dia 1 de março de 2026 com Ana Ribeiro, gestora responsável pela supervisão central da cadeia de lojas. O objetivo desta entrevista foi compreender as necessidades de gestão ao nível estratégico e identificar as funcionalidades que permitiriam melhorar a monitorização global do negócio.

Durante a entrevista, Ana Ribeiro destacou que uma das principais necessidades da gestão central é a capacidade de obter uma visão consolidada de todas as lojas da rede. Atualmente, a obtenção de informações globais exige frequentemente a recolha manual de dados provenientes de diferentes fontes, o que torna o processo moroso e suscetível a erros.

A gestora salientou que um sistema centralizado que agregue automaticamente os dados provenientes de todas as lojas permitiria acompanhar em tempo real o desempenho da rede. Esta funcionalidade facilitaria a análise comparativa entre lojas, permitindo identificar rapidamente quais os estabelecimentos com melhor desempenho e quais aqueles que necessitam de maior atenção ou intervenção.

Outro ponto considerado prioritário foi a disponibilização de relatórios analíticos avançados. Estes relatórios deveriam permitir analisar tendências de vendas, identificar padrões de consumo e avaliar o desempenho de diferentes categorias de produtos ao longo do tempo. Segundo Ana Ribeiro, este tipo de informação é fundamental para apoiar decisões estratégicas relacionadas com campanhas promocionais, reposição de stock e gestão de fornecedores.

A entrevistada mencionou também a importância de um painel de controlo (*dashboard*) com indicadores-chave de desempenho (KPIs). Este *dashboard* permitiria visualizar rapidamente métricas importantes como volume total de vendas, produtos mais vendidos, desempenho por loja e evolução das vendas ao longo do tempo.

**Conclusões:**

- Visão consolidada de todas as lojas é prioritária
- Relatórios comparativos e tendências são essenciais
- Comunicação automática com fornecedores é prioritária
- Necessita de *dashboard* com KPI's relevantes

**Entrevista 4 - Fornecedor (Carlos Mendes)**



| <b>Campo</b>            | <b>Conteúdo</b>                                                      |
|-------------------------|----------------------------------------------------------------------|
| Data                    | 1 de março de 2026                                                   |
| Perfil                  | Representante de fornecedor                                          |
| Principais Necessidades | Sistema de recepção de encomendas eletrônico, comunicação automática |

Tabela 9: Entrevista 4 — Fornecedor (Carlos Mendes)

A quarta entrevista foi realizada no dia 1 de março de 2026 com Carlos Mendes, representante de um dos principais fornecedores de produtos da cadeia de lojas. O objetivo desta entrevista foi compreender de que forma o sistema poderia facilitar a comunicação e colaboração entre a empresa e os seus fornecedores.

Durante a entrevista, Carlos Mendes referiu que atualmente muitas encomendas ainda são realizadas através de correio eletrónico ou outros meios informais de comunicação. Este processo pode gerar atrasos, erros de interpretação ou dificuldades no acompanhamento do estado das encomendas.

Nesse sentido, o fornecedor manifestou preferência por um sistema eletrónico de gestão de encomendas que permita receber pedidos de reposição diretamente através da plataforma. Um sistema deste tipo facilitaria o processamento das encomendas, reduziria a probabilidade de erros e tornaria a comunicação entre as partes mais eficiente.

O entrevistado sugeriu ainda a possibilidade de automatizar a comunicação das necessidades de reposição de produtos. Por exemplo, quando o stock de determinado produto atingir um nível mínimo, o sistema poderia gerar automaticamente uma sugestão de encomenda ou notificação ao fornecedor responsável.

Por fim, Carlos Mendes destacou a utilidade de um mecanismo de acompanhamento das encomendas (*tracking*), que permita aos gestores das lojas e à gestão central acompanhar o estado dos pedidos realizados. Esta funcionalidade aumentaria a transparência no processo de abastecimento e facilitaria a coordenação logística entre fornecedores e lojas.

#### **Conclusões:**

- Sistema de encomendas eletrónicas é preferível ao email
- Comunicação automática de necessidades de reposição
- Preferência por sistema de *tracking* de encomendas

#### **2.2.4. Observação Direta**

##### **Atividades Observadas nas Lojas:**

| Atividade             | Observações                                                                                                                        |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Vendas no caixa       | Processo manual com elevada probabilidade de erro; não há código de barras em todos os produtos; tempo médio de venda: 2-3 minutos |
| Reposição             | Baseada em verificação física manual; sem sistema de alertas; ruturas frequentes de stock; sem visibilidade de prazos de validade  |
| Receção de encomendas | Processo burocrático com papelada extensa; sem verificação automática contra encomenda; erros frequentes de quantidades            |
| Fecho de loja         | Consolidação manual consome 30-45 minutos por dia; erros frequentes de transcrição; perda de tempo significativa                   |
| Gestão de stocks      | Inventário feito mensalmente de forma manual; sem visibilidade em tempo real; difícil planear reposições                           |

Tabela 10: Atividades observadas nas lojas durante a observação direta

#### Conclusões da Observação Direta:

- Processos manuais são lentos e propensos a erros
- Falta de visibilidade em tempo real do stock
- Necessidade de automatizar consolidação diária
- Sistema de alertas de stock baixo é essencial
- Scanning de produtos pode reduzir tempo de venda

## 2.3. Documentos Analisados

Para além das técnicas de elicitação que envolvem interação direta com os *stakeholders*, foi realizada uma análise documental aprofundada de diversos documentos fornecidos pela cadeia de lojas. Esta técnica revelou-se essencial para compreender, de forma objetiva e baseada em evidências, o funcionamento atual dos processos de negócio, bem como os formatos de dados, as relações com entidades externas e as obrigações legais a que a organização se encontra sujeita. A análise destes documentos permitiu não só validar a informação recolhida nas reuniões e questionários, como também identificar requisitos adicionais que não tinham sido explicitamente referidos pelos intervenientes.

### 2.3.1. Fatura de Fornecedor

A análise das faturas emitidas pelos fornecedores constitui uma fonte privilegiada de informação sobre os processos de aprovisionamento da cadeia de lojas. Estes documentos permitem compreender a estrutura de dados envolvida nas transações com fornecedores, as condições comerciais praticadas, os prazos de pagamento e os mecanismos de identificação de produtos utilizados. A título exemplificativo, apresenta-se em seguida uma fatura representativa, cuja estrutura serviu de base para a modelação do módulo de gestão de compras do sistema.

**Fornecedor:** Distribuidora Alimentar Norte Lda **Nº Fatura:** FA-2025-01458 **Data:** 03/02/2025  
**Condição de pagamento:** NET 30

| Produto                   | Código        | Quantidade | Preço Unitário | Total  |
|---------------------------|---------------|------------|----------------|--------|
| Arroz Agulha 1kg          | 5601234567890 | 100        | €1.10          | €110   |
| Massa Espiral 500g        | 5609876543210 | 80         | €0.85          | €68    |
| Azeite Virgem Extra 750ml | 5605555555555 | 40         | €4.50          | €180   |
| Leite Meio Gordo 1L       | 5602222222222 | 120        | €0.72          | €86.40 |

Tabela 11: Fatura de Fornecedor — Distribuidora Alimentar Norte Lda (FA-2025-01458)

**Total da Fatura:** €444.40

**Condições adicionais:**

- Entrega semanal
- Penalização de 2% em pagamentos após 30 dias

A partir da análise deste documento, foi possível identificar requisitos relevantes para o sistema, nomeadamente a necessidade de suportar códigos EAN como identificador único de produto, gerir múltiplas linhas de fatura por fornecedor, controlar prazos de pagamento e calcular automaticamente totais e eventuais penalizações. Estes elementos foram posteriormente incorporados nos requisitos funcionais do módulo de gestão de compras e contas a pagar.

### 2.3.2. Relatório de Vendas

Os relatórios de vendas representam outra fonte fundamental de informação, permitindo compreender quais os dados que atualmente são recolhidos pela cadeia de lojas, o nível de detalhe pretendido e os indicadores considerados mais relevantes para a gestão. A análise destes relatórios permitiu identificar as métricas que o novo sistema deverá calcular e disponibilizar, bem como os formatos de apresentação esperados pelos utilizadores finais.

**Relatório:** Vendas Diárias **Período:** Janeiro 2025

| Data       | Produto            | Qtd Vendida | Preço | Receita | Margem |
|------------|--------------------|-------------|-------|---------|--------|
| 01/01/2025 | Arroz Agulha 1kg   | 45          | €1.50 | €67.50  | 26%    |
| 01/01/2025 | Massa Espiral 500g | 30          | €1.20 | €36.00  | 29%    |
| 02/01/2025 | Azeite 750ml       | 12          | €6.50 | €78.00  | 31%    |
| 02/01/2025 | Leite 1L           | 60          | €1.10 | €66.00  | 28%    |

Tabela 12: Relatório de Vendas Diárias — Janeiro 2025

Verifica-se que o relatório incorpora não apenas dados de receita, mas também indicadores de margem, evidenciando a preocupação da gestão em monitorizar não só o volume de vendas mas também a rentabilidade de cada produto. Esta análise reforça a necessidade de o novo sistema integrar informação de compras e vendas para o cálculo automático de margens em tempo real.

#### 2.3.2.1. Produtos Mais Vendidos

Complementarmente, a análise documental permitiu identificar os produtos com maior rotatividade, informação determinante para o dimensionamento dos mecanismos de gestão de stock e para a definição de prioridades operacionais.

| Produto    | Quantidade Mensal |
|------------|-------------------|
| Leite 1L   | 1700              |
| Arroz 1kg  | 950               |
| Massa 500g | 780               |

Tabela 13: Produtos com maior rotatividade mensal

Estes valores demonstram a existência de um conjunto restrito de produtos responsáveis por uma fatia significativa do volume de vendas, justificando a implementação de mecanismos de alerta antecipado de reposição para esta categoria de artigos críticos.

## 2.4. Procedimentos Internos

A análise dos procedimentos internos da cadeia de lojas foi igualmente determinante para o levantamento de requisitos, uma vez que estes documentos formalizam as regras de negócio que o sistema deverá obrigatoriamente respeitar. Foram analisados procedimentos relativos à política de devoluções e à gestão de stock, dos quais se destacam os elementos mais relevantes para o desenvolvimento da solução.

### 2.4.1. Política de Devoluções

A política de devoluções estabelece as condições em que um cliente pode devolver um produto adquirido, definindo prazos, requisitos e modalidades de reembolso. A correta implementação destas regras no sistema é fundamental para garantir a uniformidade do atendimento ao cliente em todas as lojas da rede.

- Produtos não perecíveis podem ser devolvidos até **7 dias** após a compra.
- O produto deve apresentar:
  - embalagem intacta
  - comprovativo de compra
- O reembolso pode ser feito através de:
  - crédito em loja
  - devolução monetária.

A partir destas regras, foi identificada a necessidade de o sistema manter um histórico completo de todas as transações, permitindo a rápida consulta do comprovativo de compra e o registo de devoluções com a respetiva forma de reembolso, garantindo total rastreabilidade e suporte a eventuais auditorias.

### 2.4.2. Gestão de Stock

A política de gestão de stock define os limites mínimos a respeitar para cada categoria de produtos, assegurando a disponibilidade contínua de artigos essenciais e prevenindo situações de rutura. Estes limites foram estabelecidos com base no histórico de vendas e na sensibilidade comercial de cada categoria.

Cada categoria possui um **stock mínimo obrigatório**.

| <b>Categoria</b> | <b>Stock Mínimo</b> |
|------------------|---------------------|
| Arroz e Massas   | 100                 |
| Laticínios       | 80                  |
| Bebidas          | 120                 |
| Conservas        | 90                  |

Tabela 14: Stock mínimo obrigatório por categoria de produto

Quando o stock fica abaixo do mínimo:

- o sistema deve gerar um **alerta de reposição**.

Esta regra de negócio traduz-se diretamente num requisito funcional do sistema, que deverá monitorizar continuamente os níveis de stock e desencadear notificações automáticas sempre que os valores mínimos forem atingidos. Adicionalmente, este mecanismo deverá integrar-se com o módulo de comunicação com fornecedores, agilizando o processo de reposição.

## 2.5. Catálogo de Produtos

O catálogo de produtos constitui o documento de referência para a identificação e caracterização de todos os artigos comercializados pela cadeia de lojas. A sua análise permitiu compreender a estrutura de classificação utilizada, as relações entre produtos e fornecedores e os identificadores únicos adotados, elementos essenciais para a modelação da base de dados do sistema.

| Produto                      | Categoria  | Fornecedor           | Código EAN    |
|------------------------------|------------|----------------------|---------------|
| Arroz Agulha 1kg             | Cereais    | Distribuidora Norte  | 5601234567890 |
| Massa Espiral 500g           | Cereais    | Alimentar Ibérica    | 5609876543210 |
| Leite Meio Gordo 1L          | Laticínios | Lacticínios do Minho | 5602222222222 |
| Azeite Virgem Extra<br>750ml | Azeites    | Oliveiras SA         | 5605555555555 |
| Atum em Lata                 | Conservas  | Mar Azul             | 5603333333333 |

Tabela 15: Catálogo de produtos da cadeia de lojas com códigos EAN

A estrutura do catálogo evidencia a necessidade de o sistema suportar uma classificação hierárquica de produtos por categoria, bem como a associação de cada artigo a um ou mais fornecedores. O uso generalizado do código EAN como identificador único reforça igualmente a importância de o sistema integrar mecanismos de leitura por código de barras, agilizando tanto o processo de receção de mercadorias como o registo de vendas no ponto de venda.

## 2.6. Síntese das Necessidades Identificadas

Após a aplicação das diversas técnicas de elicitação e a análise dos documentos disponibilizados pela cadeia de lojas, foi possível consolidar um conjunto estruturado de necessidades que o sistema a desenvolver deverá obrigatoriamente endereçar. Esta síntese resulta do cruzamento das informações recolhidas através de questionários, atas de reuniões, observação direta e análise documental, garantindo assim a robustez e a abrangência do levantamento efetuado.

A tabela seguinte apresenta as necessidades identificadas, atribuindo a cada uma um identificador único que facilitará a sua rastreabilidade ao longo das fases subsequentes do projeto, nomeadamente na especificação de requisitos, no desenho da solução e na fase de testes. A coluna “Origem” indica as técnicas de elicitação que contribuíram para a identificação de cada necessidade, evidenciando o grau de validação cruzada obtido.

Com base nas técnicas de elicitação utilizadas, foram identificadas as seguintes necessidades prioritárias:

| ID  | Necessidade                             | Origem                                         |
|-----|-----------------------------------------|------------------------------------------------|
| N01 | Gestão de stock em tempo real           | Questionários, Entrevistas, Observação         |
| N02 | Alertas automáticos de stock baixo      | Entrevistas, Atas, Observação                  |
| N03 | Processo de venda rápido (< 30s)        | Entrevistas, Questionários                     |
| N04 | Scanning de código de barras            | Entrevistas, Observação                        |
| N05 | Consolidação automática diária          | Entrevistas, Atas, Observação                  |
| N06 | Relatórios por loja/período/categoria   | Entrevistas, Questionários, Análise Documental |
| N07 | Faturação certificada                   | Atas, Análise Documental                       |
| N08 | Comunicação eletrónica com fornecedores | Entrevistas, Atas                              |
| N09 | Dashboard KPI para gestão central       | Entrevistas                                    |
| N10 | Controlo de prazos de validade          | Análise Documental, Observação                 |

Tabela 16: Síntese das necessidades identificadas e respetivas origens de elicitação

A análise da tabela permite constatar que várias necessidades foram identificadas por múltiplas fontes, o que reforça a sua importância e prioridade no contexto do projeto. As necessidades validadas por três ou mais técnicas (N01, N02, N05 e N06) constituirão o núcleo funcional do sistema e deverão ser tratadas com prioridade máxima nas fases de desenho e implementação. As restantes necessidades, embora identificadas por menos fontes, foram igualmente consideradas relevantes e serão incorporadas no plano de desenvolvimento de acordo com as restrições temporais do projeto.

## 2.7. User Stories

As *user stories* constituem um instrumento fundamental na engenharia de requisitos ágil, permitindo descrever as funcionalidades do sistema do ponto de vista dos seus utilizadores finais. Ao contrário das especificações tradicionais, frequentemente extensas e técnicas, as *user stories* adotam uma linguagem natural e centrada no utilizador, facilitando a comunicação entre a equipa de desenvolvimento e os *stakeholders*. Esta abordagem promove ainda uma maior flexibilidade no processo de desenvolvimento, permitindo priorizar funcionalidades de forma incremental e ajustar o planeamento em função do *feedback* recebido ao longo do projeto.

No presente projeto, as *user stories* foram elaboradas seguindo a estrutura clássica “*como* [perfil de utilizador] -> [ação] -> *com o objetivo de* [benefício]”, complementada por critérios de aceitação claros que definem as condições necessárias para que cada história seja considerada concluída. Esta formulação assegura que cada funcionalidade está diretamente associada a uma necessidade real de um utilizador específico, evitando o desenvolvimento de funcionalidades supérfluas ou desalinhadas com os objetivos do sistema.

Com base nas necessidades identificadas, foram definidas as seguintes *user stories*:

### 2.7.1. Módulo de Gestão de Produtos

O módulo de gestão de produtos constitui o alicerce funcional do sistema, uma vez que todos os restantes módulos dependem da existência de um catálogo de produtos corretamente estruturado e atualizado. As *user stories* associadas a este módulo refletem a necessidade de permitir aos gestores de loja a manutenção autónoma do catálogo, garantindo simultaneamente

a integridade dos dados e a possibilidade de configurar parâmetros específicos como os limiares de stock mínimo.

| ID    | Testemunho                                                                                                                              | Critérios de Aceitação                               |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| US001 | como gestor de loja -> registar produtos -> com o objetivo de vender produtos com código de barras, nome, categoria, preço e fornecedor | Formulário completo, validação, criação com ID único |
| US002 | como gestor de loja -> editar produtos -> com o objetivo de manter os dados atualizados                                                 | Edição de todos os campos, histórico guardado        |
| US003 | como gestor de loja -> consultar stock -> com o objetivo de planejar a reposição                                                        | Consulta em tempo real, filtros por categoria        |
| US004 | como gestor central -> definir limiares mínimos -> com o objetivo de receber alertas automáticos                                        | Configuração por produto e loja                      |

Tabela 17: *User Stories* — Módulo de Gestão de Produtos

### 2.7.2. Módulo de Vendas e Faturação

O módulo de vendas e faturação representa o núcleo operacional do sistema, sendo aquele com o qual os utilizadores interagem com maior frequência ao longo do dia. As *user stories* deste módulo foram desenhadas tendo em conta a necessidade de garantir uma elevada rapidez no processamento de transações, o cumprimento integral das obrigações legais em matéria de faturação e a flexibilidade necessária para acomodar diferentes formas de pagamento e políticas promocionais.

| ID    | Testemunho                                                                                   | Critérios de Aceitação                     |
|-------|----------------------------------------------------------------------------------------------|--------------------------------------------|
| US005 | como caixa -> registar vendas com scanning -> com o objetivo de processar vendas rapidamente | Leitura de código, atualização instantânea |
| US006 | como caixa -> aplicar descontos -> com o objetivo de oferecer promoções                      | Suporte a descontos percentuais e fixos    |
| US007 | como caixa -> emitir fatura/recibo -> com o objetivo de cumprir obrigações legais            | Fatura em formato legal, impressão e PDF   |
| US008 | como caixa -> processar pagamentos -> com o objetivo de servir todos os clientes             | Dinheiro, cartão, MB Way                   |

Tabela 18: *User Stories* — Módulo de Vendas e Faturação

### 2.7.3. Módulo de Gestão de Stocks

A gestão de stocks é uma das áreas críticas identificadas durante a fase de elicitação de requisitos, tendo sido apontada por uma larga maioria dos colaboradores como uma das maiores fragilidades dos sistemas atualmente utilizados. As *user stories* deste módulo procuram dar resposta a estas preocupações, automatizando processos de monitorização, prevenindo erros humanos e garantindo o cumprimento das normas legais aplicáveis, nomeadamente no que respeita ao controlo de prazos de validade.



| ID    | Testemunho                                                                                            | Critérios de Aceitação                       |
|-------|-------------------------------------------------------------------------------------------------------|----------------------------------------------|
| US009 | como repositor -> receber alertas de stock -> com o objetivo de repor atempadamente                   | Notificação automática, listagem de produtos |
| US010 | como gestor de loja -> registar entradas de stock -> com o objetivo de manter o inventário atualizado | Registo detalhado, atualização automática    |
| US011 | como sistema -> bloquear produtos fora de validade -> com o objetivo de cumprir a lei                 | Bloqueio automático, alerta ao caixa         |

Tabela 19: *User Stories* — Módulo de Gestão de Stocks

#### 2.7.4. Módulo de Fornecedores

O relacionamento com os fornecedores é um aspeto estratégico para a operação da cadeia de lojas, uma vez que dele depende a continuidade do abastecimento e, consequentemente, a capacidade de resposta às necessidades dos clientes. As *user stories* definidas para este módulo visam dotar os gestores de loja de ferramentas que permitam manter um registo organizado dos fornecedores, agilizar o processo de criação de encomendas e acompanhar o estado das entregas em curso.

| ID    | Testemunho                                                                                  | Critérios de Aceitação               |
|-------|---------------------------------------------------------------------------------------------|--------------------------------------|
| US012 | como gestor de loja -> gerir fornecedores -> com o objetivo de manter contactos atualizados | CRUD completo, histórico de entregas |
| US013 | como gestor de loja -> criar encomendas -> com o objetivo de repor stock                    | Criação, envio, tracking             |

Tabela 20: *User Stories* — Módulo de Fornecedores

#### 2.7.5. Módulo de Relatórios

A geração de relatórios constitui um dos principais requisitos identificados pela Direção Geral, sendo encarada como uma ferramenta essencial para o apoio à tomada de decisão estratégica. As *user stories* deste módulo cobrem diferentes níveis de análise, desde a consulta operacional ao nível de cada loja até à visualização consolidada do desempenho global da rede, contemplando ainda a disponibilização de *dashboards* com indicadores-chave para monitorização em tempo real.

| ID    | Testemunho                                                                                                 | Critérios de Aceitação                  |
|-------|------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| US014 | como gestor de loja -> consultar relatórios de vendas -> com o objetivo de acompanhar o desempenho         | Filtros por período, produto, categoria |
| US015 | como gestor central -> consultar relatórios consolidados -> com o objetivo de analisar o desempenho global | Agregação de todas as lojas             |
| US016 | como gestor central -> consultar dashboard KPI -> com o objetivo de monitorizar em tempo real              | KPIs: vendas, margem, stock             |

Tabela 21: *User Stories* — Módulo de Relatórios

2.7.6. Módulo de Consolidação

Por fim, o módulo de consolidação assegura a integração entre as operações realizadas localmente em cada loja e a visão centralizada exigida pela Direção Geral. Tal como definido na reunião de *kickoff*, este processo deverá ocorrer de forma automática e fiável, garantindo que a informação relevante é sincronizada com o sistema central no final de cada dia de operação. As *user stories* deste módulo refletem ainda a preocupação em assegurar a integridade dos dados consolidados, atribuindo ao administrador do sistema a responsabilidade de validar e detetar eventuais inconsistências.

| ID    | Testemunho                                                                                                           | Critérios de Aceitação                   |
|-------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------|
| US017 | como sistema -> consolidar dados automaticamente -> com o objetivo de manter dados atualizados para o gestor central | Processo automático, tratamento de erros |
| US018 | como administrador -> verificar integridade dos dados -> com o objetivo de garantir relatórios confiáveis            | Validação, deteção de inconsistências    |

Tabela 22: *User Stories* — Módulo de Consolidação

Em suma, o conjunto de *user stories* apresentado abrange a totalidade das necessidades identificadas durante a fase de elicitação, organizando-as de forma modular e facilitando o planeamento iterativo do desenvolvimento. Cada *user story* encontra-se associada a critérios de aceitação concretos, que servirão de base para a validação funcional das entregas e para a definição dos casos de teste a executar ao longo do projeto.

Avaliação de Resposta LLM

PROMPT

CONTEXTO

MODELO

AVALIAÇÃO CRÍTICA

Tendo em conta o contexto de uma cadeia de pequenas lojas de conveniência, identifica e classifica os principais stakeholders do sistema, separando-os em internos, externos e técnicos, e indicando para cada um as suas responsabilidades e interesses.

Descrição geral do projeto e do negócio da cadeia de lojas

Claude Sonnet 4.5 (Anthropic)

A resposta gerada apresentou uma divisão clara entre stakeholders internos, externos e técnicos, com tabelas bem estruturadas. No entanto, foi necessário acrescentar manualmente o stakeholder "Repositor", inicialmente omitido, e refinar a descrição do papel do Administrador de sistema para refletir a realidade pretendida no contexto académico.

### Avaliação de Resposta LLM

#### PROMPT

Elabora um questionário estruturado para aplicar a colaboradores de lojas de conveniência com o objetivo de levantar requisitos para um novo sistema de gestão. O questionário deve incluir perguntas de escala, múltipla escolha e ranking.

#### CONTEXTO

*Descrição do projeto e tipologia dos colaboradores envolvidos*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AVALIAÇÃO CRÍTICA

O modelo gerou um questionário com 15 perguntas, organizadas por tema, cobrindo frequência de uso de sistemas, dificuldades sentidas e funcionalidades prioritárias. Foi necessário reduzir o número de perguntas para evitar sobrecarregar os inquiridos e ajustar algumas escalas, uma vez que estavam demasiado granulares para o contexto pretendido.

### Avaliação de Resposta LLM

#### PROMPT

Com base nas necessidades identificadas, escreve um conjunto de user stories para um sistema de gestão de uma cadeia de lojas, seguindo o formato "como [perfil] -> [ação] -> com o objetivo de [benefício]" e incluindo critérios de aceitação claros para cada uma.

#### CONTEXTO

*Tabela de necessidades identificadas (N01 a N10) e lista de perfis de utilizador*

#### MODELO

*Gemini 2.5 Pro (Google)*

#### AVALIAÇÃO CRÍTICA

A primeira iteração produziu user stories genéricas e pouco específicas para o domínio da retalho alimentar. Após reformular o prompt com exemplos concretos e indicação dos módulos pretendidos, o modelo gerou um conjunto coerente de user stories organizadas por módulo, embora os critérios de aceitação tenham sido posteriormente refinados manualmente para incluir referências a tecnologias específicas como MB Way e leitura de código de barras.

### Avaliação de Resposta LLM

#### PROMPT

Redige uma ata de reunião de kickoff entre a equipa de desenvolvimento e a Direção Geral de uma cadeia de lojas, evidenciando a visão estratégica, os requisitos de alto nível e as decisões tomadas relativamente ao sistema a desenvolver.

#### CONTEXTO

*Resumo das principais decisões e perfil dos participantes*

#### MODELO

*Claude Sonnet 4.5 (Anthropic)*

#### AVALIAÇÃO CRÍTICA

A resposta gerou um documento bem estruturado, em tom formal e adequado a um contexto empresarial. Foi necessário ajustar manualmente a data da reunião e adicionar uma referência explícita à obrigatoriedade de faturação certificada em conformidade com a legislação portuguesa, requisito que não havia sido devidamente enfatizado no prompt inicial.

### Avaliação de Resposta LLM

#### PROMPT

A partir das user stories definidas, sugere uma divisão modular para o sistema, identificando os módulos funcionais principais e as dependências entre eles.

#### CONTEXTO

*Lista completa de user stories (US001 a US018)*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AVALIAÇÃO CRÍTICA

O modelo identificou corretamente seis módulos principais (Produtos, Vendas, Stocks, Fornecedores, Relatórios e Consolidação) e apresentou um diagrama textual das dependências. No entanto, falhou em destacar a transversalidade do módulo de Consolidação, sendo necessário reformular o prompt para que o modelo reconhecesse esta característica essencial da arquitetura distribuída pretendida.

## 3. Requisitos do Sistema

### 3.1. Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve disponibilizar para suportar as operações da cadeia de lojas. A sua identificação resultou de um processo de elicitação estruturado, que incluiu entrevistas com *stakeholders*, reuniões de levantamento e questionários aplicados a colaboradores (como demonstrado na secção anterior). Cada requisito é identificado por um código único no formato **RFx.y**, onde **x** representa o módulo funcional e **y** o número sequencial dentro desse módulo.

Os requisitos encontram-se organizados por nove módulos temáticos: **gestão de utilizadores** (RF1.x), **gestão de lojas** (RF2.x), **gestão de produtos** (RF3.x), **gestão de inventário e stock** (RF4.x), **gestão de vendas e faturação** (RF5.x), **estatísticas e relatórios** (RF6.x), **gestão de fornecedores** (RF7.x), **gestão de clientes** (RF8.x) e **consolidação de dados** (RF9.x). Para cada requisito é registada a data de identificação, o ator com permissão para executar a funcionalidade, o método de elicitação, a fonte, o analista responsável e o grau de prioridade.

| ID    | Data       | Descrição                                                                                                                                          | Ator                  | Método     | Fonte        | Analista       | Prior. |
|-------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------|--------------|----------------|--------|
| RF1.1 | 01/03/2026 | O sistema deve permitir o registo de utilizadores com perfis distintos: Funcionário, Gerente e Administrador Central.                              | Gestor, Administrador | Entrevista | Maria Santos | Afonso Martins | Alta   |
| RF1.2 | 12/02/2026 | O sistema deve permitir autenticação através de identificador único e palavra-passe, recusando o acesso a utilizadores inativos.                   | Todos                 | Entrevista | Maria Santos | Gonçalo Castro | Alta   |
| RF1.3 | 09/02/2026 | O sistema deve aplicar controlo de acessos baseado no cargo do utilizador, restringindo o acesso às funcionalidades correspondentes a cada perfil. | Sistema               | Reunião    | Maria Santos | Ricardo Neves  | Alta   |
| RF1.4 | 17/02/2026 | O sistema deve permitir ao Administrador associar um Funcionário a uma Loja específica.                                                            | Administrador         | Reunião    | Maria Santos | Francisco Lage | Alta   |

|              |            |                                                                                                                                                                                              |                       |            |              |                 |       |
|--------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------|--------------|-----------------|-------|
| <b>RF1.5</b> | 16/02/2026 | O sistema deve permitir desativar um utilizador mantendo o seu histórico de operações associado para efeitos de auditoria.                                                                   | Gestor, Administrador | Entrevista | Maria Santos | Francisco Lage  | Média |
| <b>RF2.1</b> | 16/02/2026 | O sistema deve permitir o registo de novas Lojas associadas a uma Cadeia existente, validando a existência da Cadeia antes do registo.                                                       | Administrador         | Reunião    | Ana Ribeiro  | Guilherme Costa | Alta  |
| <b>RF2.2</b> | 13/02/2026 | O sistema deve permitir consultar informação detalhada de uma Loja, incluindo localização, funcionários afetos e inventário atual.                                                           | Gestor, Administrador | Entrevista | Ana Ribeiro  | Francisco Lage  | Alta  |
| <b>RF2.3</b> | 12/02/2026 | O sistema deve permitir calcular estatísticas de desempenho por Loja (número de vendas e faturação total), filtráveis por período temporal, validando a existência da Loja antes do cálculo. | Gestor, Administrador | Reunião    | Ana Ribeiro  | Afonso Martins  | Alta  |
| <b>RF3.1</b> | 02/03/2026 | O sistema deve permitir criar novos Produtos com identificador único, nome, categoria, preço e Fornecedor(es), validando a existência dos Fornecedores referenciados.                        | Gestor, Administrador | Entrevista | Ana Ribeiro  | Guilherme Costa | Alta  |
| <b>RF3.2</b> | 26/02/2026 | O sistema deve permitir editar os atributos de um Produto existente.                                                                                                                         | Gestor, Administrador | Entrevista | Ana Ribeiro  | Francisco Lage  | Alta  |
| <b>RF3.3</b> | 11/02/2026 | O sistema deve permitir associar um ou mais Fornecedores a um Produto.                                                                                                                       | Gestor, Administrador | Reunião    | Ana Ribeiro  | Guilherme Costa | Alta  |

|              |            |                                                                                                                                                                                                    |             |              |               |                |       |
|--------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|--------------|---------------|----------------|-------|
| <b>RF3.4</b> | 27/02/2026 | O sistema deve permitir consultar e filtrar Produtos por Categoria.                                                                                                                                | Todos       | Questionário | João Ferreira | Gonçalo Castro | Média |
| <b>RF3.5</b> | 11/03/2026 | O sistema deve permitir identificar Produtos através da leitura de código de barras (EAN/UPC).                                                                                                     | Funcionário | Entrevista   | João Ferreira | Gonçalo Castro | Alta  |
| <b>RF4.1</b> | 22/02/2026 | O sistema deve manter um Inventário individualizado por Loja, operacional de forma autónoma sem dependência do sistema central.                                                                    | Sistema     | Reunião      | Ana Ribeiro   | Francisco Lage | Alta  |
| <b>RF4.2</b> | 10/02/2026 | O sistema deve registar a quantidade disponível em stock de cada Produto por Loja, impedindo valores negativos.                                                                                    | Sistema     | Reunião      | Ana Ribeiro   | Francisco Lage | Alta  |
| <b>RF4.3</b> | 09/02/2026 | O sistema deve atualizar automaticamente o stock após o registo de cada Venda, deduzindo as quantidades correspondentes às Linhas de Venda, recusando a venda caso o stock seja insuficiente.      | Sistema     | Entrevista   | Ana Ribeiro   | Ricardo Neves  | Alta  |
| <b>RF4.4</b> | 11/02/2026 | O sistema deve permitir ao Funcionário registar entradas de stock provenientes de Fornecedores, indicando o fornecedor e a quantidade entregue, validando a existência do Fornecedor referenciado. | Funcionário | Reunião      | Ana Ribeiro   | Ricardo Neves  | Média |
| <b>RF4.5</b> | 15/02/2026 | O sistema deve manter um histórico completo de movimentações de stock, incluindo entradas, saídas por venda, ajus-                                                                                 | Sistema     | Entrevista   | Ana Ribeiro   | Francisco Lage | Média |

|              |            |                                                                                                                                                                                                                                                  |                       |              |               |                 |       |
|--------------|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|--------------|---------------|-----------------|-------|
|              |            | tes manuais e devoluções.                                                                                                                                                                                                                        |                       |              |               |                 |       |
| <b>RF4.6</b> | 11/03/2026 | O sistema deve gerar alertas automáticos ao Gestor quando o stock de um Produto atingir o limiar mínimo configurado por categoria, através de notificação no próprio sistema (preferência indicada por 65% dos colaboradores nos questionários). | Sistema               | Entrevista   | Maria Santos  | Ricardo Neves   | Alta  |
| <b>RF4.7</b> | 11/03/2026 | O sistema deve permitir ao Gestor ou Administrador configurar o limiar mínimo de stock por categoria de Produto.                                                                                                                                 | Gestor, Administrador | Reunião      | Maria Santos  | Ricardo Neves   | Alta  |
| <b>RF5.1</b> | 16/02/2026 | O sistema deve permitir ao Funcionário registar uma Venda associada à sua Loja, validando a existência e disponibilidade de stock de cada Produto antes de confirmar o registo.                                                                  | Funcionário           | Entrevista   | Ana Ribeiro   | Gonçalo Castro  | Alta  |
| <b>RF5.2</b> | 25/02/2026 | O sistema deve permitir associar múltiplas Linhas de Venda a uma Venda, cada uma correspondendo a um Produto e respetiva quantidade.                                                                                                             | Funcionário           | Entrevista   | Ana Ribeiro   | Francisco Lage  | Alta  |
| <b>RF5.3</b> | 28/02/2026 | O sistema deve permitir associar opcionalmente um Cliente a uma Venda para efeitos de faturação, aceitando o NIF como identificação mínima suficiente, sem exigir perfil completo no momento da venda.                                           | Funcionário           | Questionário | João Ferreira | Guilherme Costa | Média |
| <b>RF5.4</b> | 09/02/2026 | O sistema deve calcular automaticamente                                                                                                                                                                                                          | Sistema               | Reunião      | Ana Ribeiro   | Ricardo Neves   | Alta  |



|              |            |                                                                                                                                                                       |                       |            |               |                 |       |
|--------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|------------|---------------|-----------------|-------|
|              |            | o valor total da Venda com base nas Linhas de Venda registadas.                                                                                                       |                       |            |               |                 |       |
| <b>RF5.5</b> | 26/02/2026 | O sistema deve registar automaticamente a data e hora de cada Venda no momento do seu registo.                                                                        | Sistema               | Reunião    | Ana Ribeiro   | Francisco Lage  | Alta  |
| <b>RF5.6</b> | 15/02/2026 | O sistema deve permitir emitir Fatura associada a uma Venda, em formato imprimível e eletrónico.                                                                      | Funcionário           | Entrevista | Ana Ribeiro   | Guilherme Costa | Média |
| <b>RF5.7</b> | 11/03/2026 | O sistema deve permitir ao Funcionário registar a devolução parcial ou total de Produtos de uma Venda, revertendo automaticamente o stock das quantidades devolvidas. | Funcionário           | Entrevista | João Ferreira | Afonso Martins  | Média |
| <b>RF6.1</b> | 01/03/2026 | O sistema deve calcular o número total de Vendas por Loja, filtrável por período temporal.                                                                            | Gestor, Administrador | Reunião    | Ana Ribeiro   | Francisco Lage  | Alta  |
| <b>RF6.2</b> | 26/02/2026 | O sistema deve calcular a faturação total por Loja, filtrável por período temporal e categoria de Produto.                                                            | Gestor, Administrador | Reunião    | Ana Ribeiro   | Gonçalo Castro  | Alta  |
| <b>RF6.3</b> | 22/02/2026 | O sistema deve permitir ao Administrador consultar estatísticas agregadas ao nível da Cadeia, por loja, período e categoria de Produto.                               | Administrador         | Entrevista | Ana Ribeiro   | Guilherme Costa | Média |
| <b>RF7.1</b> | 16/02/2026 | O sistema deve permitir registar Fornecedores com nome, NIF, morada, contacto telefónico e endereço eletrónico.                                                       | Gestor, Administrador | Reunião    | Ana Ribeiro   | Francisco Lage  | Alta  |
| <b>RF7.2</b> | 23/02/2026 | O sistema deve permitir consultar o histórico de fornecimen-                                                                                                          | Gestor, Administrador | Entrevista | Ana Ribeiro   | Afonso Martins  | Média |

|              |            |                                                                                                                                                                                                                                                                                                                                                 |                       |         |             |                 |       |
|--------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|---------|-------------|-----------------|-------|
|              |            | tos por Produto, incluindo fornecedor, data e quantidade.                                                                                                                                                                                                                                                                                       |                       |         |             |                 |       |
| <b>RF7.3</b> | 11/03/2026 | O sistema deve permitir ao Gestor ou Administrador associar Fornecedores a Lojas específicas, validando a existência de ambos antes da associação.                                                                                                                                                                                              | Gestor, Administrador | Reunião | Ana Ribeiro | Francisco Lage  | Alta  |
| <b>RF8.1</b> | 11/03/2026 | O sistema deve suportar dois modos de identificação de cliente: identificação mínima por NIF (para faturação rápida no ponto de venda) e perfil completo com nome, morada, NIF, número de telemóvel e endereço eletrónico (para clientes fidelizados). O NIF é o identificador único em ambos os casos, impedindo o registo de NIFs duplicados. | Funcionário           | Reunião | Ana Ribeiro | Guilherme Costa | Média |
| <b>RF8.2</b> | 11/03/2026 | O sistema deve permitir ao Funcionário editar os dados de um Cliente registado.                                                                                                                                                                                                                                                                 | Funcionário           | Reunião | Ana Ribeiro | Guilherme Costa | Média |
| <b>RF8.3</b> | 27/05/2026 | O sistema deve permitir ao Funcionário promover uma identificação mínima por NIF a perfil completo, preenchendo os dados em falta em momento posterior à venda.                                                                                                                                                                                 | Funcionário           | Análise | Ana Ribeiro | Guilherme Costa | Baixa |
| <b>RF8.4</b> | 11/03/2026 | O sistema deve permitir ao Funcionário consultar e pesquisar Clientes registados por NIF ou nome.                                                                                                                                                                                                                                               | Funcionário           | Reunião | Ana Ribeiro | Ricardo Neves   | Baixa |
| <b>RF9.1</b> | 11/03/2026 | O sistema deve consolidar automaticamente os dados operacionais de cada Loja                                                                                                                                                                                                                                                                    | Sistema               | Reunião | Ana Ribeiro | Gonçalo Castro  | Alta  |

|              |            |                                                                                                                                                                                                                                     |         |         |             |                |      |
|--------------|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|---------|-------------|----------------|------|
|              |            | no sistema central no fecho do dia, sem intervenção manual, registando o estado de cada consolidação (PENDENTE, EM_CURSO, CONCLUIDO ou ERRO).                                                                                       |         |         |             |                |      |
| <b>RF9.2</b> | 11/03/2026 | O sistema deve garantir idempotência no processo de consolidação, detetando e rejeitando eventos duplicados com base num identificador único de envio ( <i>hash</i> ), permitindo o reenvio seguro em caso de falha de comunicação. | Sistema | Reunião | Ana Ribeiro | Gonçalo Castro | Alta |
| <b>RF9.3</b> | 11/03/2026 | O sistema deve registar o motivo de falha para cada evento não processado durante a consolidação, permitindo diagnóstico e reproprocessamento posterior sem perda de dados.                                                         | Sistema | Reunião | Ana Ribeiro | Gonçalo Castro | Alta |

### 3.1.1. RF1 — Gestão de Utilizadores

A gestão de utilizadores constitui a base do controlo de acesso ao sistema. Este módulo estabelece três perfis distintos — Funcionário, Gerente e Administrador Central —, garantindo que cada colaborador acede apenas às funcionalidades correspondentes ao seu cargo (RF1.1, RF1.3). A criação de perfis compete ao Gestor ou Administrador, sendo o Administrador o único com permissão para associar um funcionário a uma loja específica (RF1.4).

A autenticação é realizada por identificador único e palavra-passe (RF1.2), incluindo a validação do estado ativo do utilizador — um utilizador desativado é recusado mesmo com credenciais válidas. Foi ainda identificada a necessidade de desativar utilizadores sem eliminar o seu histórico, preservando o registo de atividade para efeitos de auditoria (RF1.5).

### 3.1.2. RF2 — Gestão de Lojas

Os requisitos de gestão de lojas foram elicitados junto da gestora central Ana Ribeiro, refletindo a perspetiva de supervisão global da cadeia. O Administrador pode registar novas lojas na cadeia (RF2.1), sendo validada a existência da cadeia antes do registo para garantir integridade

referencial. Qualquer Gestor ou Administrador pode consultar informação detalhada sobre cada loja, incluindo localização, funcionários e inventário (RF2.2).

O cálculo de estatísticas de desempenho por loja — número de vendas e faturação total, filtráveis por período (RF2.3) — foi classificado como prioridade alta, sendo essencial para o acompanhamento operacional e para a tomada de decisão ao nível da gestão central. A validação da existência da loja antes do cálculo previne erros silenciosos em consultas sobre lojas inexistentes.

### **3.1.3. RF3 — Gestão de Produtos**

A gestão de produtos abrange o ciclo de manutenção do catálogo de cada loja. Cada produto é registado com identificador único, nome, categoria, preço e fornecedor associado (RF3.1), sendo validada a existência dos fornecedores referenciados antes do registo. Os atributos podem ser editados a qualquer momento por um Gestor ou Administrador (RF3.2), e a associação entre produtos e fornecedores (RF3.3) suporta os processos de reposição e encomenda.

A consulta e filtragem por categoria (RF3.4) responde à necessidade de navegação eficiente no catálogo durante o atendimento. A identificação por código de barras EAN/UPC (RF3.5), apontada em entrevista pelo caixa João Ferreira como essencial, permite agilizar o processo de venda e reduzir a probabilidade de erro manual.

### **3.1.4. RF4 — Gestão de Inventário e *Stock***

O controlo de inventário e *stock* foi apontado pelos colaboradores como uma das funcionalidades mais críticas do sistema. O inventário é mantido individualmente por loja (RF4.1), operando de forma autónoma sem dependência do sistema central — uma propriedade estrutural que garante continuidade das operações mesmo sem conectividade. A quantidade disponível de cada produto é registada por loja (RF4.2), com a restrição explícita de que o sistema nunca deve permitir valores de *stock* negativos.

Após cada venda, o *stock* é atualizado automaticamente (RF4.3), sendo recusado o registo caso o *stock* seja insuficiente para as quantidades solicitadas. O Funcionário pode registar entradas de *stock* provenientes de fornecedores (RF4.4), com validação da existência do fornecedor referenciado, e o histórico completo de movimentações é mantido pelo sistema (RF4.5), cobrindo entradas, saídas por venda, ajustes manuais e devoluções.

A gestora Maria Santos identificou a necessidade de alertas automáticos quando o *stock* atinge o limiar mínimo configurado por categoria (RF4.6), limiar esse definível pelo Gestor ou Administrador (RF4.7). Esta funcionalidade permite antecipar ruturas e acionar atempadamente o processo de reposição.

### **3.1.5. RF5 — Gestão de Vendas e Faturação**

O módulo de vendas e faturação constitui o núcleo operacional do sistema. Cada venda é registada pelo Funcionário e associada automaticamente à sua loja (RF5.1), com validação prévia da existência e disponibilidade de *stock* de cada produto — impedindo o registo de vendas com *stock* insuficiente. A venda pode conter múltiplas linhas correspondendo a diferentes produtos e quantidades (RF5.2), e a associação de um cliente é opcional (RF5.3), sendo relevante para emissão de fatura nominal. O valor total é calculado automaticamente (RF5.4) e a data e hora da transação são registadas no momento do registo (RF5.5).

A emissão de fatura em formato imprimível e eletrónico (RF5.6) responde a um requisito de conformidade legal. O registo de devoluções (RF5.7), parciais ou totais, com reversão automática do *stock* das quantidades devolvidas, completa o ciclo transacional suportado. A política de

devoluções — nomeadamente o prazo de 7 dias e a exigência de comprovativo — constitui uma regra de negócio definida operacionalmente e não uma restrição técnica do sistema.

### **3.1.6. RF6 — Estatísticas e Relatórios**

Este módulo responde à necessidade de monitorização do desempenho operacional identificada nas entrevistas com Ana Ribeiro. O sistema calcula o número de vendas por loja filtrável por período (RF6.1) e a faturação total filtrável por período e categoria de produto (RF6.2). O Administrador pode ainda consultar estatísticas agregadas ao nível da cadeia, por loja, período e categoria (RF6.3), funcionalidade considerada estratégica para comparar o desempenho entre lojas e identificar tendências de consumo.

### **3.1.7. RF7 — Gestão de Fornecedores**

O módulo de fornecedores suporta o relacionamento com as entidades responsáveis pelo abastecimento de produtos. O sistema regista os dados de identificação e contacto de cada fornecedor — nome, NIF, morada, contacto telefónico e endereço eletrónico (RF7.1) — e permite a sua associação a lojas específicas (RF7.3), com validação da existência de ambos antes da associação. A consulta do histórico de fornecimentos por produto, incluindo fornecedor, data e quantidade (RF7.2), permite analisar padrões de abastecimento e apoiar negociações com base em dados objetivos.

### **3.1.8. RF8 — Gestão de Clientes**

A gestão de clientes foi identificada como necessária para suportar a associação facultativa prevista em RF5.3. O sistema suporta dois modos de identificação de cliente: uma identificação mínima por NIF, suficiente para emissão de fatura rápida no ponto de venda sem interromper o fluxo de atendimento, e um perfil completo com nome, morada, NIF, número de telemóvel e endereço eletrónico, destinado a clientes fidelizados (RF8.1). Em ambos os casos o NIF funciona como identificador único, impedindo duplicados. O Funcionário pode posteriormente promover uma identificação mínima a perfil completo, preenchendo os dados em falta em momento posterior à venda (RF8.2). A edição dos dados de um cliente já registado (RF8.3) garante a manutenção da informação atualizada, assegurando a conformidade com os requisitos de faturação nominal previstos na legislação fiscal portuguesa. A consulta e pesquisa de clientes por NIF ou nome (RF8.4) permite ao Funcionário verificar rapidamente se um cliente já se encontra registado antes de iniciar um novo registo, evitando duplicações.

### **3.1.9. RF9 — Consolidação de Dados**

A consolidação diária de dados foi identificada como um requisito obrigatório nas atas de reunião com a Direção Geral e nas entrevistas com Ana Ribeiro. O sistema deve executar automaticamente, no fecho de cada dia, a sincronização dos dados operacionais de cada loja para o sistema central (RF9.1), sem necessidade de intervenção manual, registando o estado de cada consolidação — PENDENTE, EM\_CURSO, CONCLUIDO ou ERRO — para garantir rastreabilidade completa do processo.

Para assegurar a robustez do mecanismo de sincronização, o sistema deve garantir idempotência no processamento dos eventos enviados pelas lojas (RF9.2): cada envio inclui um identificador único (*hash*) que permite ao sistema central detetar e rejeitar duplicados, possibilitando o reenvio seguro em caso de falha de comunicação sem risco de processamento duplicado. Em caso de falha no processamento de um evento individual, o sistema deve registar o motivo de falha e continuar o processamento dos restantes eventos (RF9.3), preservando os dados para diagnóstico e reprocessamento posterior sem perda de informação histórica. Esta funcionalidade

elimina o processo manual que, segundo a observação direta, consumia entre 30 a 45 minutos por loja e era frequente fonte de erros de transcrição.

## 3.2. Requisitos Não Funcionais

Os requisitos não funcionais definem as propriedades de qualidade que o sistema deve satisfazer, independentemente das funcionalidades que oferece. Tal como os requisitos funcionais, cada requisito não funcional é identificado por um código no formato **RNFx.y**, onde **x** representa a categoria de qualidade e **y** o número sequencial. Os RNF aqui apresentados cobrem cinco dimensões: integridade de dados, desempenho, segurança, consistência na consolidação de informação e disponibilidade operacional.

### 3.2.1. RNF1 — Integridade de Dados

A integridade dos dados é um requisito transversal a todo o sistema. Dado o volume de transações diárias e a diversidade de entidades envolvidas, é essencial garantir que os dados se mantenham consistentes e sem contradições internas — em particular a unicidade dos identificadores e a integridade referencial entre registos relacionados.

| ID     | Descrição                                                                                                             |
|--------|-----------------------------------------------------------------------------------------------------------------------|
| RNF1.1 | O sistema deve garantir unicidade dos identificadores de Produto, Loja, Funcionário e Cliente.                        |
| RNF1.2 | O sistema deve assegurar integridade referencial entre Venda, Loja, Funcionário e Produto, impedindo registos órfãos. |
| RNF1.3 | O sistema não deve permitir stock negativo em nenhuma circunstância, incluindo durante operações concorrentes.        |

### 3.2.2. RNF2 — Performance

O desempenho é determinante para a eficiência operacional das lojas, em especial no registo de vendas. A observação direta revelou que o processo manual consome em média dois a três minutos por transação — o sistema deve reduzir significativamente este tempo. Os requisitos seguintes estabelecem os limites máximos de resposta para as operações mais críticas, sendo o RNF2.1 derivado do objetivo de processar cada venda em menos de 30 segundos, conforme identificado nas entrevistas com João Ferreira, assumindo até 15 interações por transação.

| ID     | Descrição                                                                                                                                                                                                                                                                                                     |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF2.1 | O processamento do registo de uma Venda pelo sistema deve ser concluído em menos de 2 segundos após confirmação pelo Funcionário. Este limite deriva do objetivo de processar cada venda em menos de 30 segundos (identificado nas entrevistas com João Ferreira), assumindo até 15 interações por transação. |
| RNF2.2 | A consulta e geração de relatórios de estatísticas deve ser realizada em tempo inferior a 5 segundos. Este valor constitui uma estimativa conservadora consistente                                                                                                                                            |

|  |                                                                                                          |
|--|----------------------------------------------------------------------------------------------------------|
|  | com benchmarks informais da indústria para operações de leitura analítica em sistemas de ponto de venda. |
|--|----------------------------------------------------------------------------------------------------------|

### 3.2.3. RNF3 — Segurança

A segurança é particularmente relevante dado o caráter sensível da informação gerida — dados pessoais de clientes, transações financeiras e credenciais de acesso. O sistema opera num contexto multi-utilizador com perfis diferenciados, pelo que a autenticação e o controlo de acessos devem ser aplicados de forma consistente em todas as operações.

| ID            | Descrição                                                                                                                                          |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF3.1</b> | As palavras-passe devem ser armazenadas de forma cifrada (BCrypt), não sendo armazenadas em texto simples.                                         |
| <b>RNF3.2</b> | Apenas utilizadores autenticados e ativos podem aceder ao sistema; utilizadores desativados devem ser recusados mesmo com credenciais válidas.     |
| <b>RNF3.3</b> | O sistema deve implementar controlo de acessos baseado em perfis, impedindo o acesso a funcionalidades não autorizadas para o cargo do utilizador. |

### 3.2.4. RNF4 — Consistência e Consolidação

A consolidação diária entre as lojas e o sistema central é um processo crítico. As entrevistas e atas de reunião revelaram que, no modelo atual, esta operação é feita manualmente e consome entre 30 a 45 minutos por loja, sendo frequente fonte de erros e perda de informação. O sistema deve automatizar este processo, assegurando que decorre sem inconsistências, sem perda de dados históricos e com rastreabilidade completa do estado de cada sincronização.

| ID            | Descrição                                                                                                                                                                                           |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF4.1</b> | O sistema deve garantir consistência dos dados durante e após a consolidação diária, sem duplicação ou perda de registos, assegurada por mecanismo de idempotência baseado em <i>hash</i> de envio. |
| <b>RNF4.2</b> | A consolidação de dados não deve causar perda de informação histórica de nenhuma loja, mesmo em caso de falha parcial no processamento de eventos.                                                  |

### 3.2.5. RNF5 — Disponibilidade Operacional

A autonomia das lojas face ao sistema central é uma propriedade arquitetural fundamental do sistema, diretamente derivada do requisito de operação descentralizada. Dado que cada loja funciona com a sua própria instância local, o sistema deve garantir que a indisponibilidade do sistema central não compromete as operações do dia-a-dia das lojas.

| ID | Descrição |
|----|-----------|
|----|-----------|

|               |                                                                                                                                                                                                            |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RNF5.1</b> | O sistema local de cada Loja deve operar de forma completamente autónoma, sem dependência de conectividade com o sistema central para o registo de Vendas, gestão de Stock e autenticação de Funcionários. |
| <b>RNF5.2</b> | Em caso de falha na comunicação com o sistema central durante a consolidação, o sistema local deve preservar os dados pendentes e retentar o envio na janela seguinte, sem perda de informação.            |





## Avaliação de Resposta LLM

### PROMPT

Com base nos requisitos funcionais e no contexto de um sistema distribuído para gestão de uma cadeia de lojas de conveniência, ajuda-me a identificar as principais entidades do domínio, respetivas relações e cardinalidades, construindo uma proposta de modelo conceptual UML coerente com os requisitos apresentados.

### CONTEXTO

*Requisitos Funcionais RF1 a RF9 e descrição do contexto da cadeia BelaVista*

### MODELO

*GPT-5.3 (OpenAI)*

### AVALIAÇÃO CRÍTICA

Os LLM foram utilizados como apoio na identificação inicial das entidades do domínio e respetivas associações, ajudando principalmente na validação de cardinalidades e na separação entre conceitos operacionais e analíticos. As primeiras respostas geradas incluíam algumas entidades demasiado técnicas para um modelo conceptual, ou que simplesmente não nos faziam muito sentido, pelo que foi necessário iterar os prompts para reforçar o foco num modelo de domínio mais abstrato e independente da implementação. O resultado final serviu como base de apoio à construção manual do diagrama apresentado.

## Avaliação de Resposta LLM

### PROMPT

Tendo em conta a necessidade de funcionamento autónomo das lojas e consolidação central de dados, sugere melhorias e valida relações do modelo conceptual, identificando possíveis inconsistências ou redundâncias entre entidades como Loja, Inventário, Estatísticas, Venda e Produto.

### CONTEXTO

*Versão preliminar do modelo conceptual UML desenvolvido pela equipa*

### MODELO

*Claude Sonnet 4.6 (Anthropic)*

### AVALIAÇÃO CRÍTICA

Nesta fase os modelos foram particularmente úteis na revisão estrutural do diagrama, ajudando a identificar redundâncias, relações mal definidas e possíveis simplificações do domínio. Algumas sugestões foram rejeitadas por aproximarem excessivamente o modelo conceptual de uma perspetiva técnica ou de implementação, mas várias observações contribuíram para melhorar a clareza semântica e a coerência global do modelo.

## 5. Diagrama de Casos de Uso

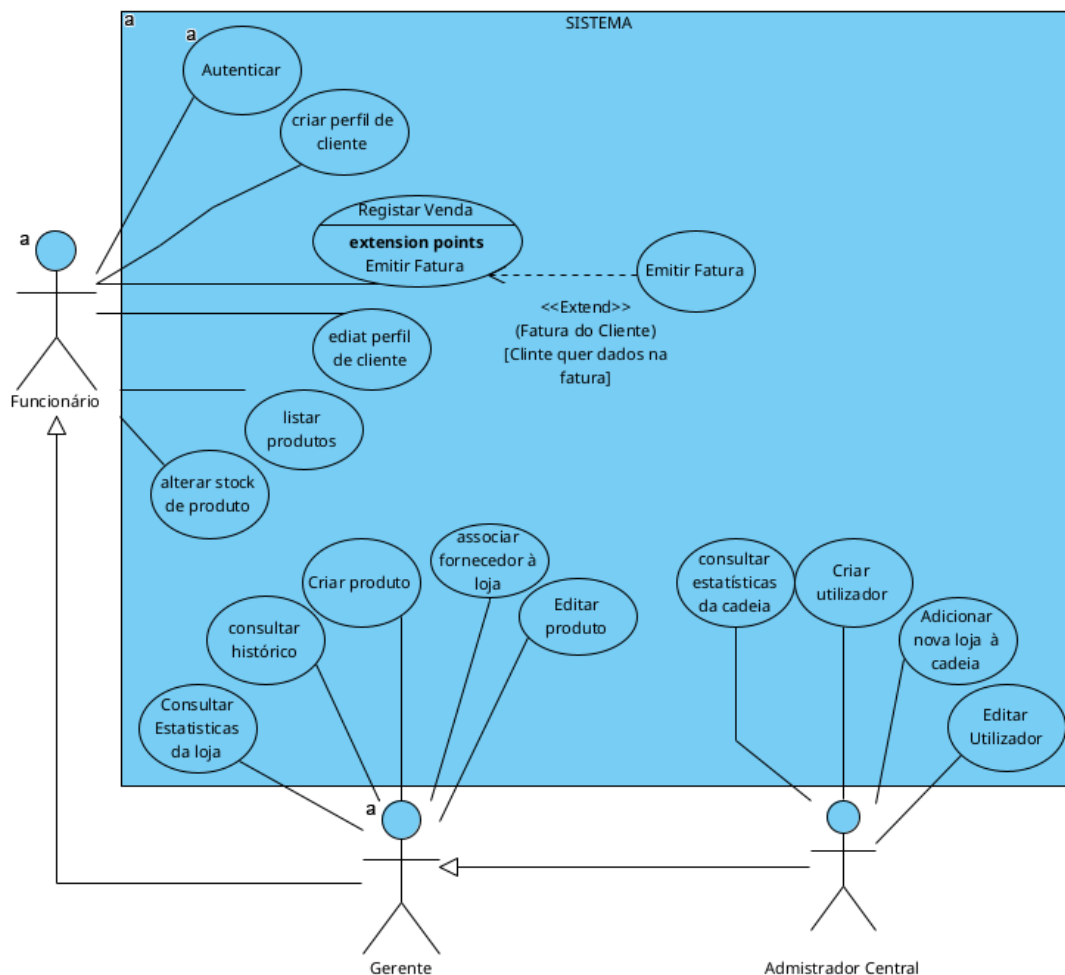


Figura 8: Diagrama de Use Cases

O diagrama de casos de uso apresentado ilustra as interações entre os três atores do sistema — **Funcionário**, **Gerente** e **Administrador Central** — e as funcionalidades disponíveis. O **Funcionário** é o ator base, tendo acesso às operações do dia-a-dia da loja, nomeadamente autenticar-se no sistema, registar vendas, criar e editar perfis de clientes, listar produtos e alterar o stock de produtos. O **Gerente** herda todas as permissões do Funcionário e dispõe adicionalmente de capacidades de gestão, como criar e editar produtos, consultar históricos de fornecimento, consultar estatísticas da loja e associar fornecedores à loja. O **Administrador Central** herda as permissões do Gerente e possui ainda acesso exclusivo às funcionalidades de maior âmbito organizacional, designadamente consultar estatísticas da cadeia, criar e editar utilizadores, e adicionar novas lojas à cadeia. De notar ainda a relação de *extend* entre o caso de uso **Registrar Venda** e **Emitir Fatura**, que ocorre opcionalmente quando o cliente solicita os seus dados na fatura.

## 5.1. Descrição dos Casos de Uso

De seguida, serão descritos individualmente cada um dos casos de uso identificados no diagrama anterior.

### 5.1.1. Use Case: Autenticar

|                      |                                                                                                                                                                                                                                                                                                                  |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE CASE             | Autenticar                                                                                                                                                                                                                                                                                                       |
| DESCRIÇÃO            | Funcionário autentica-se no sistema                                                                                                                                                                                                                                                                              |
| CENÁRIOS             | António autentica-se no sistema para registar uma venda                                                                                                                                                                                                                                                          |
| PRÉ-CONDIÇÃO         | Utilizador possui conta registada                                                                                                                                                                                                                                                                                |
| PÓS-CONDIÇÃO         | Utilizador autenticado no sistema                                                                                                                                                                                                                                                                                |
| FLUXO NORMAL         | <ol style="list-style-type: none"><li>1. Utilizador acede ao sistema</li><li>2. Sistema apresenta formulário de <i>login</i></li><li>3. Utilizador introduz <i>username</i> e <i>password</i></li><li>4. Sistema valida credenciais</li><li>5. Sistema autentica utilizador e apresenta menu principal</li></ol> |
| FLUXO DE EXCEÇÃO (1) | <p>*[Credenciais inválidas] (passo 4)*</p> <ol style="list-style-type: none"><li>4.1. Sistema deteta credenciais inválidas</li><li>4.2. Sistema apresenta mensagem de erro ao utilizador</li><li>4.3. Utilizador pode tentar autenticar-se novamente</li></ol>                                                   |

Tabela 23: Use Case: Autenticar

### 5.1.2. Use Case: Registar venda

|                       |                                                                                                                                                                                                                                                                                                                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE CASE              | Registar venda                                                                                                                                                                                                                                                                                                                                                                      |
| DESCRIÇÃO             | Funcionário                                                                                                                                                                                                                                                                                                                                                                         |
| CENÁRIOS              | António atende um cliente                                                                                                                                                                                                                                                                                                                                                           |
| PRÉ-CONDIÇÃO          | Funcionário autenticado e cliente chega ao balcão com produtos para comprar                                                                                                                                                                                                                                                                                                         |
| PÓS-CONDIÇÃO          | Venda registada no sistema                                                                                                                                                                                                                                                                                                                                                          |
| FLUXO NORMAL          | <ol style="list-style-type: none"><li>1. Funcionário seleciona 'Registar venda'</li><li>2. Sistema cria novo registo de venda</li><li>3. Funcionário adiciona produtos</li><li>4. Sistema calcula total da venda</li><li>5. Funcionário seleciona forma de pagamento e confirma pagamento (emite fatura do cliente - UC3, opcionalmente)</li><li>6. Sistema regista venda</li></ol> |
| FLUXO ALTERNATIVO (1) | <p>*[Cliente já não quer um/vários produto(s)] (passo 3)*</p> <ol style="list-style-type: none"><li>3.1. Funcionário remove produto(s) a mais</li><li>3.2. Regressa a 4</li></ol>                                                                                                                                                                                                   |
| FLUXO DE EXCEÇÃO (2)  | <p>*[Falha de ligação à base de dados] (passo 1)*</p> <ol style="list-style-type: none"><li>1.1. Sistema apresenta erro e termina operação</li></ol>                                                                                                                                                                                                                                |

Tabela 24: Use Case: Registar venda

### 5.1.3. Use Case: Emitir fatura do cliente

|                              |                                                                                                                                                                                                                                                         |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>              | Emitir fatura do cliente                                                                                                                                                                                                                                |
| <b>DESCRIÇÃO</b>             | Funcionário insere os dados do cliente para emitir a fatura do cliente                                                                                                                                                                                  |
| <b>CENÁRIOS</b>              | Cliente diz que quer a sua cópia da fatura                                                                                                                                                                                                              |
| <b>PRÉ-CONDIÇÃO</b>          | Venda paga com sucesso e funcionário autenticado                                                                                                                                                                                                        |
| <b>PÓS-CONDIÇÃO</b>          | Fatura gerada e entregue ao cliente                                                                                                                                                                                                                     |
| <b>FLUXO NORMAL</b>          | <ol style="list-style-type: none"> <li>1. Sistema pede nome, morada, NIF e telemóvel do cliente.</li> <li>2. Funcionário submete os dados necessários</li> <li>3. Sistema imprime fatura</li> </ol>                                                     |
| <b>FLUXO ALTERNATIVO (1)</b> | <p>*[Cliente já tem perfil] (passo 2)*</p> <ol style="list-style-type: none"> <li>3.1. Funcionário insere identificador de cliente</li> <li>3.2. Sistema preenche os resto dos dados conforme o perfil do cliente</li> <li>3.3. Regressa a 3</li> </ol> |
| <b>FLUXO ALTERNATIVO (2)</b> | <p>*[Cliente quer fatura eletrónica] (passo 3)*</p> <ol style="list-style-type: none"> <li>3.1. Funcionário insere endereço eletrónico do cliente</li> <li>3.2. Sistema envia fatura eletrónica para o cliente</li> </ol>                               |

Tabela 25: Use Case: Emitir fatura do cliente

### 5.1.4. Use Case: Adicionar nova loja à cadeia

|                             |                                                                                                                                                                                                                                                     |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>             | Adicionar nova loja à cadeia                                                                                                                                                                                                                        |
| <b>DESCRIÇÃO</b>            | Administrador adiciona uma nova loja à cadeia de lojas de conveniência                                                                                                                                                                              |
| <b>CENÁRIOS</b>             | Eduardo quer registar no sistema a nova loja que vai abrir                                                                                                                                                                                          |
| <b>PRÉ-CONDIÇÃO</b>         | Administrador autenticado                                                                                                                                                                                                                           |
| <b>PÓS-CONDIÇÃO</b>         | Nova loja registada                                                                                                                                                                                                                                 |
| <b>FLUXO NORMAL</b>         | <ol style="list-style-type: none"> <li>1. Administrador seleciona “Adicionar loja”</li> <li>2. Sistema apresenta formulário</li> <li>3. Administrador insere dados</li> <li>4. Sistema valida dados</li> <li>5. Sistema guarda nova loja</li> </ol> |
| <b>FLUXO DE EXCEÇÃO (1)</b> | <p>*[Dados inválidos] (passo 4)*</p> <ol style="list-style-type: none"> <li>4.1. Sistema informa que dados estão inválidos e fecha o formulário com um erro</li> </ol>                                                                              |

Tabela 26: Use Case: Adicionar nova loja à cadeia

#### 5.1.5. Use Case: Calcular estatísticas da loja

|                               |                                                                                                                                                                                                                                                                 |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>               | Calcular estatísticas da loja                                                                                                                                                                                                                                   |
| <b>DESCRIÇÃO</b>              | Utilizador acede às estatísticas da loja onde trabalha                                                                                                                                                                                                          |
| <b>CENÁRIOS</b>               | André quer saber quantos pacotes de manteiga foram vendidos na última semana; Eduardo quer saber das vendas semanais de uma loja                                                                                                                                |
| <b>PRÉ-CONDIÇÃO</b>           | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                   |
| <b>PÓS-CONDIÇÃO</b>           | Estatísticas apresentadas                                                                                                                                                                                                                                       |
| <b>FLUXO NORMAL</b>           | <ol style="list-style-type: none"> <li>1. Utilizador escolhe “Estatísticas de loja”</li> <li>2. Sistema apresenta opções de pesquisa</li> <li>3. Utilizador seleciona as opções desejadas</li> <li>4. Sistema calcula métricas e apresenta relatório</li> </ol> |
| <b>FLUXOS DE EXCEÇÃO (1)</b>  | <p>*[Não existem dados para as opções desejadas] (passo 4)*</p> <ol style="list-style-type: none"> <li>4.1. Sistema informa que não há dados para os filtros selecionados</li> </ol>                                                                            |
| <b>FLUXOS ALTERNATIVO (2)</b> | <p>*[Utilizador é um administrador] (passo 1)*</p> <ol style="list-style-type: none"> <li>1.1. Sistema pede para especificar a loja</li> <li>1.2. Administrador seleciona a loja pretendida</li> <li>1.3. Regressa a 2</li> </ol>                               |

Tabela 27: Use Case: Calcular estatísticas da loja

#### 5.1.6. Use Case: Calcular estatísticas da cadeia

|                              |                                                                                                                                                                                                                                                                 |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>              | Calcular estatísticas da cadeia                                                                                                                                                                                                                                 |
| <b>DESCRIÇÃO</b>             | Administrador acede às estatísticas da cadeia de lojas                                                                                                                                                                                                          |
| <b>CENÁRIOS</b>              | Eduardo quer saber o volume de vendas na última semana                                                                                                                                                                                                          |
| <b>PRÉ-CONDIÇÃO</b>          | Administrador autenticado                                                                                                                                                                                                                                       |
| <b>PÓS-CONDIÇÃO</b>          | Estatísticas apresentadas                                                                                                                                                                                                                                       |
| <b>FLUXO NORMAL</b>          | <ol style="list-style-type: none"> <li>1. Administrador escolhe “Estatísticas da cadeia”</li> <li>2. Sistema apresenta opções de pesquisa</li> <li>3. Gestor adiciona as opções desejadas</li> <li>4. Sistema calcula métricas e apresenta relatório</li> </ol> |
| <b>FLUXOS DE EXCEÇÃO (1)</b> | <p>*[Não existem dados para as opções desejadas] (passo 4)*</p> <ol style="list-style-type: none"> <li>4.1. Sistema informa que não há dados para os filtros selecionados</li> </ol>                                                                            |

Tabela 28: Use Case: Calcular estatísticas da cadeia

#### 5.1.7. Use Case: Consultar histórico de fornecimento de produto

|                               |                                                                                                                                                                                                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>               | Consultar histórico de fornecimento de produto                                                                                                                                                                                                               |
| <b>DESCRIÇÃO</b>              | Utilizador verifica o histórico de fornecimento de um produto                                                                                                                                                                                                |
| <b>CENÁRIOS</b>               | André quer ver os fornecimentos de garrafas de água                                                                                                                                                                                                          |
| <b>PRÉ-CONDIÇÃO</b>           | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                |
| <b>PÓS-CONDIÇÃO</b>           | Histórico apresentado                                                                                                                                                                                                                                        |
| <b>FLUXO NORMAL</b>           | <ol style="list-style-type: none"> <li>1. Utilizador seleciona “Histórico de fornecimento”</li> <li>2. Sistema pede para escolher produto</li> <li>3. Utilizador insere produto</li> <li>4. Sistema consulta base de dados e apresenta resultados</li> </ol> |
| <b>FLUXOS ALTERNATIVO (1)</b> | <p>*[Nenhum fornecimento encontrado] (passo 4)*</p> <ol style="list-style-type: none"> <li>4.1. Sistema informa que ainda não houve nenhum fornecimento do produto</li> </ol>                                                                                |
| <b>FLUXOS DE EXCEÇÃO (2)</b>  | <p>*[Produto não existe] (passo 3)*</p> <ol style="list-style-type: none"> <li>3.1. Sistema informa que ainda não o produto não existe</li> </ol>                                                                                                            |

Tabela 29: Use Case: Consultar histórico de fornecimento de produto

#### 5.1.8. Use Case: Registrar entrada de stock de produto

|                             |                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>             | Registrar entrada de stock de produto                                                                                                                                                                                                                                                                                                                                                       |
| <b>DESCRIÇÃO</b>            | Funcionário regista a entrada de um produto                                                                                                                                                                                                                                                                                                                                                 |
| <b>CENÁRIOS</b>             | António regista o fornecimento de 20 packs de água                                                                                                                                                                                                                                                                                                                                          |
| <b>PRÉ-CONDIÇÃO</b>         | Produto existente e funcionário autenticado                                                                                                                                                                                                                                                                                                                                                 |
| <b>PÓS-CONDIÇÃO</b>         | Stock atualizado                                                                                                                                                                                                                                                                                                                                                                            |
| <b>FLUXO NORMAL</b>         | <ol style="list-style-type: none"> <li>1. Funcionário seleciona “Registrar Entrada de Stock”</li> <li>2. Sistema pergunta qual o produto desejado</li> <li>3. Funcionário seleciona o produto</li> <li>4. Sistema pede qual o fornecedor que fez a entrega e a quantidade a adicionar ao produto</li> <li>5. Funcionário fornece a informação</li> <li>6. Sistema atualiza stock</li> </ol> |
| <b>FLUXO DE EXCEÇÃO (1)</b> | <p><b>[Fornecedor não associado ao produto]</b> (passo 4)</p> <ol style="list-style-type: none"> <li>4.1. Sistema informa que o fornecedor indicado não está associado ao produto</li> <li>4.2. Sistema cancela a operação ou solicita novo fornecedor</li> <li>4.3. Funcionário pode voltar ao passo 4</li> </ol>                                                                          |

Tabela 30: Use Case: Registrar entrada de stock de produto

#### 5.1.9. Use Case: Criar produto

|                               |                                                                                                                                                                                                                                                              |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>               | Criar produto                                                                                                                                                                                                                                                |
| <b>DESCRIÇÃO</b>              | Utilizador insere um novo produto no sistema                                                                                                                                                                                                                 |
| <b>CENÁRIOS</b>               | André vai passar a vender um novo tipo de bolachas na sua loja e regista o produto no sistema                                                                                                                                                                |
| <b>PRÉ-CONDIÇÃO</b>           | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                |
| <b>PÓS-CONDIÇÃO</b>           | Novo produto criado                                                                                                                                                                                                                                          |
| <b>FLUXO NORMAL</b>           | <ol style="list-style-type: none"><li>1. Utilizador seleciona “Criar produto”</li><li>2. Sistema pede nome, categoria, preço e fornecedor</li><li>3. Utilizador insere as informações</li><li>4. Sistema regista o produto</li></ol>                         |
| <b>FLUXOS ALTERNATIVO (1)</b> | <p>*[Fornecedor não existente no sistema] (passo 3)*</p> <ol style="list-style-type: none"><li>3.1. Sistema avisa o utilizador que um ou mais fornecedor não existe</li><li>3.2. Utilizador retira o fornecedor inválido</li><li>3.3. Regressa a 4</li></ol> |

Tabela 31: Use Case: Criar produto

#### 5.1.10. Use Case: Editar produto

|                     |                                                                                                                                                                                                                                                                                                                                       |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>     | Editar produto                                                                                                                                                                                                                                                                                                                        |
| <b>DESCRIÇÃO</b>    | Utilizador edita um produto                                                                                                                                                                                                                                                                                                           |
| <b>CENÁRIOS</b>     | André quer atualizar o preço de um produto                                                                                                                                                                                                                                                                                            |
| <b>PRÉ-CONDIÇÃO</b> | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                                                                                         |
| <b>PÓS-CONDIÇÃO</b> | Produto editado                                                                                                                                                                                                                                                                                                                       |
| <b>FLUXO NORMAL</b> | <ol style="list-style-type: none"><li>1. Utilizador seleciona “Editar produto”</li><li>2. Sistema lista os produtos existentes</li><li>3. Utilizador escolhe o produto</li><li>4. Sistema apresenta as informações do produto</li><li>5. Utilizador altera as informações pretendidas</li><li>6. Sistema atualiza o produto</li></ol> |

Tabela 32: Use Case: Editar produto



#### 5.1.11. Use Case: Criar utilizador

|                              |                                                                                                                                                                                                                                                                                                       |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>              | Criar utilizador                                                                                                                                                                                                                                                                                      |
| <b>DESCRIÇÃO</b>             | Utilizador cria um perfil de utilizador                                                                                                                                                                                                                                                               |
| <b>CENÁRIOS</b>              | André cria um perfil para um novo atendente de loja; Eduardo cria um perfil para um novo gestor                                                                                                                                                                                                       |
| <b>PRÉ-CONDIÇÃO</b>          | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                                                         |
| <b>PÓS-CONDIÇÃO</b>          | Novo utilizador registado                                                                                                                                                                                                                                                                             |
| <b>FLUXO NORMAL</b>          | <ol style="list-style-type: none"><li>1. Utilizador seleciona “Criar utilizador”</li><li>2. Sistema pede tipo de perfil e palavra-passe</li><li>3. Utilizador insere as informações</li><li>4. Sistema regista o utilizador e informa o identificador único do mesmo</li></ol>                        |
| <b>FLUXO ALTERNATIVO (1)</b> | <p>*[Utilizador é um gestor] (passo 2)*</p> <ol style="list-style-type: none"><li>2.1. Sistema pede palavra-passe para o novo Funcionário</li><li>2.2. Utilizador insere as informações</li><li>2.3. Sistema regista o utilizador como Funcionário e informa o identificador único do mesmo</li></ol> |

Tabela 33: Use Case: Criar utilizador

#### 5.1.12. Use Case: Editar utilizador

|                     |                                                                                                                                                                                                                                                                                                                                                             |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>     | Editar utilizador                                                                                                                                                                                                                                                                                                                                           |
| <b>DESCRIÇÃO</b>    | Administrador edita um perfil de utilizador                                                                                                                                                                                                                                                                                                                 |
| <b>CENÁRIOS</b>     | Eduardo muda o tipo do perfil de um funcionário que promoveu                                                                                                                                                                                                                                                                                                |
| <b>PRÉ-CONDIÇÃO</b> | Utilizador autenticado e utilizador é gestor ou administrador                                                                                                                                                                                                                                                                                               |
| <b>PÓS-CONDIÇÃO</b> | Perfil de utilizador editado                                                                                                                                                                                                                                                                                                                                |
| <b>FLUXO NORMAL</b> | <ol style="list-style-type: none"><li>1. Administrador seleciona “Editar utilizador”</li><li>2. Sistema lista os utilizadores existentes</li><li>3. Administrador escolhe o utilizador</li><li>4. Sistema apresenta as informações do utilizador</li><li>5. Utilizador altera as informações pretendidas</li><li>6. Sistema atualiza o utilizador</li></ol> |

Tabela 34: Use Case: Editar utilizador

#### 5.1.13. Use Case: Listar produtos

|                              |                                                                                                                                                                                                                     |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>              | Listar produtos                                                                                                                                                                                                     |
| <b>DESCRIÇÃO</b>             | Administrador acede às estatísticas da cadeia de lojas                                                                                                                                                              |
| <b>CENÁRIOS</b>              | António quer saber quais produtos de limpeza diferentes são vendidos                                                                                                                                                |
| <b>PRÉ-CONDIÇÃO</b>          | Utilizador autenticado                                                                                                                                                                                              |
| <b>PÓS-CONDIÇÃO</b>          | Lista de produtos apresentada                                                                                                                                                                                       |
| <b>FLUXO NORMAL</b>          | 1. Utilizador seleciona “Catálogo de produtos”<br>2. Sistema apresenta lista de produtos com as respetivas informações                                                                                              |
| <b>FLUXO ALTERNATIVO (1)</b> | *[Utilizador quer aplicar filtros] (passo2)*<br>2.1 Utilizador seleciona “Filtros” 2.2. Sistema apresenta opções de filtros 2.3 Utilizador seleciona os filtros desejados 2.4. Sistema atualiza a lista apresentada |

Tabela 35: Use Case: Listar produtos

#### 5.1.14. Use Case: Associar Funcionário à loja

|                              |                                                                                                                                                                                                                                          |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>USE CASE</b>              | Associar Funcionário à loja                                                                                                                                                                                                              |
| <b>DESCRIÇÃO</b>             | Utilizador associa um Funcionário a uma loja específica                                                                                                                                                                                  |
| <b>CENÁRIOS</b>              | Eduardo quer associar um novo Funcionário da cadeia a uma loja próxima                                                                                                                                                                   |
| <b>PRÉ-CONDIÇÃO</b>          | Utilizador é gestor ou administrador, está autenticado e Funcionário e loja existem                                                                                                                                                      |
| <b>PÓS-CONDIÇÃO</b>          | Funcionário associado à loja                                                                                                                                                                                                             |
| <b>FLUXO NORMAL</b>          | 1. Utilizador seleciona “Gestão de Funcionário”<br>2. Sistema apresenta lista de Funcionário<br>3. Utilizador seleciona Funcionário<br>4. Sistema apresenta lista de lojas<br>5. Utilizador escolhe loja<br>6. Sistema guarda associação |
| <b>FLUXO ALTERNATIVO (1)</b> | *[Utilizador é um gestor] (passo 4)*<br>4.1. Sistema guarda associação usando a loja associada ao gestor                                                                                                                                 |

Tabela 36: Use Case: Associar Funcionário à loja

#### 5.1.15. Use Case: Criar perfil de cliente

|                            |                                                                                                                                                                                                             |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE CASE                   | Criar perfil de cliente                                                                                                                                                                                     |
| DESCRIÇÃO                  | Utilizador regista um cliente                                                                                                                                                                               |
| CENÁRIOS                   | Cliente frequente pede a António para lhe criar um perfil de cliente para agilizar as faturas                                                                                                               |
| PRÉ-CONDIÇÃO               | Cliente não registado e utilizador autenticado                                                                                                                                                              |
| PÓS-CONDIÇÃO               | Perfil de cliente criado                                                                                                                                                                                    |
| FLUXO NORMAL               | <ol style="list-style-type: none"> <li>1. Funcionário seleciona "Registar cliente"</li> <li>2. Sistema apresenta formulário</li> <li>3. Funcionário insere dados</li> <li>4. Sistema cria perfil</li> </ol> |
| FLUXOS DE ALTER-NATIVO (1) | <p>*[Dados obrigatórios em falta] (passo 3)*</p> <p>4.1. Sistema pede os dados obrigatórios em falta 4.2. Funcionário insere os dados obrigatórios em falta 4.3. Regressa a 4</p>                           |

Tabela 37: Use Case: Criar perfil de cliente

#### 5.1.16. Use Case: Editar perfil de cliente

|              |                                                                                                                                                                                                                                                                                                                                               |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USE CASE     | Editar perfil de cliente                                                                                                                                                                                                                                                                                                                      |
| DESCRIÇÃO    | Funcionário atualiza as informações de um cliente                                                                                                                                                                                                                                                                                             |
| CENÁRIOS     | Cliente frequente pede a António para atualizar informação no perfil de cliente                                                                                                                                                                                                                                                               |
| PRÉ-CONDIÇÃO | Cliente registado e funcionário autenticado                                                                                                                                                                                                                                                                                                   |
| PÓS-CONDIÇÃO | Perfil de cliente atualizado                                                                                                                                                                                                                                                                                                                  |
| FLUXO NORMAL | <ol style="list-style-type: none"> <li>1. Funcionário seleciona "Editar cliente"</li> <li>2. Sistema apresenta lista de clientes</li> <li>3. Funcionário seleciona cliente</li> <li>4. Sistema apresenta as informações do cliente</li> <li>5. Utilizador altera as informações pretendidas</li> <li>6. Sistema atualiza o cliente</li> </ol> |

Tabela 38: Use Case: Editar perfil de cliente

### Avaliação de Resposta LLM

#### PROMPT

Tendo em conta os requisitos levantados para implementar um "Sistema de gestão integrada para uma cadeia de pequenas lojas de conveniência" e os possíveis Use Cases identificados faz um ficheiro onde elabores os passos de cada Use Case. Valida e dá opinião sobre qualquer UC que aches que falta.

#### CONTEXTO

*Requisitos levantados e uma lista de Use Cases identificados de forma direta*

**MODELO**

*GPT-5.3 (OpenAI)*

**AVALIAÇÃO  
CRÍTICA**

A primeira resposta a esta instrução continha um ficheiro perto do que era pretendido, porém realçou a falta de atenção ao não mencionar que queríamos seguir a especificação UML, após confirmar que era essa a intenção gerou um novo ficheiro muito próximo do que era pretendido, mas foi preciso fazer algumas alterações manuais em termos de contexto e fluxos alternativos/exceção. No geral foi útil para o primeiro brainstorming de ideias de possíveis Use Cases.

## 6. Arquitetura Geral

A figura nesta secção apresenta a arquitetura geral do sistema de gestão integrada da Cadeia BelaVista, organizada em três camadas horizontais: o nível do cliente/loja, o sistema local de cada loja (*Sistema Local/Loja*) e o sistema central da cadeia (*Sistema Central/Cadeia*). Esta estrutura reflete uma decisão arquitetural fundamental — a adoção de um **modelo híbrido distribuído com consolidação diária** — em detrimento de alternativas centralizadas ou de microsserviços.

Cada loja disponibiliza aos funcionários uma interface local própria, através da qual são executadas as operações operacionais do dia-a-dia, como o registo de vendas, atualização de inventário, reposição de stock, gestão de produtos e emissão de faturas. Estas operações são processadas localmente pela respetiva lógica de negócio da loja e persistidas numa base de dados independente, permitindo que cada unidade continue a funcionar autonomamente mesmo em situações de indisponibilidade de rede ou falhas de comunicação com o sistema central.

A arquitetura adotada segue uma abordagem distribuída baseada em separação clara de responsabilidades entre os sistemas locais e o sistema central. Enquanto a lógica local se concentra nas operações transacionais e no suporte às atividades operacionais da loja, o sistema central assume funções de supervisão, consolidação e análise agregada da informação. Esta separação reduz a dependência contínua de conectividade entre lojas e minimiza o impacto de falhas isoladas sobre o funcionamento global da cadeia.

O mecanismo de Consolidação Diária desempenha um papel central nesta arquitetura. No fecho de cada dia, os eventos e dados operacionais registados localmente são exportados para o sistema central, onde são processados por uma lógica orientada a eventos (*event-driven*). Esta abordagem permite transformar automaticamente os dados recebidos em estatísticas, relatórios e indicadores globais da cadeia, assegurando simultaneamente a preservação do histórico e a consistência dos dados consolidados.

A utilização de bases de dados independentes por loja contribui ainda para melhorar a escalabilidade do sistema, uma vez que novas lojas podem ser adicionadas sem impacto direto sobre as restantes instâncias locais. Paralelamente, a existência de uma base de dados central possibilita a obtenção de uma visão global do negócio, permitindo comparar desempenho entre lojas, acompanhar tendências de vendas e apoiar processos de tomada de decisão estratégica.

Do ponto de vista arquitetural, esta solução aproxima-se de um modelo híbrido entre sistemas distribuídos clássicos e arquiteturas orientadas a eventos, privilegiando simplicidade operacional, tolerância a falhas e independência das lojas, sem abdicar da necessidade de análise centralizada e consolidação periódica da informação.

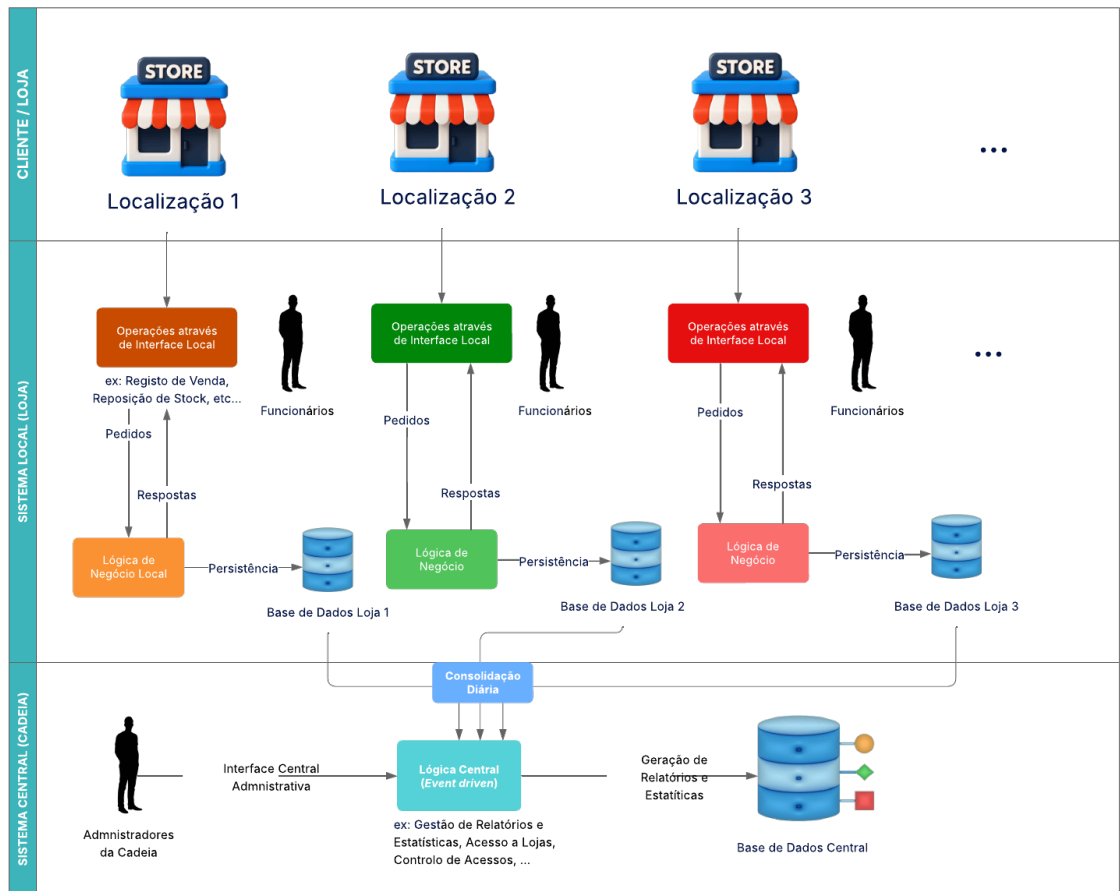


Figura 9: Diagrama que esquematiza a Arquitetura de todo o Sistema BelaVista

### Avaliação de Resposta LLM

#### PROMPT

Tendo em conta os requisitos funcionais e não funcionais do sistema Bela-Vista, quais são as minhas opções para uma arquitetura geral adequada para um sistema de gestão de lojas com funcionamento autónomo local e consolidação central periódica.

#### CONTEXTO

*Requisitos RF1-RF9, RNF1-RNF4 e descrição do contexto da cadeia de lojas*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AValiação CRÍTICA

O principal contributo da IA neste caso foram as diferentes alternativas arquiteturais, incluindo soluções centralizadas, cliente-servidor clássico e arquiteturas distribuídas com sincronização periódica. Ainda assim por vezes a melhor decisão era ambígua entre várias conversas e vários modelos, e foi aí que entrou todo o trabalho que tivemos de modelação: decidir qual das

opções fazia mais sentido afinal. As respostas ajudaram principalmente na comparação entre vantagens e limitações de cada abordagem, permitindo justificar a escolha que fizemos. Algumas propostas iniciais incluíam complexidade excessiva para o contexto acadêmico do projeto, nomeadamente sugestões baseadas em microsserviços completos e filas de mensagens distribuídas, tendo sido necessário refinar os prompts para privilegiar simplicidade e adequação ao domínio do problema.

### Avaliação de Resposta LLM

#### PROMPT

Com base numa arquitetura com bases de dados independentes por loja e sincronização diária para um sistema central, identifica possíveis problemas de consistência, tolerância a falhas e sincronização de dados, sugerindo mecanismos para mitigar esses riscos.

#### CONTEXTO

*Descrição da arquitetura geral e requisito RF9.1 relativo à consolidação automática*

#### MODELO

*Claude Sonnet 4.6 (Anthropic)*

#### AVALIAÇÃO CRÍTICA

Os LLM foram particularmente úteis na identificação de potenciais problemas relacionados com sincronização, duplicação de registos e falhas de consolidação. As sugestões recebidas ajudaram a reforçar aspetos como utilização de eventos, identificadores únicos, processamento incremental e preservação de histórico. Apesar disso, várias soluções propostas inicialmente eram demasiado avançadas para o âmbito do projeto, sendo necessário adaptar as recomendações a uma implementação mais simples e compatível com os nossos objetivos.

### Avaliação de Resposta LLM

#### PROMPT

Analisa esta arquitetura distribuída para uma cadeia de lojas e valida se a separação entre lógica local e lógica central é coerente com os requisitos de autonomia operacional, geração de estatísticas e consolidação diária.

#### CONTEXTO

*Diagrama da arquitetura geral do sistema BelaVista*

#### MODELO

*GPT-5.3 (OpenAI)*

#### AVALIAÇÃO CRÍTICA

Desta vez os modelos serviram sobretudo como mecanismo de validação conceptual da separação arquitetural adotada. As respostas permitiram con-

firmar que a divisão entre operações locais e processamento centralizado estava alinhada com os requisitos funcionais identificados, especialmente no que toca à independência operacional das lojas e à geração de estatísticas agregadas após consolidação. Por vezes foram ambiguos nas respostas em relação a coisas que mudariam, exigiu muito mais raciocínio do que os modelos ajudarem.

## 6.1. Comparação de Alternativas

A separação adotada não é apenas uma preferência estética — é uma consequência direta dos requisitos do sistema. O enunciado estabelece que cada loja *“atua de forma autónoma”* e que a informação é consolidada *“todos os dias, ao fim do dia”*. Estas duas propriedades — autonomia e consolidação periódica — são incompatíveis com um modelo unificado: autonomia exige que a loja não dependa do central para operar, e consolidação periódica implica que os dois contextos têm ciclos de vida distintos.

Por modelo unificado referimo-nos ao de juntar tudo apenas numa Lógica de Negócio, misturando os conceitos de Cadeia e Lógica num meio de convivência. Foi a primeira alternativa que consideramos, começando até inicialmente a produzir o diagrama de classes desta forma unificada (como vamos mencionar em secções posteriores). A autonomia teria de ser simulada através de mecanismos de *cache* ou réplica que teriam de ser geridos manualmente, sem fronteiras claras no próprio modelo. Mudamos desta ideia para a separação em duas lógicas em conversa entre os membros do grupo e também com os docentes da presente UC.

A separação adotada torna a autonomia uma propriedade estrutural do sistema: o *backend* local não referencia nenhuma classe do *backend* central, e a única dependência entre os dois é uma Payload de Sincronização — um contrato de dados minimalista trocado uma vez por dia.

O custo desta separação é real: dois *backends* para desenvolver e manter, uma camada de sincronização que tem de ser robusta a falhas de rede, e a necessidade de gerir consistência eventual entre os dados locais e os dados consolidados. Estes custos são aceitáveis e previstos no desenho — o SubGestSync e o SubGestConsolidacao existem precisamente para os encapsular e torná-los invisíveis para os restantes subsistemas. O que não seria aceitável seria pagar estes custos de forma implícita num modelo unificado, sem fronteiras claras e sem um contrato explícito entre os dois contextos.



| Dimensão                              | Modelo unificado                                                                                                                            | Modelos separados (adotado)                                                                                                                                        |
|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Separação de responsabilidades</b> | <b>Violada</b> — operações locais e centrais partilham as mesmas classes e interfaces                                                       | <b>Respeitada</b> — cada modelo tem um único contexto e uma única razão para mudar                                                                                 |
| <b>Acoplamento (Dependências)</b>     | <b>Elevado</b> — uma alteração num subsistema local pode forçar revisão de classes centrais                                                 | <b>Baixo</b> — os dois modelos evoluem independentemente; o contrato de consolidação é o único ponto de contacto                                                   |
| <b>Autonomia operacional</b>          | <b>Comprometida</b> — a loja depende estruturalmente do modelo central para operar                                                          | <b>Garantida</b> — o backend local funciona sem qualquer dependência do central                                                                                    |
| <b>Controlo de acessos</b>            | <b>Complexo</b> — a mesma interface expõe operações de funcionário e de administrador, exigindo mais condições em cada método               | <b>Simples e explícito</b> — <code>ILocalLN</code> e <code>ICentralLN</code> segregam os contratos por perfil; um funcionário nunca vê <code>gerarRelatorio</code> |
| <b>Legibilidade do modelo</b>         | Degradada com o crescimento — um diagrama que mistura <code>LinhaVenda</code> com <code>RelatorioConsolidado</code> é difícil de raciocinar | Cada diagrama é coeso e auto-contido; o leitor compreende o contexto sem analisar o sistema completo                                                               |
| <b>Complexidade inicial</b>           | <b>Menor</b> — um único projeto, uma única BD, uma única fachada, o acesso à informação dos Relatórios é direta                             | <b>Maior</b> — dois <i>backends</i> , duas BDs, mecanismo de <i>sync</i> explícito                                                                                 |
| <b>Escalabilidade</b>                 | <b>Limitada</b> — adicionar uma loja implica rever o modelo global                                                                          | <b>Natural</b> — adicionar uma loja é fazer <i>deploy</i> de uma nova instância local sem alterar o central                                                        |
| <b>Testabilidade</b>                  | Difícil — ex: testar uma funcionalidade pode exigir mais <i>overhead</i> de preparação de dados devido às dependências entre subsistemas    | Direta — cada subsistema é testável de forma isolada com as suas próprias dependências                                                                             |

Tabela 39: Comparação entre modelo unificado e modelos separados por contexto

## 7. Modelação Arquitetural

O sistema de gestão da Cadeia BelaVista serve dois contextos operacionais com naturezas fundamentalmente distintas: **a loja**, onde ocorrem as transações do dia-a-dia em regime autónomo, e **a cadeia**, onde se agrega, analisa e governa a informação de todas as lojas. Como entendemos que esta era uma das decisões mais importantes da modelação foi esta a decisão que discutimos mais entre nós e com os docentes para ter a certeza que terminávamos com a melhor arquitetura que nos era possível. A grande questão que se coloca é se estes dois contextos devem ser modelados num único modelo unificado ou em dois modelos separados com fronteiras explícitas.

A opção mais imediata (a que chegamos a modelar inicialmente) seria a de um modelo único — uma única hierarquia de classes, uma única fachada, e uma única camada de dados que servisse simultaneamente as operações locais e as operações centrais. Esta abordagem tem a aparente vantagem da simplicidade: menos ficheiros, menos interfaces, e uma visão global do sistema num só diagrama. No entanto, esta simplicidade é ilusória. Um modelo único que contenha `registarVenda` e `gerarRelatorioConsolidado` na mesma interface, ou que coloque `LinhaInventario` e `EstatisticasLoja` no mesmo contexto de persistência, viola diretamente o **Single Responsibility Principle (SRP)**: cada módulo deve ter uma única razão para mudar. Num modelo unificado, uma alteração aos requisitos de faturação local obrigaria a rever o mesmo modelo que gere a consolidação central — dois domínios com ritmos de mudança, consumidores e ciclos de vida completamente diferentes ficam artificialmente acoplados. Além disso, a mesma interface teria a função de servir ambos os Utilizadores da Cadeia (Administradores) e os Utilizadores locais das Lojas, e isso exigiria um maior esforço para isolarmos a lógica dentro da Interface unificada.

A separação adotada — `LógicaLocal` e `LógicaCentral` como modelos independentes, com a `LojaFacadeLN` e a `CadeiaFacadeLN` como pontos de entrada distintos — resolve este acoplamento de forma estrutural. Cada modelo tem uma única razão para existir e uma única razão para mudar. A comunicação entre os dois contextos é reduzida a um único ponto de contacto explícito: o mecanismo de sincronização diária entre o `SubGestSync` local e o `SubGestConsolidacao` central. Se pensarmos na eventualidade, já na fase de desenvolvimento de alterar algum módulo/requisito relativo apenas à lógica do Cliente (estritamente local), não termos de mexer em código unificado e potencialmente dependente da lógica ligada à Cadeia.

### 7.1. Lógica Local (Loja) - Diagrama de Classes

O diagrama seguinte apresenta o modelo de classes da lógica local, instanciada em cada loja da cadeia de forma independente. A divisão em cinco subsistemas — **SubGestSync**, **SubGestFuncionarios**, **SubGestInventario**, **SubGestProdutos** e **SubGestVendas** — reflete o princípio de separação de responsabilidades aplicado ao contexto operacional de uma loja autónoma.

A **LojaFacadeLN** agrega o acesso a todos os subsistemas através de uma única interface, expondo aos seus consumidores apenas as operações relevantes para o contexto local: gestão de funcionários, consulta e atualização de inventário, processamento de vendas, e exportação de dados para sincronização. Esta centralização do ponto de entrada evita acoplamento direto entre os subsistemas e simplifica o controlo de acessos baseado em cargo (RF1.3).

O **SubGestSync** ocupa uma posição periférica deliberada: a **GestSyncFacade** e a entidade **SincronizacaoDiaria** existem exclusivamente para gerir o ciclo de vida da exportação diária, sem interferir nas operações transacionais. A **ConfigLoja**, modelada como *singleton*, fornece o **lojaId** e as credenciais de sincronização sem ser referenciada por qualquer outra entidade via chave estrangeira — o contexto de loja está implícito em todo o *backend*.

A camada **LojaDL** suporta a persistência local através de seis DAOs especializados: **EventosDAO**, **InventarioDAO**, **FuncionariosDAO**, **InventarioDAO**, **ProdutosDAO**, **FornecedoresDAO** e **ClientesDAO**. Cada DAO serve exclusivamente o subsistema que lhe corresponde, garantindo que alterações ao esquema de persistência de um domínio não afetam os restantes.

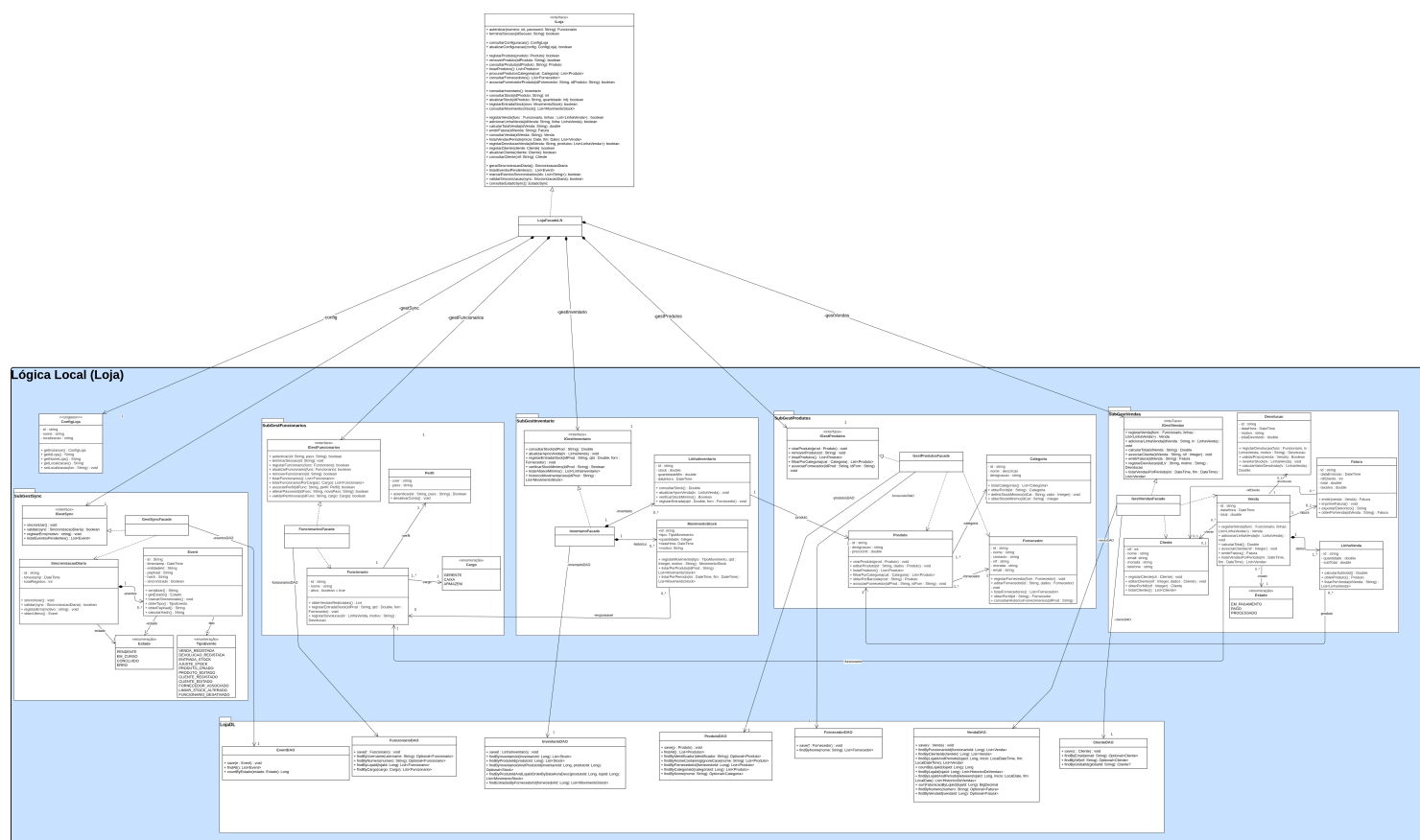


Figura 10: Diagrama de Classes da Lógica de Negócio Local (Loja)

## 7.2. Lógica Central (Cadeia) - Diagrama de Classes

O diagrama seguinte apresenta o modelo de classes da lógica central, que opera ao nível da cadeia e é partilhado por todos os administradores. A divisão em quatro subsistemas — **SubGestLojas**, **SubGestEstadisticas**, **SubGestAuth** e **SubGestConsolidacao** — separa as responsabilidades de gestão operacional, análise de dados, controlo de identidades e consolidação de informação proveniente das lojas.

A **CadeiaFacadeLN** centraliza o acesso à lógica central, expondo uma interface coesa que cobre desde o registo de lojas e fornecedores até à geração de relatórios e importação de sincronizações. A separação desta fachada da **LojaFacadeLN** local é uma decisão explícita de segregação de interfaces: um administrador central não deve/precisa de ter visibilidade sobre operações

transacionais de loja (como registrarVenda ou gerirInventario), e um funcionário local não acede a operações de gestão da cadeia (como gerarRelatorioConsolidado ou gerirLojas).

O **SubGestConsolidacao** é o subsistema de fronteira entre os dois contextos. A GestConsolidacaoFacade recebe os *payloads* enviados pelo SubGestSync de cada loja, valida a integridade via payloadHash para garantir idempotência dos reenvios, e transita a ConsolidacaoDiaria pelos estados PENDENTE → EM\_CURSO → CONCLUIDO (ou ERRO), assegurando rastreabilidade completa de cada sincronização (RNF4.1, RNF4.2). O resultado do processamento alimenta diretamente o SubGestEstatisticas, onde as classes GestEstatisticasFacade, e RelatorioConsolidado produzem as métricas de desempenho por loja, por período e por categoria (RF6.1, RF6.2, RF6.3).

A camada CadeiaDL suporta a persistência central através de quatro DAOs: **LojasDAO**, **EstatisticasDAO**, **UtilizadoresDAO** e **ConsolidacaoDAO**. A granularidade reduzida face à camada local justifica-se pela natureza dos dados centrais — predominantemente agregados e de leitura — em contraste com a elevada frequência de escrita das operações transacionais locais.

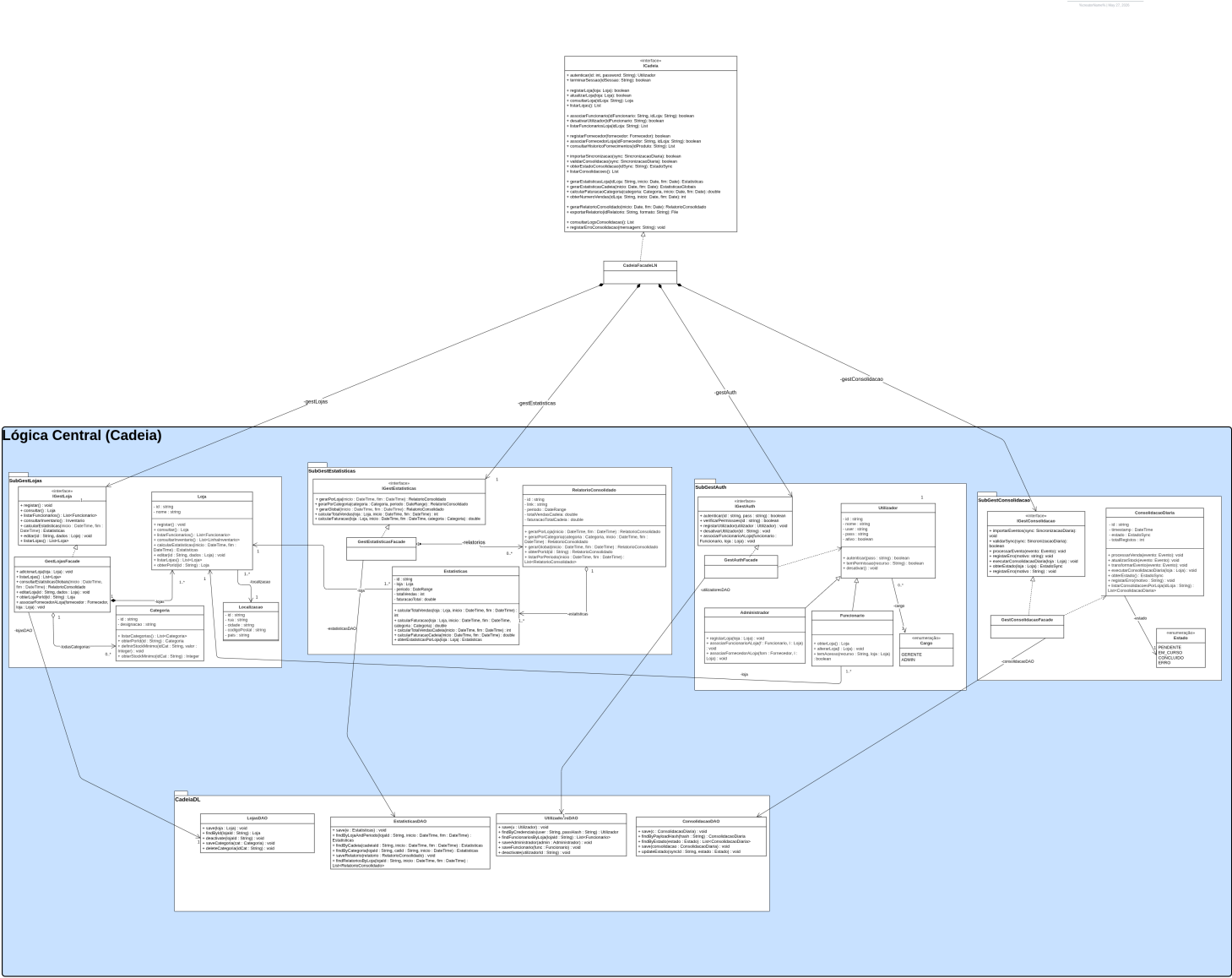


Figura 11: Diagrama de Classes da Lógica de Negócio Central (Cadeia)

### Avaliação de Resposta LLM

#### PROMPT

Com base nos requisitos funcionais já identificados, ajuda-me a dividir o sistema em subsistemas coesos e com responsabilidades bem definidas, justificando possíveis fronteiras arquiteturais entre lógica local e lógica central.

#### CONTEXTO

*Requisitos RF1-RF9, necessidade de funcionamento autónomo das lojas e consolidação diária*

#### MODELO

*Claude Sonnet 4.6 (Anthropic)*

#### AVALIAÇÃO CRÍTICA

Os modelos foram utilizados sobretudo como apoio à decomposição inicial do sistema em módulos funcionais independentes. As respostas ajudaram a perceber quais as responsabilidades que faziam sentido permanecer exclusivamente no contexto local e quais deveriam pertencer à lógica central da cadeia. Algumas sugestões iniciais misturavam demasiado aspetos técnicos com responsabilidades de negócio, pelo que foi necessário refinar iterativamente os prompts para manter uma separação mais limpa entre domínios.

### Avaliação de Resposta LLM

#### PROMPT

Analisa este modelo com duas fachadas principais (LojaFacadeLN e CadeiaFacadeLN) e valida se esta separação respeita princípios de desenho orientado a objetos como SRP, separação de responsabilidades e baixo acoplamento.

#### CONTEXTO

*Estrutura arquitetural proposta para os subsistemas locais e centrais da cadeia BelaVista*

#### MODELO

*Claude Sonnet 4.6 (Anthropic)*

#### AVALIAÇÃO CRÍTICA

Neste caso os LLM foram usados principalmente como ferramenta de validação de princípios arquiteturais e de desenho orientado a objetos. As respostas ajudaram a justificar formalmente decisões que já tinham surgido nas discussões da equipa, especialmente a separação de fachadas e a existência de interfaces distintas para os diferentes contextos do sistema. Algumas respostas eram demasiado teóricas ou genéricas, exigindo adaptação manual para o contexto específico do projeto.

### Avaliação de Resposta LLM

|                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PROMPT            | Sugere uma possível organização de DAOs e camadas de persistência para um sistema distribuído onde cada loja possui uma base de dados independente e o sistema central mantém apenas informação consolidada.                                                                                                                                                                                                                                                 |
| CONTEXTO          | <i>Arquitetura lógica da BelaVista com separação entre LojaDL e CadeiaDL</i>                                                                                                                                                                                                                                                                                                                                                                                 |
| MODELO            | <i>Claude Sonnet 4.6 (Anthropic)</i>                                                                                                                                                                                                                                                                                                                                                                                                                         |
| AVALIAÇÃO CRÍTICA | A utilização dos modelos ajudou sobretudo na validação da granularidade da camada de persistência e na associação entre DAOs e subsistemas específicos. Algumas respostas sugeriam centralizar demasiado a persistência ou reutilizar DAOs entre domínios distintos, o que acabaria por aumentar o acoplamento entre módulos. A discussão destas alternativas acabou por reforçar a decisão de manter DAOs especializados e isolados por contexto funcional. |

|                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Avaliação de Resposta LLM</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| PROMPT                           | Tendo em conta uma arquitetura baseada em sincronização diária, propõe uma forma de modelar o fluxo entre exportação local (SubGestSync) e importação central (SubGestConsolidacao), incluindo estados de sincronização e validação de integridade.                                                                                                                                                                                                                                                   |
| CONTEXTO                         | <i>Necessidade de consolidação diária automática definida em RF9.1 e RNF4</i>                                                                                                                                                                                                                                                                                                                                                                                                                         |
| MODELO                           | <i>Claude Sonnet 4.6 (Anthropic)</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| AVALIAÇÃO CRÍTICA                | Os modelos foram particularmente úteis para estruturar o fluxo conceptual da sincronização entre loja e cadeia, incluindo ideias relacionadas com estados de processamento, validação de payloads e rastreabilidade das sincronizações. Apesar disso, algumas propostas iniciais recorriam a mecanismos demasiado complexos para o âmbito do projeto, tendo sido necessário simplificar as soluções sugeridas para manter coerência com os objetivos académicos e com a arquitetura global escolhida. |

### 7.3. Diagrama de Componentes

O diagrama seguinte apresenta a arquitetura de componentes do sistema, organizada novamente em dois *backends* independentes — CadeiaLN e LojaLN — cada um com a sua própria camada de lógica de negócio e camada de dados. A separação física entre os dois contextos é o reflexo direto das decisões de modelação descritas nas secções anteriores: autonomia operacional por loja, segregação de interfaces por perfil de utilizador, e comunicação reduzida a um único ponto de contacto explícito.

O único ponto de contacto entre os dois *backends* é a ligação assinalada no diagrama entre o SubGestSync (local) e o SubGestConsolidacao (central), realizada via *Socket* / *API REST*. Esta dependência é unidirecional e assíncrona: é sempre a loja que inicia a comunicação, enviando um *payload* serializado no final do dia. O central não mantém qualquer ligação ativa às lojas nem acede diretamente às suas bases de dados.

Esta arquitetura de comunicação garante três propriedades essenciais. Em primeiro lugar, a **resiliência**: uma falha de conectividade não interrompe as operações locais, apenas adia a consolidação para a janela seguinte. Em segundo lugar, a **idempotência**: o *payloadHash* incluído em cada envio permite ao SubGestConsolidacao detectar e rejeitar reenvios duplicados sem reprocessamento. Em terceiro lugar, o **isolamento**: os dois *backends* evoluem de forma independente — alterar o esquema de consolidação central não afeta o contrato de exportação local, desde que o formato do *PayloadSync* se mantenha estável.

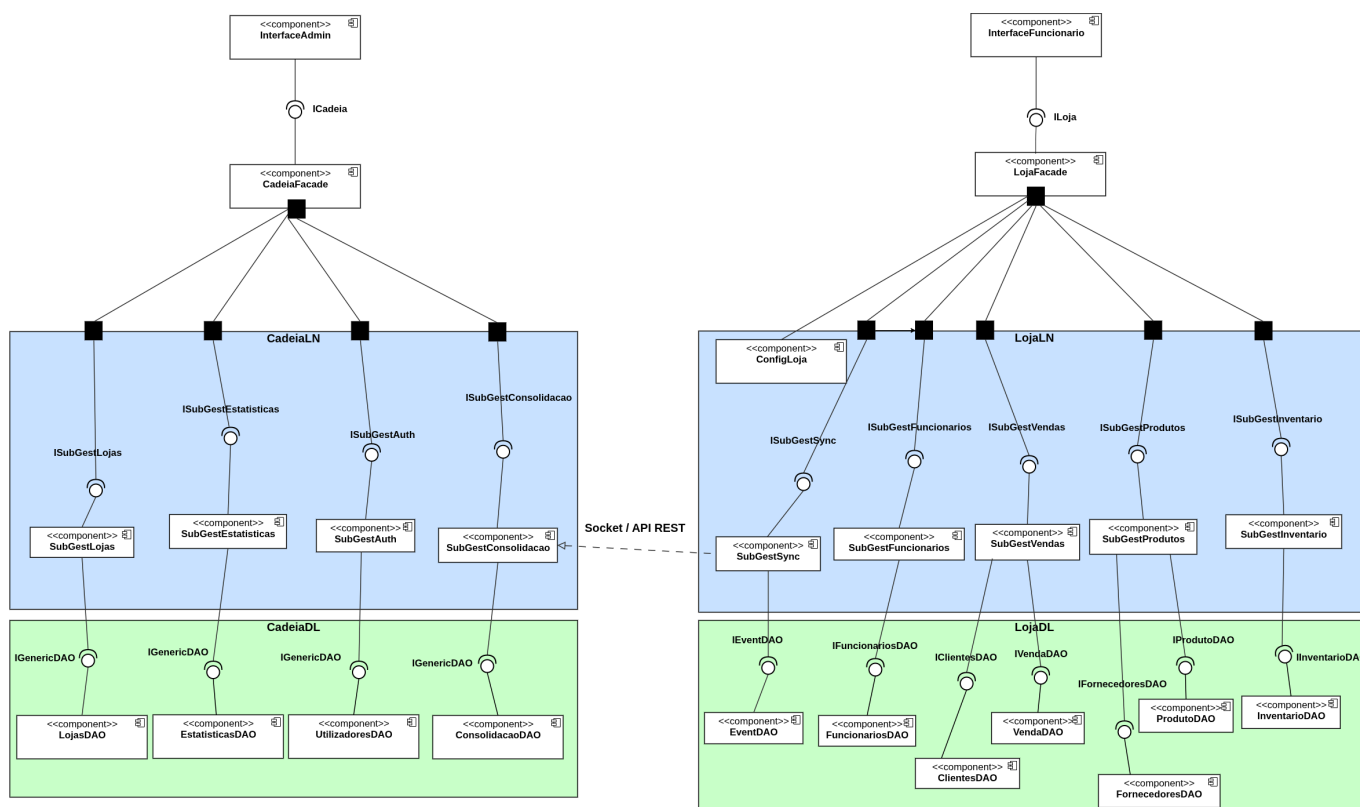


Figura 12: Diagrama de Componentes de todo o Sistema BelaVista

## 7.4. Diagramas de Sequência

Os diagramas de sequência seguintes ilustram os fluxos de interação entre os componentes do sistema para algumas operações escolhidas de cada uma das Interfaces principais. Em cada diagrama, os objetos participantes são representados no topo com as respetivas linhas de vida, e as mensagens trocadas entre eles seguem a ordem temporal de cima para baixo.

### 7.4.1. Consolidação Diária

O diagrama representa o fluxo de sincronização iniciado pelo scheduler local no final do dia. O GestSyncFacade prepara o payload, gera um hash de integridade e envia-o via *POST /api/sincronizacao/push* para o backend central. O GestConsolidacaoFacade valida o hash e, se

válido, delega ao ConsolidacaoDAO o processamento de cada venda e a atualização de stock. O estado da sincronização é atualizado para CONCLUIDO ou ERRO consoante o resultado.

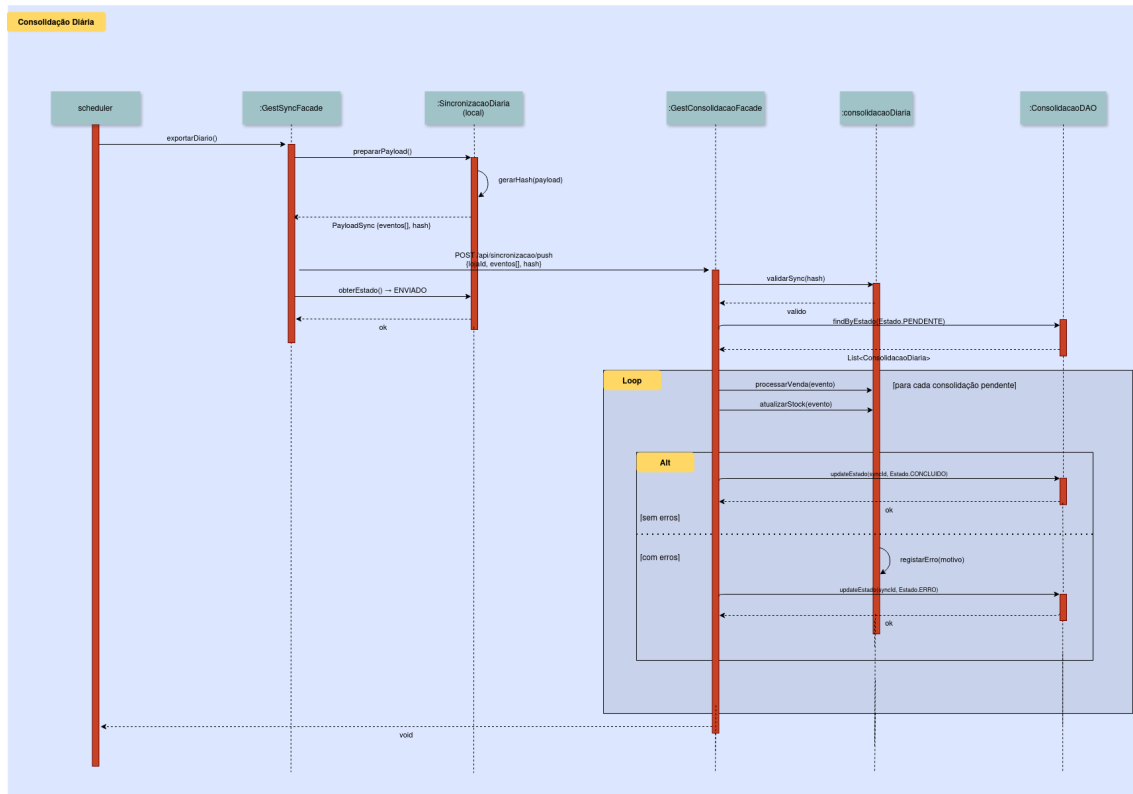


Figura 13: Diagrama de Sequência — Consolidação Diária

#### 7.4.2. Gerar Relatório de Estatísticas por Loja

O diagrama ilustra o fluxo de geração de relatórios estatísticos por loja. O CadeiaFacadeLN delega ao GestEstatisticasFacade, que consulta o EstatisticasDAO com o intervalo de datas e o identificador da loja. Em caso de loja inexistente, o fluxo termina com um erro 404. Se a loja existir, os dados são agregados e devolvidos num EstatisticasLojaResponse com totais de vendas, faturação e período.



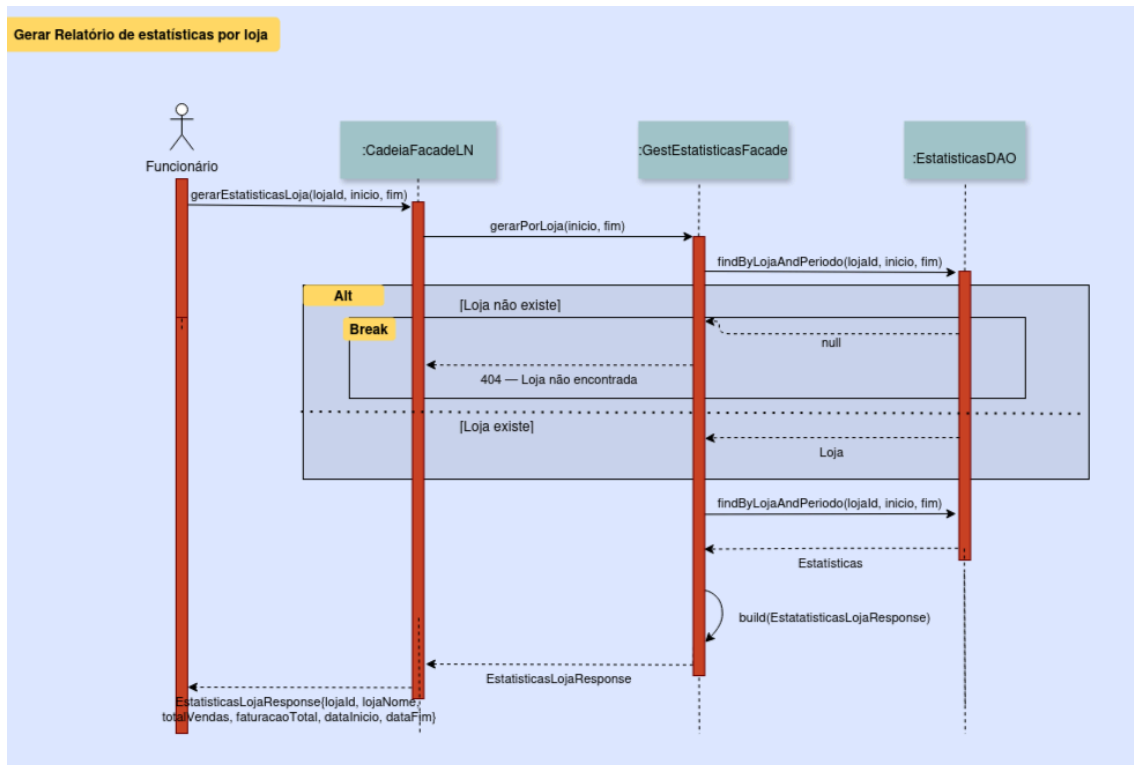


Figura 14: Diagrama de Sequência — Gerar Relatório de Estatísticas por Loja

#### 7.4.3. Autenticar

O diagrama descreve o fluxo de autenticação no backend central. O CadeiaFacadeLN recebe as credenciais e delega ao GestAuthFacade, que invoca o UtilizadoresDAO com o username e o hash da password. Se as credenciais forem válidas, é devolvido o utilizador autenticado; caso contrário, o fluxo termina com erro de autenticação.

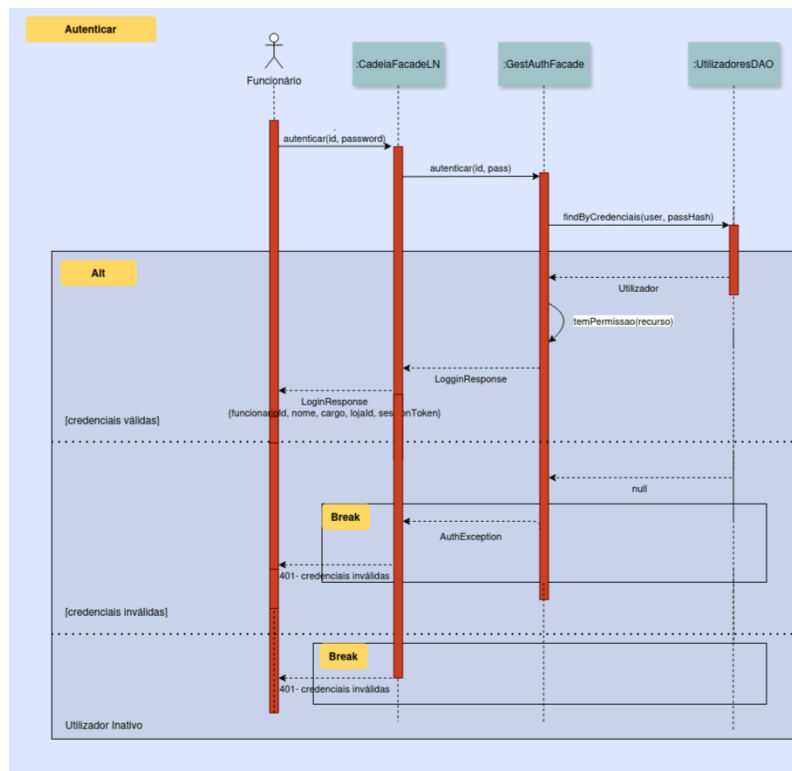


Figura 15: Diagrama de Sequência — Autenticar

#### 7.4.4. Registrar Nova Loja

O diagrama representa o fluxo de registo de uma nova loja na cadeia. O CadeiaFacadeLN delega ao GestLojasFacade, que verifica se já existe uma loja com o mesmo identificador antes de persistir. Se não existir, a loja é criada e devolvida; se já existir, o fluxo termina com um erro de conflito.

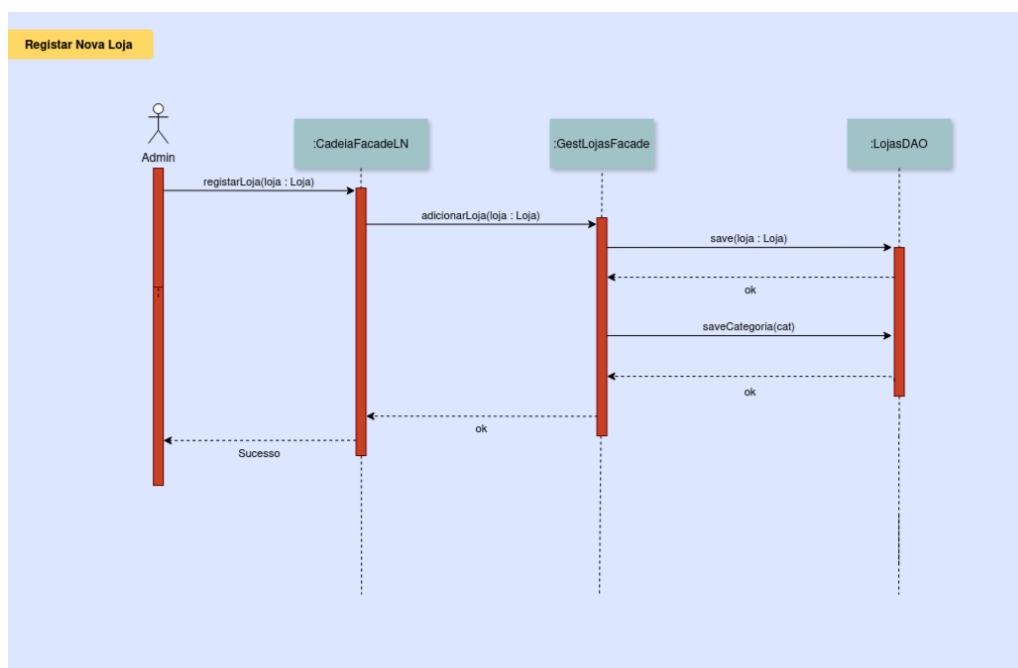


Figura 16: Diagrama de Sequência — Registrar nova loja

#### 7.4.5. Listar Produtos

O diagrama descreve o fluxo de registo de uma venda no sistema. O LojaFacadeLN recebe as credenciais do funcionário e as linhas de venda, delegando ao FuncionarioDAO a validação do funcionário pelo seu número — se não existir, o fluxo é interrompido. Caso seja válido, o controlo passa para o GestVendasFacade, que itera sobre cada linha de venda e invoca o InventarioFacade para atualizar o stock do produto correspondente, persistindo as alterações via InventarioDAO e VendaDAO. No final, o sucesso é devolvido ao funcionário.

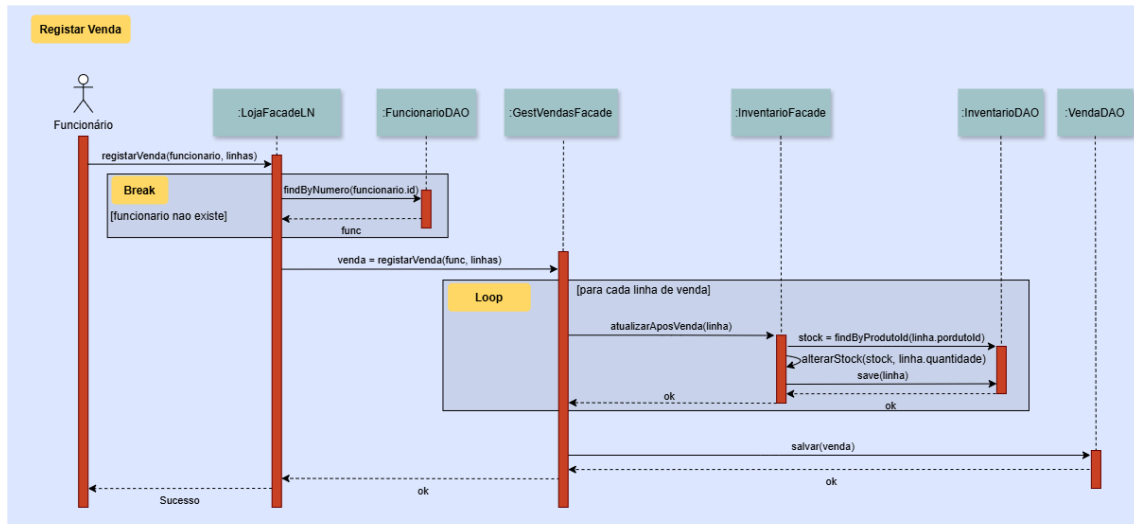


Figura 17: Diagrama de Sequência — Registrar venda

#### 7.4.6. Registrar Entrada de Stock

O diagrama ilustra o fluxo de registo de uma entrada de stock. O LojaFacadeLN recebe o identificador do produto, a quantidade e o fornecedor, validando primeiro a existência do fornecedor via FornecedorDAO — se não existir, o fluxo é interrompido. Em seguida, o produto é validado de forma análoga via ProdutoDAO. Com ambos validados, o controlo é delegado ao InventarioFacade, que localiza o stock associado ao produto, aplica a alteração de quantidade e persiste via InventarioDAO. O sucesso é devolvido ao funcionário.

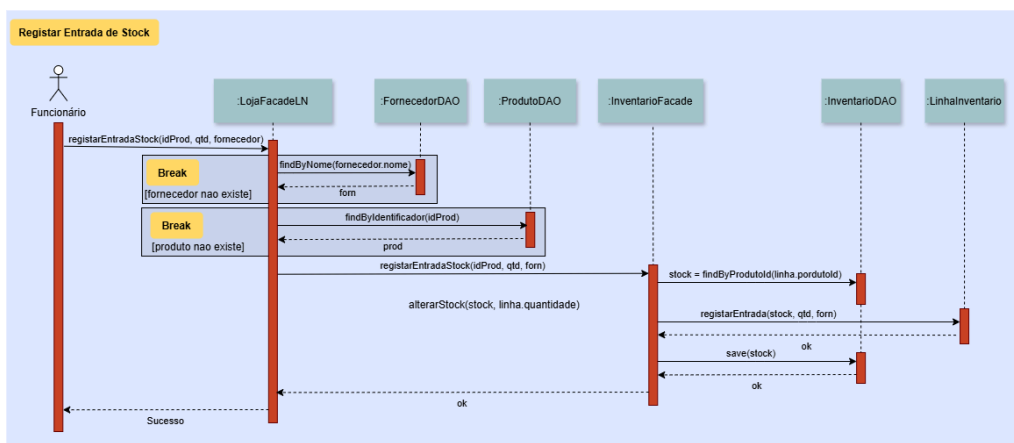


Figura 18: Diagrama de Sequência — Registrar entrada de stock

#### 7.4.7. Exportar Eventos Pendentes

O diagrama representa o fluxo de exportação de eventos pendentes com erro. O LojaFacadeLN recebe o pedido do administrador e delega ao GestSyncFacade, que solicita ao SincronizacaoDiaria a lista de todos os eventos. Para cada evento, o estado é consultado diretamente no objeto Event e, caso seja ERRO, o evento é adicionado à lista de resultado. No final, a lista filtrada é devolvida ao administrador.

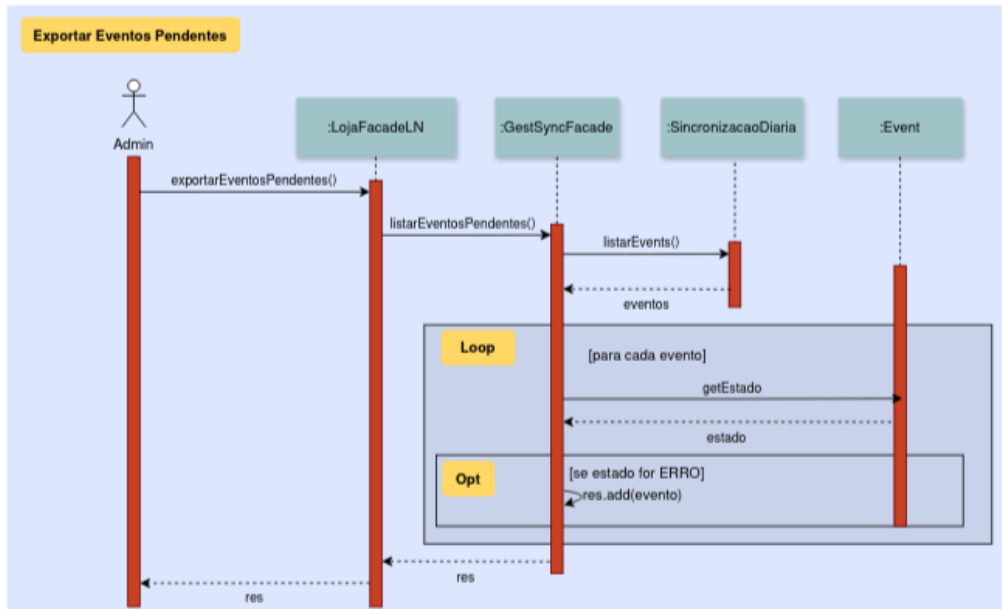


Figura 19: Diagrama de Sequência — Exportar eventos pendentes

#### 7.4.8. Listar Produtos

O diagrama descreve o fluxo de listagem de produtos disponíveis na loja. O LojaFacadeLN recebe o pedido do funcionário e delega ao GestProdutosFacade, que invoca o ProdutoDAO para obter todos os produtos via `findAll()`. A lista resultante é propagada de volta pela cadeia de chamadas até ser devolvida ao funcionário.

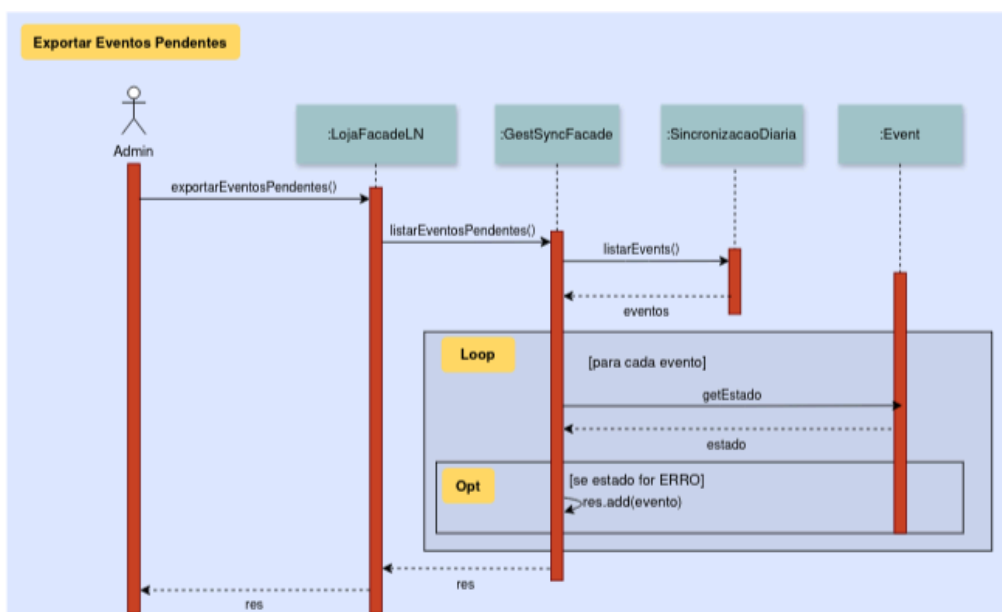


Figura 20: Diagrama de Sequência — Exportar eventos pendentes

## 8. Principais Decisões

### 8.1. Camada Cliente/Loja

A camada superior do diagrama de Arquitetura Geral (Figura 9) representa as localizações físicas da cadeia — Localização 1, Localização 2, Localização 3 e eventuais expansões futuras. Cada localização é tratada como uma unidade autónoma e independente, sem dependências diretas entre si. Esta decisão evita que uma falha numa loja afete as restantes, e é consistente com o requisito de que cada loja atue de forma autónoma (RF2.1, RF4.1).

Uma alternativa viável seria um modelo de loja sem infraestrutura local própria, onde os terminais de venda comunicariam diretamente com o sistema central em tempo real (*thin client*). Esta abordagem simplificaria a gestão de versões do *software* local, mas tornaria toda a operação dependente da disponibilidade da ligação à rede — um risco inaceitável para lojas de conveniência onde a continuidade das vendas é crítica.

### 8.2. Sistema Local (por Loja)

Cada loja possui uma instância própria composta por uma **Interface Local**, uma **Lógica de Negócio** e uma **Base de Dados Local**. Os funcionários interagem exclusivamente com a Interface Local, submetendo pedidos que são processados pela Lógica de Negócio local e persistidos na Base de Dados da respetiva loja.

A Lógica de Negócio local engloba as operações do dia-a-dia: registo de vendas, reposição de stock, gestão de inventário e emissão de faturas. Esta separação garante que todas as operações transacionais ocorrem localmente, sem latência de rede e sem dependência do sistema central, cumprindo o requisito de operação autónoma (RF4.1, RF4.2, RF5.1).

### 8.3. Sistema Central (Cadeia)

O sistema central é composto por uma Interface Central Administrativa, uma Lógica Central implementada como *facade* sobre os subsistemas, um componente de Geração de Relatórios e Estatísticas, e a Base de Dados Central. Os administradores da cadeia interagem exclusivamente com esta camada, nunca com os sistemas locais diretamente.

A separação entre a Lógica Central e o componente de Geração de Relatórios e Estatísticas é uma decisão deliberada — as operações de leitura analítica, potencialmente pesadas, são isoladas das operações de gestão, evitando contenção de recursos e permitindo otimizações independentes, como índices específicos para *queries* analíticas na Base de Dados Central (RNF2.2).

| Subsistema central        | Responsabilidade                                                    | Interface exposta |
|---------------------------|---------------------------------------------------------------------|-------------------|
| Gestão de Lojas           | Registo e consulta de lojas da cadeia (RF2.1, RF2.2)                | IGestLojas        |
| Relatórios e Estatísticas | Agregação por loja, período e categoria (RF6.1, RF6.2, RF6.3)       | IGestRelatorios   |
| Autenticação Central      | Autenticação e gestão de administradores (RF1.1, RF1.2, RF1.3)      | IGestaAuth        |
| Consolidacao (import)     | Receção e consolidação dos <i>payloads</i> diários (RNF4.1, RNF4.2) | IGestConsolidacao |

Tabela 40: Subsistemas do sistema central, requisitos cobertos e interfaces expostas

## 8.4. Mecanismo de Consolidação Diária

O componente de **Consolidação Diária**, representado no diagrama entre as camadas local e central, é o elemento arquitetural que concilia a autonomia operacional de cada loja com a necessidade de agregação e monitorização centralizada da cadeia. No final de cada dia operacional, cada instância local executa um processo de sincronização responsável por exportar os eventos registados ao longo do dia — vendas, devoluções, entradas de stock, alterações de produtos ou atualizações de clientes — para o sistema central.

Ao contrário de uma abordagem baseada apenas em tabelas agregadas, o modelo adotado assenta num mecanismo orientado a **eventos persistidos**. Cada operação relevante executada localmente gera uma instância da classe *Event*, armazenada através do *EventosDAO* e posteriormente associada a uma *SincronizacaoDiaria*. Desta forma, a sincronização deixa de representar apenas um “snapshot” do estado da base de dados e passa a representar uma sequência rastreável de operações de negócio.

A classe *Event* funciona como unidade atómica de sincronização. Cada evento possui:

- um identificador único (*id*);
- um instante temporal (*timestamp*);
- o tipo da operação (*TipoEvento*);
- o identificador da entidade afetada (*entidadeId*);
- o conteúdo serializado da operação (*payload*);
- um mecanismo de verificação de integridade (*hash*);
- e o respetivo estado de sincronização (*Estado*).

Os diferentes tipos de evento modelados refletem diretamente os requisitos funcionais identificados, incluindo:

- *VENDA\_REGISTADA*,
- *DEVOLUCAO\_REGISTADA*,
- *ENTRADA\_STOCK*,
- *PRODUTO\_EDITADO*,
- *CLIENTE\_REGISTADO*,
- entre outros.

A entidade *SincronizacaoDiaria* representa o processo de exportação propriamente dito, agregando uma coleção de eventos produzidos durante o período operacional da loja. O ciclo de vida da sincronização é controlado através do enum *Estado*, transitando pelos estados *PENDENTE*, *EM\_CURSO*, *CONCLUIDO* ou *ERRO*. Esta separação entre o estado da sincronização e o estado

individual de cada Event permite distinguir falhas globais do processo de exportação de falhas específicas associadas a eventos isolados.

O subsistema SubGestSync encontra-se isolado da restante lógica transacional precisamente para minimizar acoplamento arquitetural. Esta decisão permite encapsular toda a lógica relacionada com exportação, serialização, validação e recuperação de falhas sem afetar os restantes subsistemas locais.

A abordagem adotada segue um modelo de **sincronização push**, onde é a própria loja que inicia o envio do *payload* para o sistema central. Esta solução foi preferida em detrimento de uma abordagem **pull**, na qual o sistema central acederia diretamente às bases de dados locais. A opção escolhida apresenta três vantagens principais:

- **Segurança:** as bases de dados locais não necessitam de estar diretamente expostas à rede externa;
- **Consistência:** os eventos apenas são exportados após o fecho operacional do dia, evitando leituras intermédias de transações ainda em processamento;
- **Rastreabilidade:** cada sincronização possui identificador próprio, estados explícitos e mecanismos de validação de integridade através de *hashes*, assegurando conformidade com os requisitos RNF4.1 e RNF4.2.

Uma alternativa possível seria uma arquitetura de sincronização em tempo real baseada em **event streaming** (por exemplo, Apache Kafka ou RabbitMQ), onde cada Event seria publicado imediatamente após a operação local. Embora esta solução permitisse estatísticas praticamente instantâneas e maior desacoplamento temporal entre lojas e sistema central, introduziria complexidade técnica excessiva para o contexto do projeto, incluindo gestão de filas distribuídas, tolerância a falhas em tempo real, mecanismos de confirmação (*acknowledgement*) e tratamento de duplicação de mensagens. A sincronização diária baseada em eventos persistidos revelou-se assim a solução mais equilibrada entre robustez arquitetural, simplicidade de implementação e adequação aos requisitos do sistema BelaVista.

## 8.5. Categoria como Entidade Escalável em vez de Enumeração

O tipo de categoria de produto foi modelado como uma entidade Categoria com os seus próprios atributos, em vez de uma enumeração fechada com valores fixos como FrutasVegetais, Padaria ou Bebidas.

Uma enumeração implicaria uma alteração ao código-fonte sempre que fosse necessário adicionar ou remover uma categoria — uma operação de gestão que deveria ser feita em tempo de execução pelo administrador, sem intervenção técnica. A entidade Categoria permite criar, editar e desativar categorias dinamicamente, cobrindo RF3.1 e RF3.4 de forma sustentável ao longo do tempo.

## 8.6. MovimentoStock Unificado como Operação de Inventário

Em vez de classes distintas para cada tipo de movimentação de stock (saída por venda, entrada por fornecedor, ajuste manual, devolução), foi adotada uma classe MovimentoStock única com um atributo tipo que discrimina a origem do movimento.

A alternativa hierárquica — subclasses MovimentoVenda, MovimentoEntrada, MovimentoDevolucao — introduziria complexidade de herança sem benefício proporcional, dado que os atributos relevantes são os mesmos em todos os casos: produto, quantidade, data/hora,

motivo e referência à origem. A classe unificada simplifica as *queries* de histórico (RF4.5) e os alertas de reposição (RF4.6), que operam sobre o stock resultante independentemente da causa do movimento.

## 8.7. Entidade Inventário

A entidade Inventário foi removida do modelo. Na versão anterior, funcionava como contentor das *LinhaInventario*, mas não acrescentava semântica própria — era essencialmente um agrupador sem atributos relevantes além do identificador e da data de última atualização.

Com a sua remoção, *LinhaInventario* passa a existir diretamente no contexto da loja, dado que cada *backend* local serve exatamente uma loja. O estado do stock de cada produto é mantido diretamente na *LinhaInventario* (evitando que este valor fique associado a um Produto), atualizada após cada venda (RF4.3), cada entrada de fornecedor (RF4.4) e cada devolução (RF5.7). O histórico de movimentos fica como uma lista de *MovimentoStock*, que é um *log* de eventos e não uma entidade de estado.

## 8.8. Utilizador como Classe Base com Herança para Funcionario

Em vez de classes completamente separadas para cada perfil de utilizador, foi adotada uma classe base *Utilizador* com os atributos comuns de autenticação (*id*, *nome*, *user*, *pass*), da qual *Funcionario* herda e especializa com o atributo *Cargo*.

Esta decisão evita duplicação dos atributos de autenticação e permite que o mecanismo de *login* (RF1.2) seja implementado uma única vez na classe base. O controlo de acessos (RF1.3) é depois aplicado com base no *Cargo* do *Funcionario* instanciado, sem necessidade de lógica condicional sobre o tipo do objeto. A desativação de utilizadores (RF1.5) é também gerida na classe base, preservando o histórico de operações associado.

## 8.9. Cliente com NIF como Chave Primária Natural

O *Cliente* foi modelado com o NIF como identificador primário, em vez de um UUID sintético gerado pelo sistema.

O RF8.1 especifica explicitamente que “o NIF é utilizado como identificador único”. Usar um UUID sintético introduziria uma chave artificial que coexistiria com uma chave natural já garantida externamente — redundância sem benefício. O NIF é único por definição legal, imutável, e serve diretamente para a emissão de faturas (RF5.6), tornando-o o identificador natural correto.

## 8.10. Devolucao como Entidade separada de LinhaVenda

As devoluções foram modeladas numa entidade *Devolucao* independente, em vez de um atributo booleano ou *flag* na *LinhaVenda*.

Uma devolução pode ser parcial (parte das unidades de uma linha), pode ocorrer em momento distinto da venda original, e tem atributos próprios: motivo, data/hora e total devolvido. Se fosse um *flag* na *LinhaVenda*, o cálculo do total da venda (RF5.4) ficaria dependente de lógica condicional sobre o estado de cada linha, e o registo histórico das devoluções (RF5.7) seria ambíguo.



A entidade separada preserva a integridade do registo original da venda e modela corretamente o ciclo de vida de uma transação comercial.

### 8.11. ConfigLoja como *Singleton*

A entidade ConfigLoja existe no modelo local com o estereótipo «singleton» — uma única instância por *backend* — em vez de ser tratada como uma Loja convencional com relações para outras entidades.

Dado que cada *backend* serve exatamente uma loja, não faz sentido manter Loja como entidade relacional no contexto local — não há coleção de lojas para gerir. ConfigLoja guarda apenas os dados de identificação da loja (*lojaId*, *nome*, *localizacao*) e o *token* de sincronização necessário para o SubGestSync se autenticar junto ao central. Nenhuma outra entidade referencia ConfigLoja via chave estrangeira — o contexto de loja está implícito em todo o *backend*.

### 8.12. Bases de Dados Separadas por Loja

Cada instância local possui a sua própria Base de Dados, independente das restantes lojas e do sistema central. A alternativa seria uma Base de Dados central partilhada com controlo de acessos por loja via RBAC.

A BD local garante que as operações de venda e stock funcionam mesmo sem conectividade (RF4.2, RF4.3, RF5.1), elimina contenção entre lojas em operações de escrita concorrente, e evita que uma loja aceda inadvertidamente a dados de outra — o isolamento é físico, não apenas lógico. O custo é a necessidade de um mecanismo de sincronização, tratado pelo SubGestSync. O requisito de integridade referencial (RNF1.2) é garantido dentro de cada BD local de forma independente.

### 8.13. Diagrama Lógico de Persistência

Os diagramas seguintes apresentam o modelo lógico das duas bases de dados do sistema — a Base de Dados Central e a Base de Dados Local — derivados diretamente do modelo de classes descrito nas secções anteriores. A separação física em duas bases de dados é, por si só, uma decisão de design: cada base de dados tem um esquema otimizado para o tipo de operações que suporta, e nenhuma tabela cruza os limites entre os dois contextos. A comunicação entre as duas bases de dados não é feita por *joins* ou ligações diretas, mas exclusivamente através do mecanismo de sincronização diária descrito na arquitetura.

Todas as tabelas incluem os campos *createdAt* : *DateTime* e *deleted* : *Boolean*, aplicando um padrão de *soft delete* uniforme em todo o sistema. A eliminação física de registos nunca ocorre — os registos são marcados como eliminados e excluídos das *queries* operacionais, preservando a integridade referencial e o histórico de auditoria exigidos pelos requisitos de rastreabilidade (RNF4.2).

#### 8.13.1. Base de Dados Central

A base de dados central armazena os dados de governação da cadeia e os resultados das consolidações diárias. O seu esquema é predominantemente de leitura — a escrita ocorre na janela de consolidação noturna e nas operações de gestão executadas pelos administradores.

A tabela `Config_Cadeia` é o ponto de ancoragem do sistema central, identificando a cadeia e registrando o momento da última sincronização via `lastSync`. A tabela `Loja` referencia `Config_Cadeia` via `idCadeia` e mantém igualmente um `lastSync` que permite ao sistema central identificar lojas cujos dados não chegaram na janela esperada. A tabela `Utilizador` centraliza as identidades de todos os perfis com acesso ao sistema central, com a `password_hash` armazenada de acordo com RNF3.1, e o cargo como chave estrangeira para a tabela `Cargo`, que materializa o controlo de acessos baseado em perfil (RF1.3). A tabela `Fornecedor_Loja` é uma tabela de associação que reflete a relação muitos-para-muitos entre fornecedores e lojas, permitindo que o mesmo fornecedor sirva múltiplas lojas com contratos distintos. A tabela `Categoria_Globais` define o catálogo de categorias gerido centralmente, evitando a fragmentação de categorias entre lojas — é esta tabela que torna `Categoria` uma entidade escalável em vez de uma enumeração estática. As tabelas `ConsolidacaoDiaria` e `RegistoConsolidado` são o núcleo do subsistema de sincronização: a primeira regista o ciclo de vida de cada importação por loja (com estado, `totalRegistos` e motivo para diagnóstico de falhas), e a segunda persiste o detalhe de cada registo consolidado, referenciando a loja, a sincronização de origem, o produto e a quantidade transacionada.

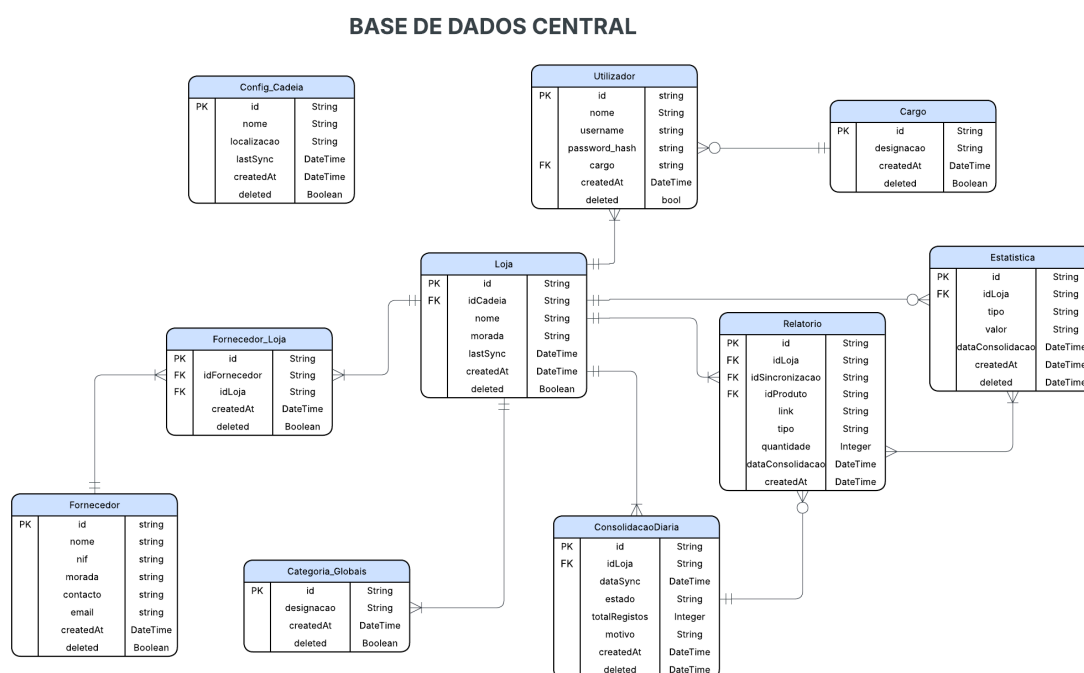


Figura 21: Esquema Lógico da Base de Dados Cadeia

### 8.13.2. Base de Dados Local

A base de dados local armazena os dados operacionais de uma única loja. O seu esquema é otimizado para escrita frequente — cada venda, cada movimento de stock e cada entrada de fornecedor geram registos novos em tempo real.

A tabela `Config_Loja` é o *singleton* de configuração do backend local, com nome, localização e `lastSync`, fornecendo ao subsistema de sincronização os dados de identificação necessários para autenticar cada exportação junto ao central. A tabela `Funcionario` contém as credenciais operacionais dos utilizadores locais — `username`, `password_hash` e `cargo` como chave estrangeira para a tabela `Cargo` local, independente da tabela `Cargo` central. Esta independência é intencional: a

autenticação local funciona sem qualquer ligação ao central (RF1.2, RNF3.1). A tabela Produto referencia idFornecedor e idCategoria, e inclui precUnit para suportar o cálculo do valor das linhas de venda sem necessitar de *joins* adicionais. A gestão de stock é suportada pelas tabelas Linha\_Inventario e Movimento\_Stock: a primeira mantém o estado atual do stock de cada produto com quantidadeAtual e quantidadeMin para geração de alertas de reposição (RF4.6), e a segunda regista cada alteração ao stock com tipo, quantidade e motivo, servindo como log de auditoria (RF4.5). A tabela Produto\_Fornecedor materializa a associação entre produtos e fornecedores locais, permitindo registar entradas de stock por fornecedor (RF4.4). O ciclo de vida de uma venda é suportado pelas tabelas Venda, Linha\_Venda, Fatura e Devolucao: Venda referencia o funcionário e opcionalmente o cliente via nifCliente, com estado a acompanhar o progresso da transação; Linha\_Venda detalha os produtos e quantidades; Fatura é opcionalmente gerada a partir da venda (RF5.6); e Devolucao referencia a linha de venda original, o funcionário responsável e regista valorDevolvido e motivo (RF5.7). Por fim, a tabela SincronizacaoDiaria regista localmente o histórico de exportações com payload, estado e motivo, espelhando do lado local o que a tabela ConsolidacaoDiaria regista do lado central — garantindo rastreabilidade bidirecional de cada sincronização (RNF4.1, RNF4.2).

#### BASE DE DADOS LOCAL

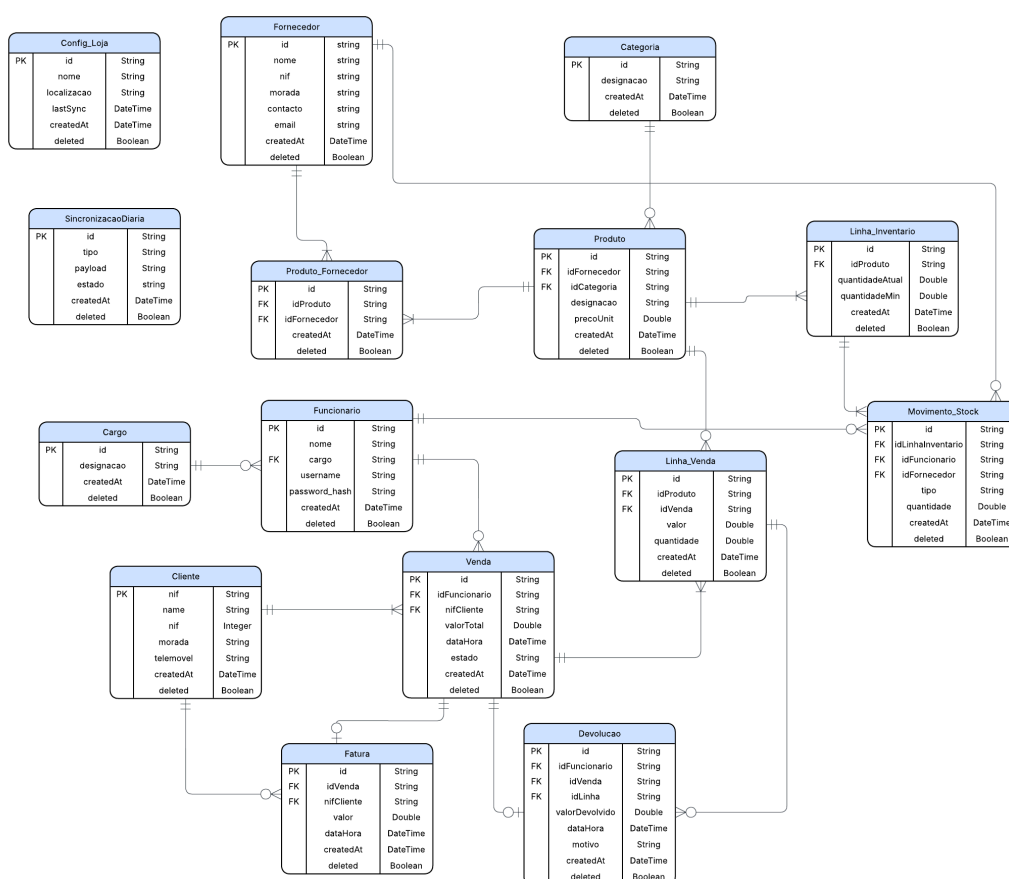


Figura 22: Esquema Lógico da Base de Dados Local (Loja)

## 9. Stack Tecnológica

Para cada camada do sistema — *frontend*, *backend* e base de dados — foram avaliadas algumas alternativas, tendo sido escolhida a que melhor se adequava aos requisitos do projeto, e à nossa própria experiência pessoal.

### 9.1. Frontend

Entre as opções consideradas para a interface da nossa solução, denotam-se:

- **React + Vite** é uma solução minimalista que oferece velocidade de desenvolvimento e total liberdade de estrutura. No entanto, exige que o programador configure manualmente o *router*, a gestão de estado e a organização das páginas, o que aumenta a complexidade inicial e a inconsistência entre projetos de equipa.
- **Angular** é um *framework* completo e fortemente opinado, com injeção de dependências, roteamento e formulários integrados. É uma escolha sólida para aplicações *enterprise* de grande escala, mas impõe uma curva de aprendizagem acentuada e uma estrutura mais rígida, que se revelaria excessiva para o âmbito deste projeto.
- **Next.js** é um *framework* construído sobre React que oferece roteamento baseado em ficheiros, suporte a *Server* e *Client Components*, e uma estrutura de projeto bem definida. Combina a flexibilidade do React com convenções que reduzem decisões de arquitetura repetitivas.

#### 9.1.1. Escolha: Next.js + TypeScript + Tailwind CSS

Foi escolhido **Next.js 16** pela estrutura de roteamento nativa — que se mapeou diretamente sobre as páginas do sistema (*dashboard*, *login*, inventário, vendas, relatórios, lojas) — e pela integração natural com **TypeScript**, que fornece tipagem estática e melhora a deteção de erros em tempo de desenvolvimento, em particular na definição dos contratos com a API (`lib/api.ts`). A estilização foi feita com **Tailwind CSS 4**, que permite definir estilos diretamente nos componentes sem ficheiros CSS separados, acelerando o desenvolvimento de interfaces consistentes. Para gráficos e visualizações usámos **Recharts** e para ícones **Lucide React**.

### 9.2. Backend

Analogamente, apresentamos de seguida algumas opções que consideramos para o *backend* da nossa aplicação:

- **Java + Spring Boot** é a escolha clássica para *backends* JVM. O ecossistema é maduro, a documentação é extensa e a integração com Spring é nativa. A principal desvantagem é a verbosidade da linguagem — *getters*, *setters*, *builders* e tratamento de nulos exigem muito código repetitivo que Kotlin elimina.
- **Python + FastAPI** oferece desenvolvimento rápido, sintaxe expressiva e um servidor assíncrono moderno. É uma excelente escolha para APIs simples ou de prototipagem, mas perde em tipagem estrita em tempo de compilação, *performance* sob carga elevada e maturidade do ecossistema para aplicações *enterprise* com múltiplos módulos e ORM complexo.

- **Kotlin + Spring Boot** combina a robustez da JVM e do ecossistema Spring com uma linguagem moderna. A *null safety* nativa elimina uma categoria inteira de erros em *runtime*, e funcionalidades como *data classes* e *extension functions* reduzem significativamente o *boilerplate*.

### 9.2.1. Escolha: Kotlin + Spring Boot

Foi escolhido **Kotlin 2.1.10** com **Spring Boot 3.4.4**, gerido pelo **Gradle** com scripts em `.kts`. A decisão baseou-se em três fatores principais. Primeiro, a *null safety* integrada na linguagem reduz erros em *runtime* sem necessidade de anotações adicionais. Segundo, a **concisão** do Kotlin permitiu definir entidades de domínio (Produto, Funcionario, Loja), DTOs e repositórios com muito menos código que o equivalente em Java. Terceiro, a **interoperabilidade total** com Java garante acesso a todo o ecossistema Spring sem restrições, incluindo o `jackson-module-kotlin` para serialização de JSON e o `kotlin.plugin.spring` para compatibilidade com *proxies* do Spring. No geral era uma linguagem que tínhamos curiosidade em aprender além do Java que já tínhamos utilizado no curso.

## 9.3. Base de Dados

No que toca à base de dados, eis algumas das opções que consideramos também:

- **PostgreSQL** é um sistema de gestão de bases de dados relacional de grau *enterprise*, com suporte a transações complexas, tipos avançados e escalabilidade horizontal. É a escolha natural para sistemas em produção, mas exige a instalação e configuração de um servidor dedicado, o que aumenta a complexidade do ambiente de desenvolvimento.
- **MySQL** é outra alternativa relacional amplamente utilizada, com bom desempenho em leituras e ecossistema maduro. Tal como o PostgreSQL, requer um servidor separado e configuração adicional, sem vantagens significativas face ao PostgreSQL para este contexto.
- **SQLite** é uma base de dados relacional *embebida* que armazena toda a informação num único ficheiro. Não requer servidor, configuração de rede nem gestão de utilizadores, sendo iniciada automaticamente com a aplicação.

### 9.3.1. Escolha: SQLite

Foi escolhido **SQLite** (via `sqlite-jdbc 3.45.3.0`) por adequação ao contexto académico do projeto: elimina a necessidade de instalar um servidor de base de dados separado, simplificando o *deployment* e o desenvolvimento local. Cada serviço tem a sua própria base de dados independente (`cadeia.db` e `loja.db`), respeitando o isolamento entre os contextos da cadeia e das lojas. As chaves estrangeiras foram ativadas explicitamente via `PRAGMA foreign_keys = ON`. A integração foi feita através do dialeto `SQLiteDialect` da comunidade Hibernate (`hibernate-community-dialects`), mantendo a camada de persistência compatível com Spring Data JPA. Em produção real, a migração para PostgreSQL seria trivial — bastaria alterar o *driver* e o dialeto, sem alterações ao código de repositórios ou serviços.

## 9.4. Resumo

| Camada          | Alternativas avaliadas         | Escolha final                          |
|-----------------|--------------------------------|----------------------------------------|
| <i>Frontend</i> | React + Vite, Angular, Next.js | Next.js 16 + TypeScript + Tailwind CSS |

| Camada         | Alternativas avaliadas                                     | Escolha final                     |
|----------------|------------------------------------------------------------|-----------------------------------|
| <i>Backend</i> | Java + Spring Boot, Python + FastAPI, Kotlin + Spring Boot | Kotlin 2.1.10 + Spring Boot 3.4.4 |
| Base de dados  | PostgreSQL, MySQL, SQLite                                  | SQLite 3.45.3.0                   |

## 10. Esboço das Interfaces do Sistema

### 10.1. Interface e Prototipagem

O protótipo da interface foi desenvolvido em duas fases. Numa primeira fase, utilizámos o **Stitch** - uma ferramenta de prototipagem assistida por inteligência artificial que gera *wireframes* e ecrãs de alta-fidelidade a partir de descrições textuais dos requisitos. Esta abordagem permitiu iterar rapidamente sobre o *layout* e a organização das diferentes secções do sistema sem investir tempo de desenvolvimento real.

Numa segunda fase, o protótipo gerado pelo *Stitch* foi importado para o **Figma**, onde foi refinado, organizado em *frames* e anotado para servir de referência visual durante o desenvolvimento. É de notar que a transição entre as duas ferramentas introduziu algumas inconsistências de formatação — espaçamentos, tamanhos de fonte e alinhamentos sofreram ligeiras alterações no processo de importação, pelo que o protótipo no Figma deve ser interpretado como uma referência de estrutura e fluxo de navegação, e não como especificação de exatamente como ficou a solução final.

#### 10.1.1. Vantagens do *Stitch*

- Geração rápida de ecrãs de alta-fidelidade a partir de descrições textuais, acelerando significativamente a fase inicial de prototipagem.
- Permite explorar múltiplas variantes de *layout* sem custo de desenvolvimento.
- Adequado para comunicar ideias de interface a elementos da equipa sem background técnico.

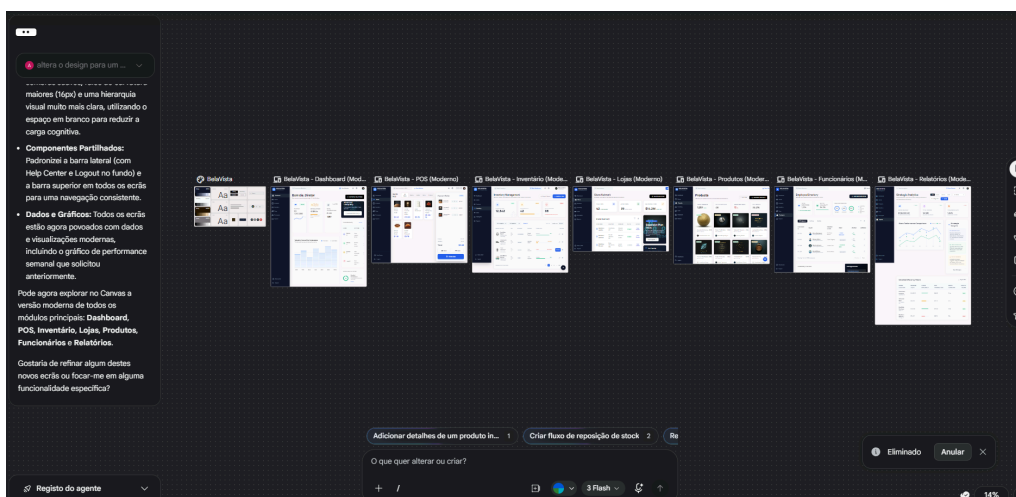


Figura 23: Interface da plataforma Stitch

### 10.1.2. Desvantagens do *Stitch*

- O output gerado não é totalmente controlável — detalhes visuais como paletas de cor, tipografia e espaçamento resultam de decisões automáticas que podem não coincidir com a identidade visual pretendida.
- A exportação para Figma introduz perda de fidelidade: componentes podem desformatar, grupos podem perder estrutura e algumas propriedades de estilo não são mapeadas corretamente.
- Não substitui um processo de *design* iterativo com utilizadores reais — serve apenas como ponto de partida.

### 10.1.3. Vantagens do Figma

- Ferramenta colaborativa que permite que toda a equipa visualize e comente o protótipo em tempo real.
- Suporta *auto-layout*, componentes reutilizáveis e sistemas de design, o que facilita a manutenção de consistência visual.
- Integração direta com ferramentas de desenvolvimento — os programadores podem inspecionar espaçamentos, cores e tipografia diretamente no Figma.
- Permite criar protótipos interactivos com navegação entre ecrãs, simulando o fluxo real da aplicação.

### 10.1.4. Desvantagens do Figma

- A versão gratuita limita o número de ficheiros e de editores simultâneos.
- A curva de aprendizagem para tirar partido das funcionalidades avançadas (componentes, variantes, *auto-layout*) é moderada.
- Para equipas sem experiência em design, manter consistência entre ecrãs pode tornar-se trabalhoso sem uma estrutura de componentes bem definida desde o início.

## 10.2. Descrição das Interfaces Prototipadas

O protótipo cobre as principais secções funcionais do sistema BelaVista, com uma estrutura de navegação lateral consistente em todas as páginas — *Dashboard*, *Stores*, *Products*, *Inventory*, *Sales*, *Employees* e *Reports* — acompanhada de uma barra superior com pesquisa global, seletor de loja, notificações e acesso ao perfil do utilizador.

***Dashboard*** — A página inicial apresenta ao diretor um resumo executivo com os indicadores mais relevantes: receita total, *stock* crítico e total de transações nas últimas 24 horas. Um gráfico de barras compara o desempenho de vendas semanal face à semana anterior. No painel lateral direito são exibidas notificações de eventos de formação e um registo de atividade recente por utilizador.



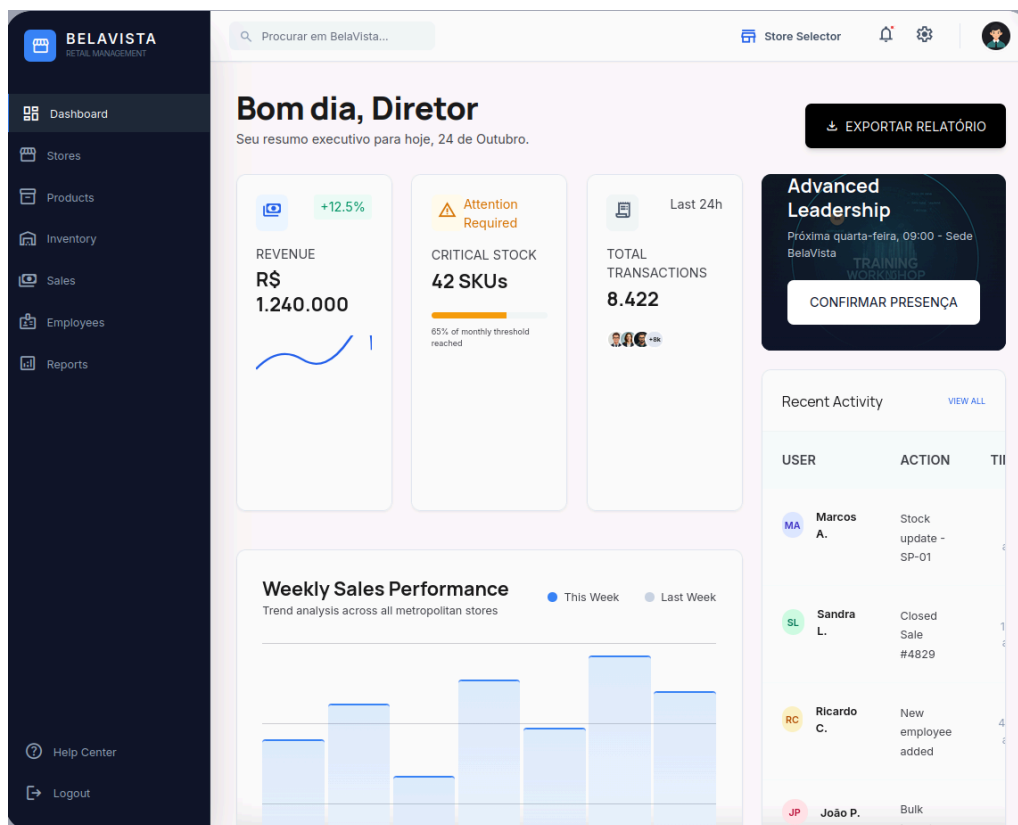


Figura 24: Ecrã de *Dashboard* — resumo executivo para o diretor

**Vendas (Sales)** — A interface de ponto de venda permite ao funcionário adicionar produtos rapidamente através de uma grelha com filtros por categoria (Bakery, Coffee, Dairy, Produce). Do lado direito é apresentada a encomenda atual com quantidade, preço por linha, subtotal, imposto e total. O processo de pagamento é concluído com a seleção do método (Cash ou Card) e confirmação via *Finish Sale*.

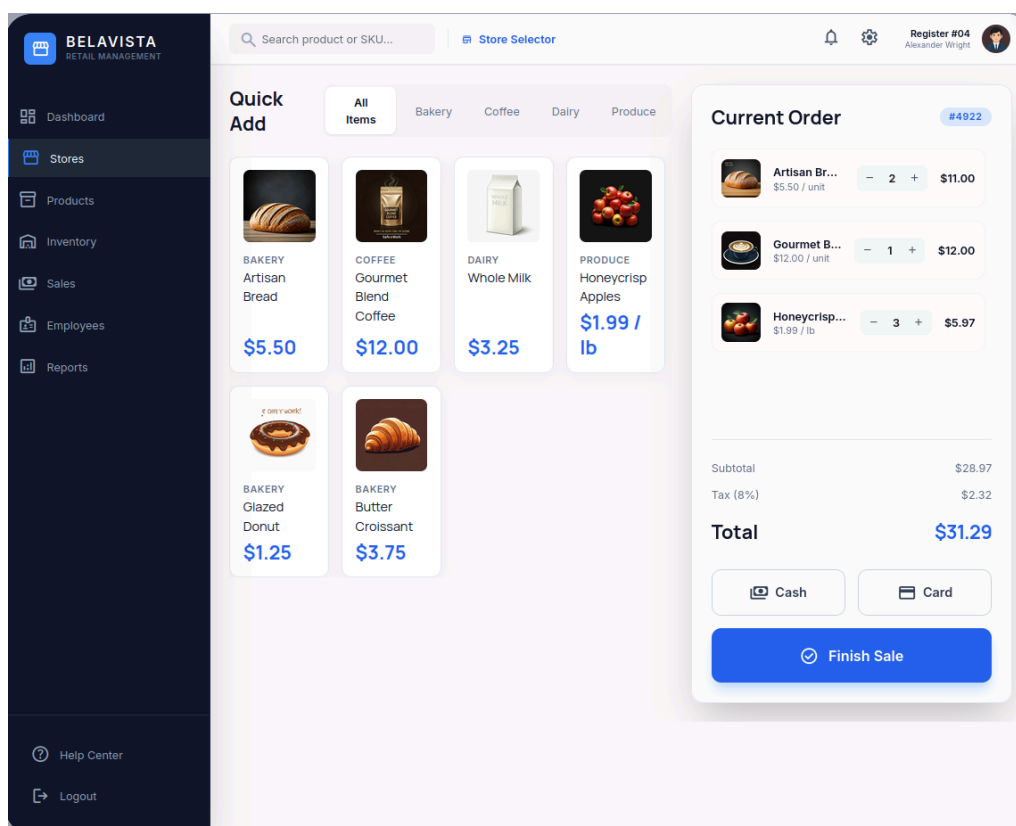


Figura 25: Ecrã de Vendas — ponto de venda com encomenda atual

**Lojas (Stores)** — A secção de gestão da rede de lojas apresenta três métricas globais (total de lojas, lojas ativas e receita média por loja) e uma tabela com o inventário de lojas, incluindo localização, gestor responsável e estado operacional. Um painel lateral destaca iniciativas estratégicas em curso, como planos de expansão e novas aberturas.

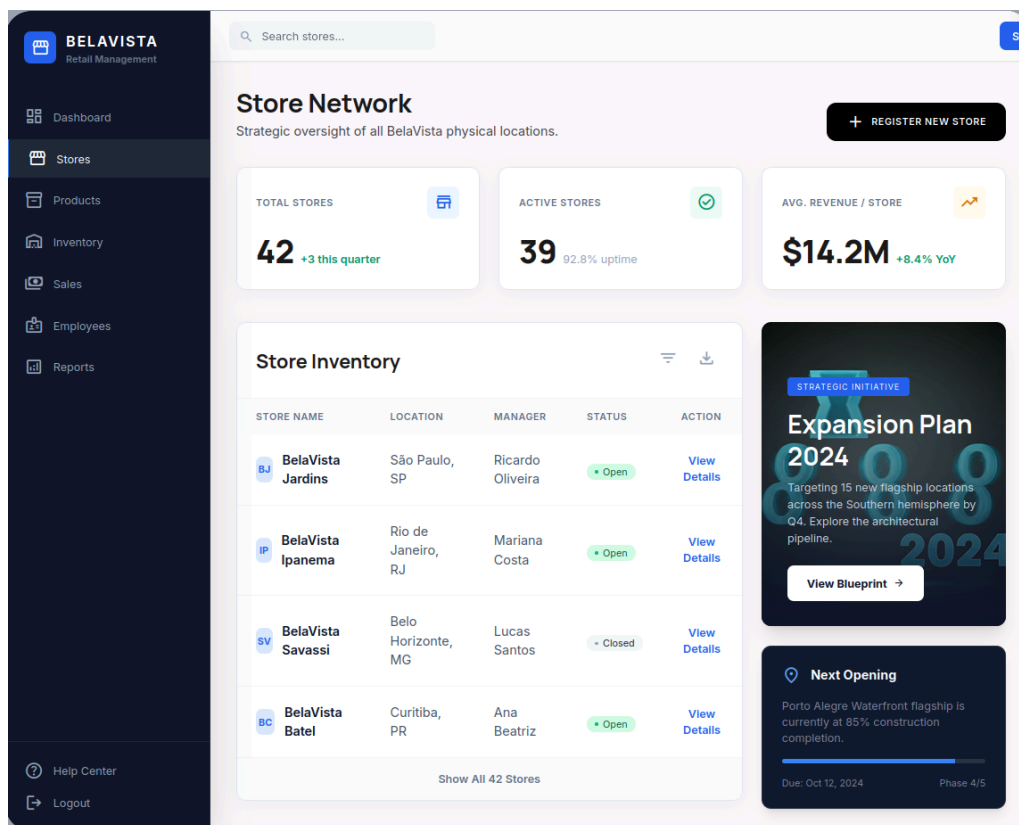


Figura 26: Ecrã de Lojas — visão geral da rede de lojas

**Produtos (*Products*)** — A página de produtos exibe o catálogo global em grelha, com fotografia, nome, SKU, fornecedor, preço e etiqueta de estado de *stock* (In Stock, Low Stock, Out of Stock). Quatro métricas de topo resumem o total de produtos, fornecedores ativos, alertas de *stock* baixo e margem média.

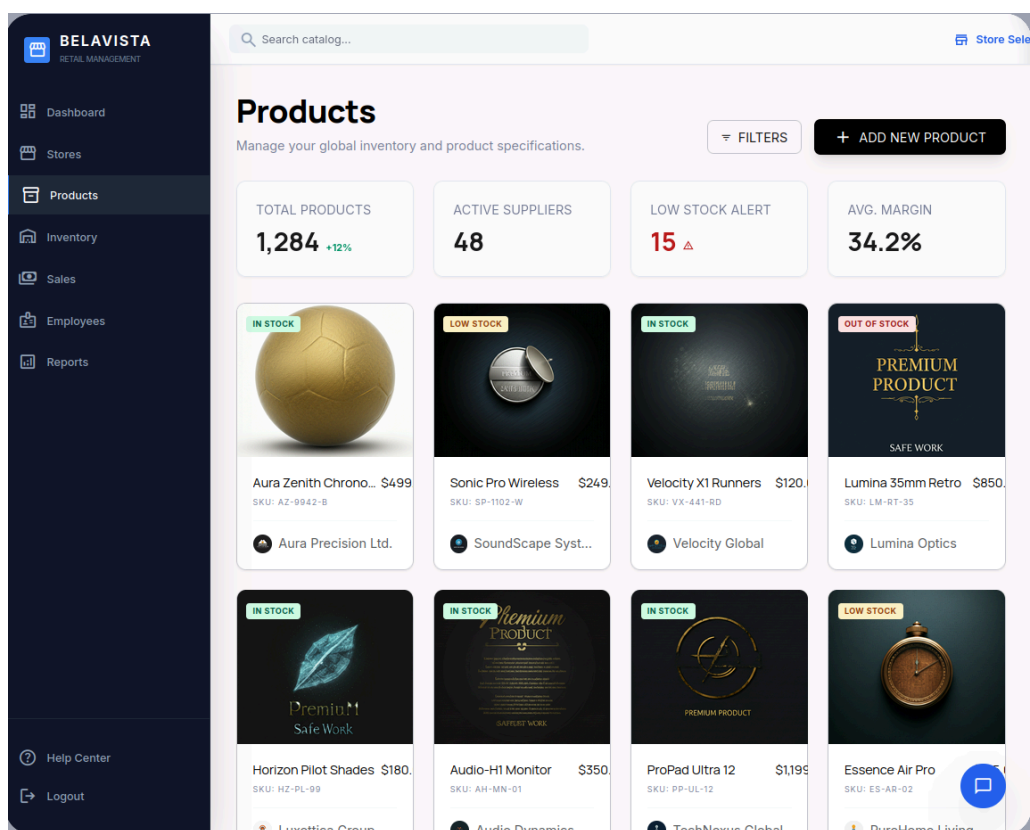


Figura 27: Ecrã de Produtos — catálogo global em grelha

**Inventário (*Inventory*)** — A gestão de inventário apresenta alertas de *stock* baixo e itens esgotados em destaque, com estimativa de perda de receita associada. A listagem tabular inclui SKU, categoria, fornecedor, nível de *stock* com barra de progresso visual e ações de reposição diretamente acessíveis.

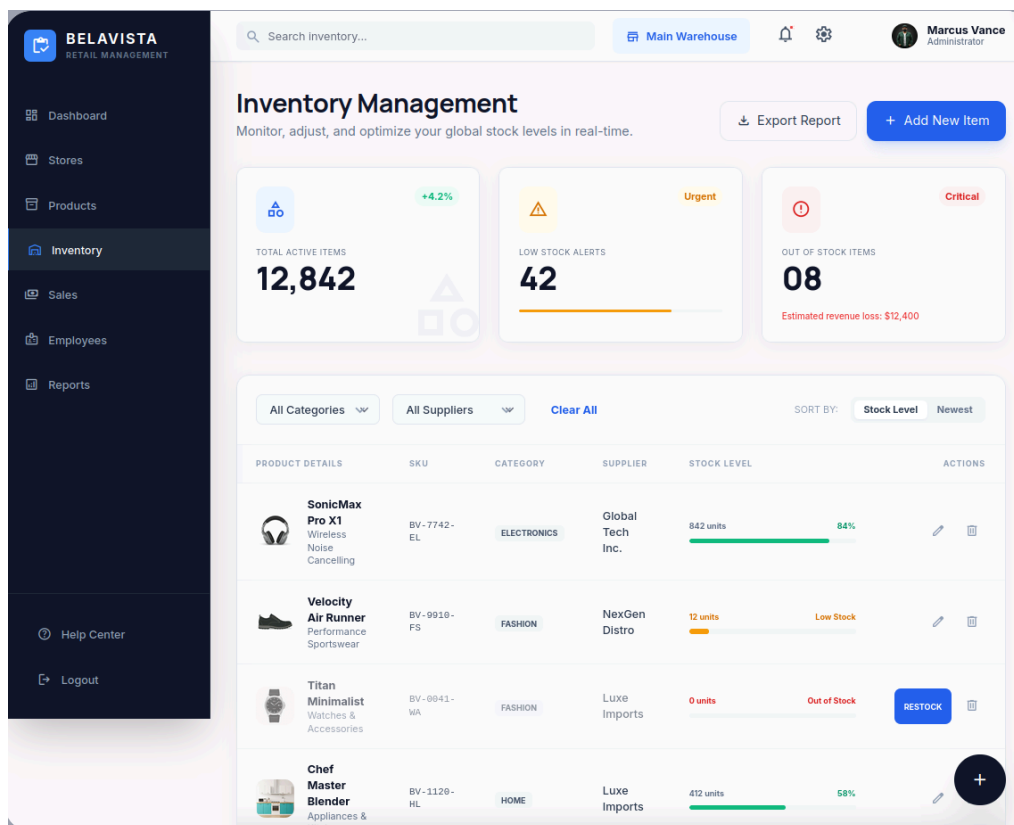


Figura 28: Ecrã de Inventário — gestão de stock e alertas de reposição

**Funcionários (*Employees*)** — O diretoria de funcionários agrega métricas de força de trabalho total, disponibilidade ativa e distribuição por papel (gestores, associados, logística), complementadas com uma tabela paginada com ID, nome, loja principal, função e estado de serviço.

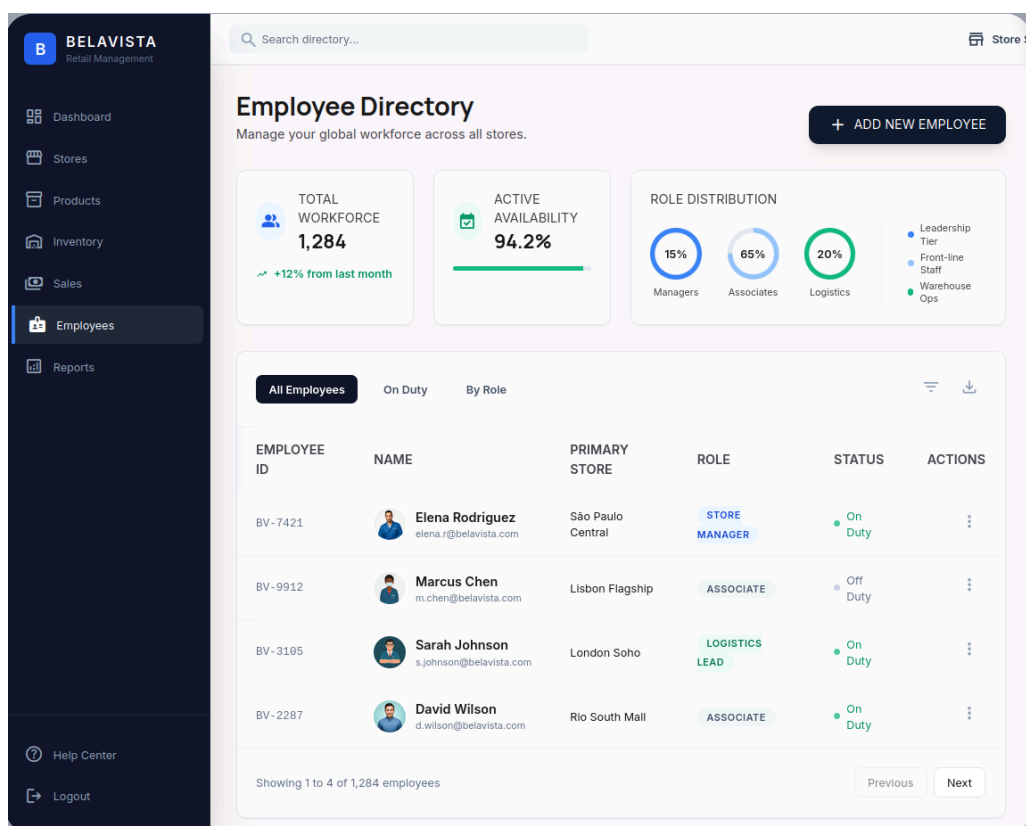


Figura 29: Ecrã de Funcionários — diretoria e distribuição por papel

**Relatórios (Reports)** — A secção analítica permite filtrar por período (diário, semanal, mensal, anual) e por loja, apresentando receita bruta, total de encomendas e taxa de conversão. Um gráfico de linhas compara receita e encomendas ao longo da semana, e um painel de *Strategic Insights* gerado automaticamente destaca alertas de inventário, otimização de horários e oportunidades de preço.

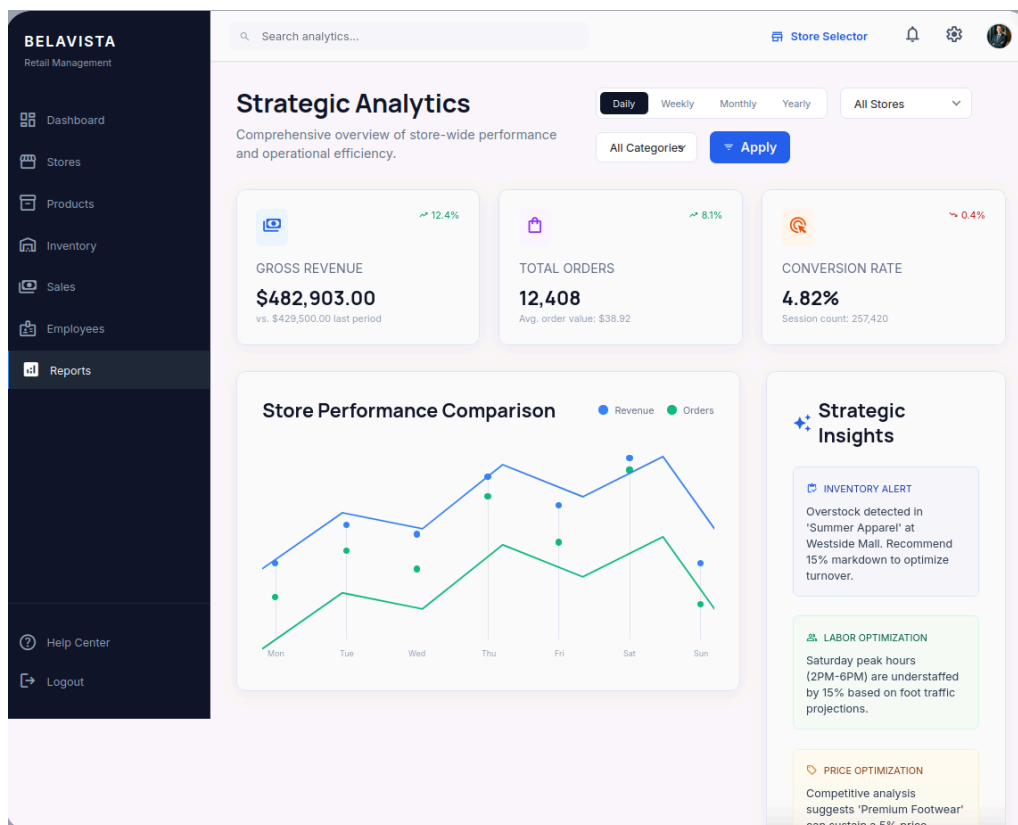


Figura 30: Ecrã de Relatórios — análise estratégica e *insights* automáticos

## Avaliação de Resposta LLM

### PROMPT

Cria o protótipo de uma aplicação de gestão integrada para uma cadeia de pequenas lojas de conveniência chamada BelaVista. A aplicação deve incluir as seguintes secções: Dashboard com resumo executivo (receita, stock crítico, transações), gestão de Lojas, catálogo de Produtos, gestão de Inventário, ponto de Vendas, diretório de Funcionários e Relatórios analíticos. Usa uma navegação lateral escura com ícones e uma barra superior com pesquisa global e seletor de loja.

### CONTEXTO

*Requisitos funcionais e não funcionais do sistema BelaVista, incluindo os perfis de utilizador (Diretor, Gestor de Loja, Funcionário) e as principais funcionalidades de cada secção*

### MODELO

*Stitch (Google)*

### AValiação Crítica

O Stitch gerou ecrãs de alta-fidelidade para todas as secções pedidas, com uma estrutura visual coerente e próxima do pretendido. A principal limitação foi a falta de controlo sobre detalhes visuais como paletas de cor e tipografia, que resultaram de decisões automáticas da ferramenta. A exportação posterior para Figma introduziu algumas inconsistências de formatação — espaçamentos e alinhamentos sofreram ligeiras alterações no processo de importação.

## 11. Especificação da API

O sistema BelaVista expõe duas APIs REST distintas, correspondentes aos dois contextos operacionais definidos na arquitetura: a **API Central**, consumida pelos administradores da cadeia, e a **API Local**, consumida pelos funcionários e gerentes de cada loja. Esta separação é um reflexo direto das decisões de modelação descritas na secção anterior — cada API expõe exclusivamente as operações relevantes para o perfil de utilizador que a consome, evitando que um funcionário local tenha visibilidade sobre operações de gestão da cadeia, e vice-versa.

Ambas as APIs seguem o estilo arquitectural **REST**, com comunicação em **JSON** e autenticação por **HTTP Basic Auth** — o identificador e a palavra-passe do funcionário são enviados no cabeçalho de cada pedido. Todas as respostas de erro seguem uma estrutura uniforme com os campos `status`, `erro` e `mensagem`, e os códigos de estado HTTP convencionais: 200 para sucesso, 400 para pedido inválido, 401 para não autenticado, 403 para sem permissão, 404 para recurso não encontrado e 409 para conflito de estado.

A especificação completa de cada API, incluindo os *schemas* de todos os *request bodies* e *responses*, encontra-se nos ficheiros `api-central.yaml` e `api-local.yaml` disponibilizados em anexo, em formato OpenAPI 3.1. As secções seguintes apresentam uma visão estruturada dos *endpoints* de cada API, organizados por módulo funcional.

### 11.1. API Central

A API Central é instanciada no sistema da cadeia e é acessível exclusivamente a utilizadores com cargo `ADMINISTRADOR_CENTRAL` ou `GERENTE`. Cobre seis módulos funcionais: autenticação, gestão da cadeia e lojas, gestão de funcionários, estatísticas, sincronização e fornecedores.

| Endpoint        | Mé-todo | Descrição                                                                                                                                                                    | Roles   |
|-----------------|---------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| /api/auth/login | POST    | Autentica o utilizador com <i>username</i> e <i>password</i> . Devolve <code>LoginResponse</code> com <code>cargo</code> , <code>lojald</code> e <code>sessionToken</code> . | Público |

Tabela 41: API Central — Módulo de Autenticação

| Endpoint               | Mé-todo | Descrição                                                                                                                                                                        | Roles |
|------------------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| /api/cadeia            | GET     | Lista todas as cadeias registadas.                                                                                                                                               | ADMIN |
| /api/cadeia            | POST    | Cria uma nova cadeia. O nome é enviado como parâmetro de <i>query</i> .                                                                                                          | ADMIN |
| /api/cadeia/{id}/lojas | POST    | Adiciona uma nova loja à cadeia identificada por <code>id</code> . Recebe <code>CriarLojaRequest</code> com <code>nome</code> , <code>morada</code> e <code>localização</code> . | ADMIN |

Tabela 42: API Central — Módulo de Gestão da Cadeia e Lojas



| Endpoint                     | Método | Descrição                                                               | Roles |
|------------------------------|--------|-------------------------------------------------------------------------|-------|
| /api/usuarios                | GET    | Lista todos os funcionários registados na cadeia.                       | ADMIN |
| /api/usuarios                | POST   | Cria um novo funcionário com cargo, <i>username</i> e <i>password</i> . | ADMIN |
| /api/usuarios/{id}           | GET    | Devolve os dados de um funcionário por identificador.                   | ADMIN |
| /api/usuarios/{id}           | PUT    | Edita o nome ou cargo de um funcionário existente.                      | ADMIN |
| /api/usuarios/{id}/loja      | PATCH  | Associa um funcionário a uma loja específica.                           | ADMIN |
| /api/usuarios/{id}/desativar | PATCH  | Desativa um funcionário mantendo o seu histórico de operações (RF1.5).  | ADMIN |

Tabela 43: API Central — Módulo de Gestão de Funcionários

| Endpoint                 | Método | Descrição                                                                                                                                                             | Roles          |
|--------------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| /api/estatisticas/loja   | GET    | Calcula o número de vendas e faturação total de uma loja, filtráveis por <i>dataInicio</i> e <i>dataFim</i> (RF6.1, RF6.2). Devolve <i>EstatisticasLojaResponse</i> . | ADMIN, GERENTE |
| /api/estatisticas/cadeia | GET    | Agrega estatísticas de todas as lojas da cadeia por período, devolvendo <i>EstatisticasCadeiaResponse</i> com detalhe por loja (RF6.3).                               | ADMIN          |

Tabela 44: API Central — Módulo de Estatísticas

| Endpoint           | Método | Descrição                                                                          | Roles |
|--------------------|--------|------------------------------------------------------------------------------------|-------|
| /api/sincronizacao | GET    | Consulta o estado da última sincronização de uma loja (PENDENTE, CONCLUÍDO, ERRO). | ADMIN |

Tabela 45: API Central — Módulo de Sincronização

| Endpoint          | Método | Descrição                                                        | Roles          |
|-------------------|--------|------------------------------------------------------------------|----------------|
| /api/fornecedores | GET    | Lista todos os fornecedores registados na cadeia.                | ADMIN, GERENTE |
| /api/fornecedores | POST   | Cria um novo fornecedor com nome, NIF, morada, contacto e email. | ADMIN, GERENTE |

Tabela 46: API Central — Módulo de Fornecedores

## 11.2. API Local

A API Local é instanciada em cada loja de forma independente e é acessível a todos os funcionários autenticados, com restrições adicionais por cargo para operações sensíveis. Cobre seis módulos: autenticação, vendas, produtos, stock, clientes e sincronização com a central.

| Endpoint        | Método | Descrição                                                                                                   | Roles   |
|-----------------|--------|-------------------------------------------------------------------------------------------------------------|---------|
| /api/auth/login | POST   | Autentica o funcionário na instância local da loja. Devolve LoginResponse com cargo, lojaId e sessionToken. | Público |

Tabela 47: API Local — Módulo de Autenticação

| Endpoint                   | Método | Descrição                                                                                                                             | Roles         |
|----------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------|---------------|
| /api/vendas                | POST   | Regista uma nova venda com as linhas de venda e, opcionalmente, o cliente associado. Atualiza o stock automaticamente (RF5.1, RF4.3). | FUNC, GERENTE |
| /api/vendas/{id}/fatura    | POST   | Emite a fatura associada a uma venda, em formato imprimível e eletrónico (RF5.6).                                                     | FUNC, GERENTE |
| /api/vendas/{id}/devolucao | POST   | Regista a devolução de produtos de uma venda, revertendo o stock correspondente (RF5.7).                                              | FUNC, GERENTE |
| /api/vendas/historico      | GET    | Consulta o histórico de vendas da loja filtrável por lojaId, inicio, fim e categoriaId (RF6.1).                                       | GERENTE       |

Tabela 48: API Local — Módulo de Vendas e Faturação

| Endpoint           | Método | Descrição                                                                            | Roles   |
|--------------------|--------|--------------------------------------------------------------------------------------|---------|
| /api/produtos      | GET    | Lista produtos com filtros opcionais por nome ou código de barras (RF3.4, RF3.5).    | Todos   |
| /api/produtos      | POST   | Cria um novo produto com nome, categoria, preço e fornecedor(es) associados (RF3.1). | GERENTE |
| /api/produtos/{id} | GET    | Devolve os dados de um produto por identificador.                                    | Todos   |
| /api/produtos/{id} | PUT    | Edita os atributos de um produto existente (RF3.2).                                  | GERENTE |

Tabela 49: API Local — Módulo de Produtos

| Endpoint                           | Método | Descrição                                                                                                      | Roles   |
|------------------------------------|--------|----------------------------------------------------------------------------------------------------------------|---------|
| /api/produtos/stock/entrada        | POST   | Regista uma entrada de stock proveniente de um fornecedor, indicando produto, quantidade e fornecedor (RF4.4). | GERENTE |
| /api/produtos/stock/ajuste         | POST   | Realiza um ajuste manual de stock com indicação do motivo (RF4.5).                                             | GERENTE |
| /api/produtos/{id}/stock/historico | GET    | Consulta o histórico de movimentações de stock de um produto — entradas, saídas e devoluções (RF4.5).          | GERENTE |
| /api/lojas/{id}/fornecedores       | POST   | Associa um fornecedor à loja (RF7.3).                                                                          | GERENTE |

Tabela 50: API Local — Módulo de Stock e Fornecedores

| Endpoint           | Método | Descrição                                                                                                           | Roles         |
|--------------------|--------|---------------------------------------------------------------------------------------------------------------------|---------------|
| /api/clientes      | GET    | Lista todos os clientes registados na loja.                                                                         | Todos         |
| /api/clientes      | POST   | Cria um perfil de cliente com nome, NIF, email, telemóvel e morada. O NIF é usado como identificador único (RF8.1). | FUNC, GERENTE |
| /api/clientes/{id} | GET    | Devolve os dados de um cliente por identificador.                                                                   | Todos         |
| /api/clientes/{id} | PUT    | Edita os dados de um cliente registado (RF8.2).                                                                     | FUNC, GERENTE |

Tabela 51: API Local — Módulo de Clientes

| Endpoint                | Método | Descrição                                                                                                                                                          | Roles   |
|-------------------------|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| /api/sincronizacao/push | POST   | Envia os eventos de venda pendentes da loja para consolidação na cadeia central. Inclui um hash por evento para garantir idempotência em caso de reenvio (RNF4.1). | Sistema |

Tabela 52: API Local — Módulo de Sincronização

### 11.3. Contratos de Dados Relevantes

Os *schemas* completos encontram-se nos ficheiros OpenAPI em anexo. São aqui destacados os contratos mais relevantes para a compreensão dos fluxos principais do sistema.

| Schema                      | Contexto        | Campos principais                                                                                |
|-----------------------------|-----------------|--------------------------------------------------------------------------------------------------|
| LoginResponse               | Central e Local | funcionarioId, nome, cargo (FUNCIONARIO   GERENTE   ADMINISTRADOR_CENTRAL), lojaId, sessionToken |
| LojaResponse                | Central         | id, nome, cidade, cadeiaId, cadeiaNome                                                           |
| EstatisticasLojaResponse    | Central         | lojaId, lojaNome, totalVendas, faturacaoTotal, dataInicio, dataFim                               |
| EstatisticasCadeiaResponse  | Central         | cadeiaId, cadeiaNome, totalVendas, faturacaoTotal, porLoja[]                                     |
| VendaResponse               | Local           | id, lojaId, dataHora, total, estado (PENDENTE   CONCLUIDA   DEVOLVIDA), itens[]                  |
| StockResponse               | Local           | produtoId, produtoNome, quantidade, quantidadeMinima, abaixoMinimo, precoVenda                   |
| SincronizacaoPushRequest    | Local → Central | lojaId, eventos[] com tipo (VENDA   DEVOLUCAO   AJUSTE_STOCK), hash, payload                     |
| SincronizacaoStatusResponse | Central         | lojaId, estado (PENDENTE   CONCLUIDO   ERRO), totalEventos, eventosProcessados, erros            |

Tabela 53: Principais *schemas* de dados das APIs Central e Local

## 12. Guia de Utilização

### BELAVISTA — Guia de Instalação, Operação e Manutenção

Sistema de Gestão de Cadeia de Lojas de Conveniência  
LI4 · Grupo 9 · Universidade do Minho · 2025/2026

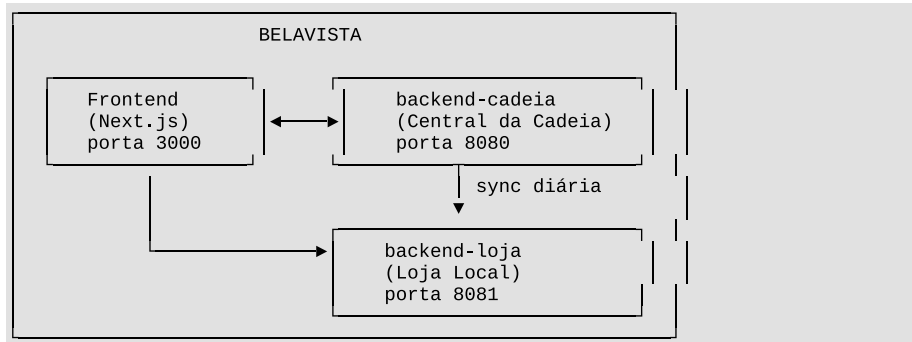
---

#### Índice

1. [Visão Geral do Sistema](#)
2. [Requisitos de Sistema](#)
3. [Instalação](#)
  - 3.1 [Clonar o Repositório](#)
  - 3.2 [Compilar os Backends](#)
  - 3.3 [Instalar Dependências do Frontend](#)
4. [Configuração](#)
  - 4.1 [backend-cadeia](#)
  - 4.2 [backend-loja](#)
  - 4.3 [Frontend](#)
5. [Iniciar o Sistema](#)
6. [Credenciais de Acesso](#)
7. [Guia de Operação](#)
  - 7.1 [Interface Web \(Frontend\)](#)
  - 7.2 [Perfis de Utilizador](#)
  - 7.3 [Fluxos Principais](#)
8. [API REST — Referência Rápida](#)
  - 8.1 [backend-cadeia \(porta 8080\)](#)
  - 8.2 [backend-loja \(porta 8081\)](#)
9. [Sincronização Cadeia ↔ Loja](#)
10. [Base de Dados](#)
11. [Manutenção](#)
  - 11.1 [Logs](#)
  - 11.2 [Backup](#)
  - 11.3 [Reinício dos Serviços](#)
  - 11.4 [Atualização do Sistema](#)

## 1. Visão Geral do Sistema

O **BELAVISTA** é um sistema distribuído de gestão para uma cadeia de lojas de conveniência, composto por três componentes independentes que comunicam entre si:



| Componente     | Tecnologia               | Porta | Responsabilidade                                                                                 |
|----------------|--------------------------|-------|--------------------------------------------------------------------------------------------------|
| backend-cadeia | Spring Boot 3.4 + Kotlin | 8080  | Gestão central: lojas, fornecedores, produtos do catálogo, funcionários, relatórios consolidados |
| backend-loja   | Spring Boot 3.4 + Kotlin | 8081  | Gestão local: vendas, stock, clientes, devoluções, faturas                                       |
| frontend       | Next.js 16 + TypeScript  | 3000  | Interface web para todos os utilizadores                                                         |

**Base de dados:** SQLite (ficheiros `cadeia.db` e `loja.db` na raiz de cada backend).

## 2. Requisitos de Sistema

### Software obrigatório

| Software   | Versão mínima | Verificação                |
|------------|---------------|----------------------------|
| Java (JDK) | 21            | <code>java -version</code> |
| Node.js    | 18 LTS        | <code>node -v</code>       |
| npm        | 9             | <code>npm -v</code>        |

### Software opcional (para desenvolvimento/rebuild)

| Software | Uso                                               |
|----------|---------------------------------------------------|
| Gradle   | Recompilar os backends (o projeto inclui gradlew) |
| Git      | Clonar e atualizar o repositório                  |

### Recursos de hardware (mínimos)

| Recurso | Mínimo recomendado                     |
|---------|----------------------------------------|
| RAM     | 1 GB disponível                        |
| Disco   | 500 MB                                 |
| CPU     | Dual-core                              |
| Rede    | Comunicação local entre as três portas |

### Sistema operativo

Compatível com **Linux**, **macOS** e **Windows** (com WSL ou PowerShell).

## 3. Instalação

### 3.1 Clonar o Repositório

```
git clone <url-do-repositorio>
cd LI4
```

### 3.2 Compilar os Backends

**Nota:** Os ficheiros JAR já compilados encontram-se em `backend-cadeia/build/libs/` e `backend-loja/build/libs/`. Se não for necessário recompilar, avance para a secção 5.

Para recompilar (requer Java 21):

```
# Na raiz do projeto
./gradlew :backend-cadeia:bootJar :backend-loja:bootJar
```

No Windows (sem WSL):

```
gradlew.bat :backend-cadeia:bootJar :backend-loja:bootJar
```

Os JARs são gerados em:

- `backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar`
- `backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar`

)

Continua no **Anexo 1...**

13. Planeamento Inicial VS Execução Final

Relembrando o diagrama, demonstrado na secção “1.8. Planeamento”, que representa o planeamento inicial do projeto, vai servir de ponto de referência para a análise que iremos fazer ao nosso trabalho assistido por LLM.

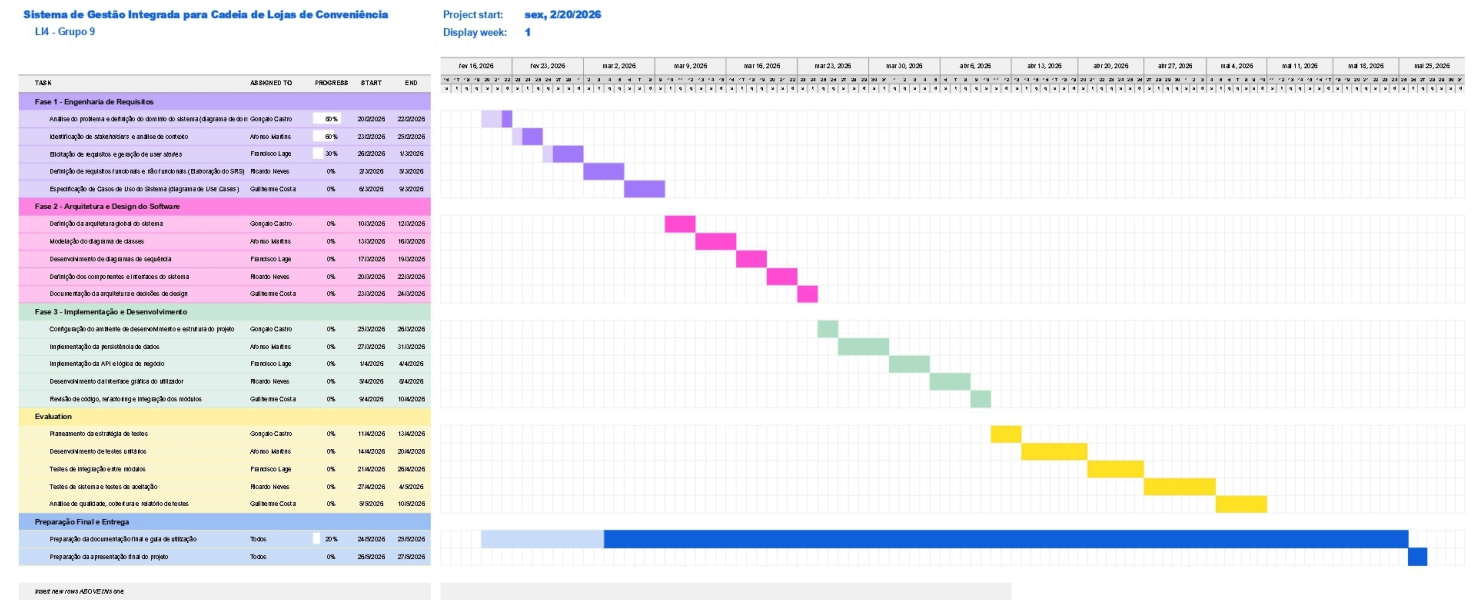


Figura 31: Diagrama de Gantt com o planeamento temporal

Concluído o projeto, é possível contrastar o planeamento inicial com a execução efetivamente registada no diagrama de Gantt final. A figura seguinte denota esse mesmo diagrama:

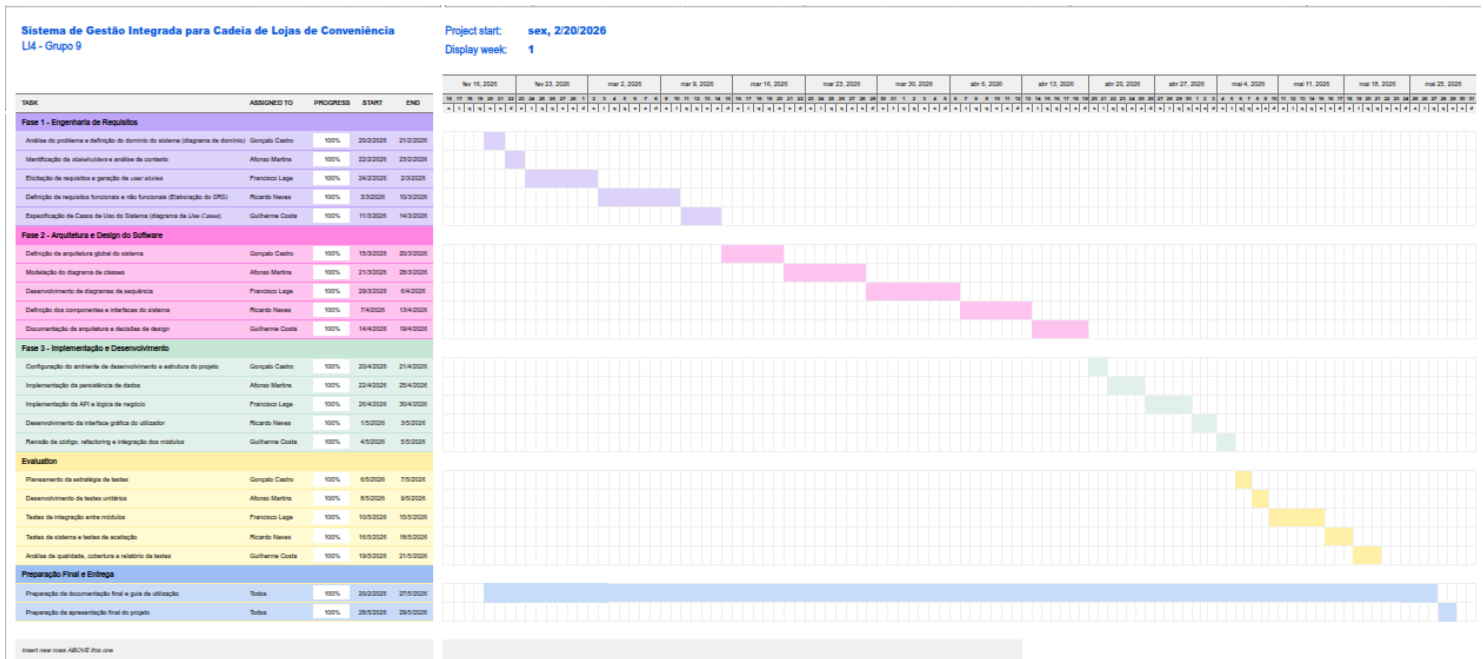


Figura 32: Diagrama de Gantt com a execução final do projeto



O desvio mais expressivo concentrou-se na **Fase 2 — Arquitetura e Design do Software**, com um prolongamento de aproximadamente quatro semanas face ao previsto. A causa principal foi a complexidade da separação das duas lógicas — central e local — e a consequente necessidade de iterar sobre os modelos de classes, as interfaces segregadas, o mecanismo de sincronização e a documentação das decisões de *design*. Estas são tarefas com elevada carga de raciocínio e revisão mútua entre membros e docentes, difíceis de comprimir sem comprometer a coerência dos resultados produzidos.

Em sentido inverso, a **Fase 3 — Implementação e Desenvolvimento** foi concluída com antecedência face ao inicialmente previsto. O recurso sistemático a ferramentas de LLM ao longo do desenvolvimento — para geração de código, sugestão de estruturas de dados, *scaffolding* de DAOs e serviços, e resolução de problemas de integração entre os dois *backends* — reduziu significativamente o tempo necessário para traduzir os modelos de *design* em código funcional. Tarefas que, num fluxo de desenvolvimento tradicional, consumiriam vários dias de implementação manual foram concluídas em horas, libertando capacidade da equipa para revisão, testes e refinamento. Este resultado constitui uma das principais evidências do valor do auxílio por LLM no desenvolvimento de software: o impacto não se faz sentir nas fases de análise e *design* — que continuam a exigir raciocínio humano e iteração — mas sim na fase de concretização, onde a capacidade de gerar e adaptar código rapidamente a partir de especificações bem definidas acelera de forma mensurável o ritmo de entrega.

Em retrospectiva, a adoção de uma metodologia com revisões intercaladas — por exemplo, *checkpoints* semanais com validação dos artefactos de arquitetura — teria permitido detetar e acomodar o desvio da Fase 2 mais cedo, sem o efeito de propagação linear. Ainda assim, a estrutura sequencial adotada mostrou-se adequada para garantir consistência entre os artefactos de cada fase, dado que cada diagrama e cada decisão de *design* dependem directamente dos anteriores.

## 14. Conclusões e Trabalho Futuro

### 14.1. Conclusões

O desenvolvimento do Sistema de Gestão BelaVista permitiu concretizar, de forma integrada, as quatro etapas do ciclo de vida do *software* definidas no enunciado — Engenharia de Requisitos, Arquitectura e *Design*, Implementação e Verificação e Validação — com recurso sistemático a LLM como agentes de apoio em cada uma delas. O resultado é um sistema funcional que cobre o âmbito operacional proposto: cada loja opera de forma autónoma com o seu *backend* e base de dados local, os dados são consolidados diariamente no sistema central, e os administradores da cadeia dispõem de relatórios, estatísticas e controlo de acessos adequados às suas necessidades de gestão.

Do ponto de vista técnico, a decisão arquitectural mais determinante foi a separação em duas lógicas independentes — LojaLN e CadeiaLN — com interfaces segregadas por contexto e comunicação reduzida a um único ponto de contacto explícito via API REST. Esta separação, inicialmente mais custosa em termos de planeamento e modelação, revelou-se a mais correcta: garantiu autonomia operacional por loja, isolamento entre contextos e uma base de código coesa e testável de forma independente. A complexidade desta decisão foi também a principal causa do desvio observado na Fase 2, cujo prolongamento se propagou às fases seguintes — uma consequência típica de uma metodologia sequencial quando as fases de design são subestimadas.

Em sentido inverso, a Fase 3 — Implementação — foi concluída com antecedência, evidenciando o impacto mais tangível do recurso a LLM: a capacidade de gerar código estruturado a partir de especificações bem definidas reduziu significativamente o tempo de concretização, libertando a equipa para revisão, integração e testes. Esta experiência reforça uma conclusão que o próprio enunciado antecipa: **o valor dos LLM no desenvolvimento de *software* não reside na substituição do raciocínio de engenharia** — que continua a ser indispensável nas fases de análise, arquitectura e validação crítica — **mas na aceleração das tarefas de concretização quando esse raciocínio já foi aplicado com rigor.**

A avaliação crítica do contributo dos LLM, documentada ao longo de cada secção do relatório, permite concluir que a sua integração é mais eficaz quando precedida de especificações claras e contexto bem delimitado. *Prompts* vagos ou sem contexto produziram resultados genéricos que exigiram revisão significativa; em contraste, *prompts* estruturados com artefactos concretos — diagramas, requisitos, exemplos de código — produziram contributos directamente utilizáveis com iterações mínimas algumas das vezes. Esta observação é consistente com a perspectiva do enunciado: os LLM funcionam como agentes de suporte inteligente cuja utilidade depende da qualidade do conhecimento de Engenharia de *Software* que os orienta.

### 14.2. Trabalho Futuro

O sistema desenvolvido constitui uma base sólida sobre a qual diversas extensões são viáveis e desejáveis:

- **Sincronização em tempo real:** a arquitectura actual suporta consolidação diária; uma evolução natural seria a adopção de *event streaming* (por exemplo, Apache Kafka) para aproximar os dados do central em tempo real, sem comprometer a autonomia local.

- **Algoritmo preditivo de reposição de *stock*:** integração de modelos de *machine learning* que analisem padrões de vendas históricos e condições externas (sazonalidade, meteorologia, eventos locais) para sugerir automaticamente encomendas aos fornecedores.
- **Aplicação móvel para clientes:** desenvolvimento de uma interface mobile orientada aos clientes frequentes, com suporte a programas de fidelização, histórico de compras e notificações de promoções personalizadas.
- **Dashboard central em tempo real:** substituição dos relatórios gerados por batch por um painel de controlo dinâmico que apresente métricas de desempenho da cadeia com actualização contínua, acessível aos administradores a partir de qualquer dispositivo.
- **Expansão do modelo de testes:** cobertura de testes de carga e stress sobre o mecanismo de sincronização, validando o comportamento do sistema em cenários de falha de rede, reenvios múltiplos e consolidações concorrentes de várias lojas.
- **Deploy em Produção:** quanto a este ponto importante do Ciclo de Vida do *Software* a ideia inicial era chegarmos a uma fase final e experimentarmos a fase de produção propriamente dita (numa máquina na cloud gratuita, ainda que pudesse ser lenta). Em conjunto com a tecnologia **Docker** que alguns membros da equipa já tinham alguma experiência podia ter valorizado o nosso trabalho nesse sentido.

## Referências

- MANFRINI, Luiza, 2023. **Modelo em V como ferramenta para gestão de projetos**. LinkedIn. Disponível em: <https://pt.linkedin.com/pulse/modelo-em-v-como-ferramenta-para-gest%C3%A3o-de-projetos-luiza-manfrini> [Acesso em: 27 maio 2026].
- MOTA, Ana Paula, 2015. **Waterfall Model**. Disponível em: <https://anapaulamota.me/wp-content/uploads/2015/12/waterfallmodel.jpg> [Acesso em: 27 maio 2026].

## Lista de Siglas e Acrónimos

**BD** Base de Dados

**IA** Inteligência Artificial

**LLM** Large Language Models

**RGPD** Regulamento Geral sobre a Proteção de Dados

## **Anexos**

### **Anexo 1: Guia Completo do Sistema**

# BELAVISTA — Guia de Instalação, Operação e Manutenção

Sistema de Gestão de Cadeia de Lojas de Conveniência  
LI4 · Grupo 9 · Universidade do Minho · 2025/2026

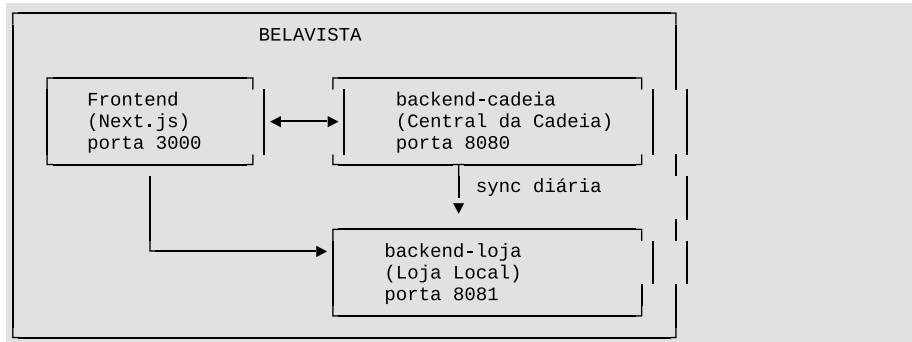
---

## Índice

1. [Visão Geral do Sistema](#)
2. [Requisitos de Sistema](#)
3. [Instalação](#)
  - 3.1 [Clonar o Repositório](#)
  - 3.2 [Compilar os Backends](#)
  - 3.3 [Instalar Dependências do Frontend](#)
4. [Configuração](#)
  - 4.1 [backend-cadeia](#)
  - 4.2 [backend-loja](#)
  - 4.3 [Frontend](#)
5. [Iniciar o Sistema](#)
6. [Credenciais de Acesso](#)
7. [Guia de Operação](#)
  - 7.1 [Interface Web \(Frontend\)](#)
  - 7.2 [Perfis de Utilizador](#)
  - 7.3 [Fluxos Principais](#)
8. [API REST — Referência Rápida](#)
  - 8.1 [backend-cadeia \(porta 8080\)](#)
  - 8.2 [backend-loja \(porta 8081\)](#)
9. [Sincronização Cadeia ↔ Loja](#)
10. [Base de Dados](#)
11. [Manutenção](#)
  - 11.1 [Logs](#)
  - 11.2 [Backup](#)
  - 11.3 [Reinício dos Serviços](#)
  - 11.4 [Atualização do Sistema](#)

## 1. Visão Geral do Sistema

O **BELAVISTA** é um sistema distribuído de gestão para uma cadeia de lojas de conveniência, composto por três componentes independentes que comunicam entre si:



| Componente     | Tecnologia               | Porta | Responsabilidade                                                                                 |
|----------------|--------------------------|-------|--------------------------------------------------------------------------------------------------|
| backend-cadeia | Spring Boot 3.4 + Kotlin | 8080  | Gestão central: lojas, fornecedores, produtos do catálogo, funcionários, relatórios consolidados |
| backend-loja   | Spring Boot 3.4 + Kotlin | 8081  | Gestão local: vendas, stock, clientes, devoluções, faturas                                       |
| frontend       | Next.js 16 + TypeScript  | 3000  | Interface web para todos os utilizadores                                                         |

**Base de dados:** SQLite (ficheiros `cadeia.db` e `loja.db` na raiz de cada backend).

## 2. Requisitos de Sistema

### Software obrigatório

| Software   | Versão mínima | Verificação                |
|------------|---------------|----------------------------|
| Java (JDK) | 21            | <code>java -version</code> |
| Node.js    | 18 LTS        | <code>node -v</code>       |
| npm        | 9             | <code>npm -v</code>        |



### Software opcional (para desenvolvimento/rebuild)

| Software | Uso                                               |
|----------|---------------------------------------------------|
| Gradle   | Recompilar os backends (o projeto inclui gradlew) |
| Git      | Clonar e atualizar o repositório                  |

### Recursos de hardware (mínimos)

| Recurso | Mínimo recomendado                     |
|---------|----------------------------------------|
| RAM     | 1 GB disponível                        |
| Disco   | 500 MB                                 |
| CPU     | Dual-core                              |
| Rede    | Comunicação local entre as três portas |

### Sistema operativo

Compatível com **Linux**, **macOS** e **Windows** (com WSL ou PowerShell).

## 3. Instalação

### 3.1 Clonar o Repositório

```
git clone <url-do-repositorio>
cd LI4
```

### 3.2 Compilar os Backends

**Nota:** Os ficheiros JAR já compilados encontram-se em `backend-cadeia/build/libs/` e `backend-loja/build/libs/`. Se não for necessário recompilar, avance para a secção 5.

Para recompilar (requer Java 21):

```
# Na raiz do projeto
./gradlew :backend-cadeia:bootJar :backend-loja:bootJar
```

No Windows (sem WSL):

```
gradlew.bat :backend-cadeia:bootJar :backend-loja:bootJar
```

Os JARs são gerados em:

- `backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar`
- `backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar`

### 3.3 Instalar Dependências do Frontend

```
cd frontend
npm install
```

## 4. Configuração

### 4.1 backend-cadeia

Ficheiro: backend-cadeia/src/main/resources/application.yml

```
spring:
  datasource:
    url: jdbc:sqlite:./cadeia.db      # caminho para a base de dados
  jpa:
    hibernate:
      ddl-auto: create               # "create" reinicia o schema; usar
      "update" em produção
server:
  port: 8080                         # porta do backend-cadeia
loja:
  url: http://localhost:8081         # URL do backend-loja
```

**Importante:** `ddl-auto: create` apaga e recria as tabelas a cada arranque.  
Em produção ou para preservar dados, alterar para `ddl-auto: update`.

### 4.2 backend-loja

Ficheiro: backend-loja/src/main/resources/application.yml

```
spring:
  datasource:
    url: jdbc:sqlite:./loja.db       # caminho para a base de dados
  jpa:
    hibernate:
      ddl-auto: create               # alterar para "update" em produção
server:
  port: 8081                         # porta do backend-loja
loja:
  id: 1                              # identificador único desta loja
cadeia:
  url: http://localhost:8080         # URL do backend-cadeia
```

Para instalar o sistema em **múltiplas lojas físicas**, cada instância do `backend-loja` deve ter um `loja.id` diferente e apontar `cadeia.url` para o servidor central correto.

### 4.3 Frontend

Ficheiro: frontend/next.config.ts

```
// As chamadas ao frontend são reencaminhadas para os backends:  
"/loja-api/*" → http://localhost:8081/api/* (backend-loja)  
"/cadeia-api/*" → http://localhost:8080/api/* (backend-cadeia)
```

Se os backends estiverem em servidores diferentes, alterar os valores de `destination` neste ficheiro.

## 5. Iniciar o Sistema

Os três componentes devem ser iniciados **pela seguinte ordem**:

### Passo 1 — Iniciar o backend-cadeia

```
java -Xmx256m -jar backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar
```

Aguardar até aparecer no terminal:

```
Started CadeiaApplication in X.XXX seconds
```

### Passo 2 — Iniciar o backend-loja

Abrir um **novo terminal** e executar:

```
java -Xmx256m -jar backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar
```

Aguardar até aparecer:

```
Started LojaApplication in X.XXX seconds
```

### Passo 3 — Iniciar o Frontend

Abrir um **terceiro terminal**:

```
cd frontend  
  
# Modo produção (recomendado):  
npm run build  
npm run start  
  
# Modo desenvolvimento:  
npm run dev
```

## Verificação

Após iniciar todos os componentes, aceder a:

| URL                                   | Componente | Esperado                  |
|---------------------------------------|------------|---------------------------|
| http://localhost:3000                 | Frontend   | Página de login BELAVISTA |
| http://localhost:8080/swagger-ui.html | API Cadeia | Documentação Swagger      |
| http://localhost:8081/swagger-ui.html | API Loja   | Documentação Swagger      |

---

## 6. Credenciais de Acesso

### backend-cadeia (Sede da Cadeia)

| Número | Senha      | Perfil        |
|--------|------------|---------------|
| ADM001 | admin123   | Administrador |
| GER001 | gerente123 | Gestor        |

### backend-loja (Loja Local)

| Número | Senha      | Perfil        |
|--------|------------|---------------|
| ADM001 | admin123   | Administrador |
| GER001 | gerente123 | Gestor        |
| FUN001 | func123    | Funcionário   |
| FUN002 | func123    | Funcionário   |

**Segurança:** As senhas estão armazenadas em texto simples na base de dados.  
Em ambiente de produção real, devem ser substituídas por hashes (ex.: bcrypt).

---

## 7. Guia de Operação

### 7.1 Interface Web (Frontend)

Aceder a **http://localhost:3000** num browser moderno (Chrome, Firefox, Edge).

O sistema deteta automaticamente se o utilizador pertence à **cadeia** ou à **loja** e apresenta o dashboard correspondente.

### 7.2 Perfis de Utilizador

#### **Administrador da Cadeia (ADM)**

- Gerir lojas da cadeia (criar, consultar)
- Gerir funcionários
- Gerir fornecedores
- Gerir catálogo de produtos central
- Consultar estatísticas consolidadas
- Gerar relatórios da cadeia
- Consultar sincronizações

#### ***Gestor da Cadeia (GER)***

- Consultar estatísticas e relatórios
- Consultar lojas e fornecedores

#### ***Administrador da Loja (ADM)***

- Gerir stock local (entradas, ajustes)
- Gerir funcionários da loja
- Gerir clientes
- Consultar vendas e histórico
- Processar devoluções
- Forçar sincronização com a cadeia

#### ***Gestor da Loja (GER)***

- Consultar stock e produtos
- Consultar vendas e histórico

#### ***Funcionário da Loja (FUN)***

- Registrar vendas
- Emitir faturas
- Registrar devoluções
- Consultar produtos e stock

### **7.3 Fluxos Principais**

#### ***Registrar uma Venda***

1. Autenticar como Funcionário ou Gestor da Loja
2. Navegar para **Vendas** → **Nova Venda**
3. Selecionar cliente (ou venda anónima)
4. Adicionar linhas de venda (produto + quantidade)
5. Confirmar a venda
6. Emitir fatura se necessário

#### ***Gerir Stock***

1. Autenticar como Administrador da Loja
2. Navegar para **Stock**
3. Para entrada de mercadoria: **Entrada de Stock** → produto + quantidade

4. Para ajuste de inventário: **Ajuste de Stock** → produto + nova quantidade

#### **Registrar uma Devolução**

1. Autenticar como Funcionário ou superior
2. Navegar para **Vendas** → localizar a venda
3. Selecionar **Registrar Devolução**
4. Indicar os itens a devolver

#### **Consultar Relatórios Consolidados (Cadeia)**

1. Autenticar na cadeia como ADM ou GER
2. Navegar para **Central** → **Relatórios**
3. Criar novo relatório ou consultar os existentes
4. Consultar estatísticas por loja

#### **Adicionar uma Nova Loja à Cadeia**

1. Autenticar na cadeia como ADM
2. Navegar para **Cadeia** → **Lojas** → **Nova Loja**
3. Preencher nome e localização
4. Configurar uma nova instância do backend - loja com o `loja.id` correspondente

---

## **8. API REST — Referência Rápida**

A documentação interativa completa está disponível via Swagger:

- Cadeia: <http://localhost:8080/swagger-ui.html>
- Loja: <http://localhost:8081/swagger-ui.html>

### **8.1 backend-cadeia (porta 8080)**

| Método | Endpoint                     | Descrição                  |
|--------|------------------------------|----------------------------|
| POST   | /api/auth/login              | Autenticação               |
| GET    | /api/cadeia                  | Obter informação da cadeia |
| POST   | /api/cadeia                  | Criar cadeia               |
| POST   | /api/cadeia/{id}/lojas       | Associar loja à cadeia     |
| GET    | /api/lojas                   | Listar todas as lojas      |
| GET    | /api/lojas/{id}              | Obter loja por ID          |
| GET    | /api/lojas/{id}/estatisticas | Estatísticas de uma loja   |

| Método | Endpoint                     | Descrição                                                   |
|--------|------------------------------|-------------------------------------------------------------|
| POST   | /api/lojas/{id}/fornecedores | Associar fornecedor a loja                                  |
| POST   | /api/lojas/{id}/sync         | Forçar sync de uma loja                                     |
| GET    | /api/fornecedores            | Listar fornecedores                                         |
| GET    | /api/fornecedores/{id}       | Obter fornecedor                                            |
| POST   | /api/fornecedores            | Criar fornecedor                                            |
| GET    | /api/produtos                | Listar produtos do catálogo                                 |
| GET    | /api/produtos/{id}           | Obter produto                                               |
| POST   | /api/produtos                | Criar produto                                               |
| PUT    | /api/produtos/{id}           | Atualizar produto                                           |
| GET    | /api/usuarios                | Listar funcionários                                         |
| GET    | /api/usuarios/{id}           | Obter funcionário                                           |
| POST   | /api/usuarios                | Criar funcionário                                           |
| PUT    | /api/usuarios/{id}           | Atualizar funcionário                                       |
| PATCH  | /api/usuarios/{id}/loja      | Atribuir loja ao funcionário                                |
| GET    | /api/central/estatisticas    | Estatísticas consolidadas                                   |
| POST   | /api/central/relatorios      | Gerar relatório                                             |
| GET    | /api/central/relatorios      | Listar relatórios                                           |
| GET    | /api/central/sincronizacoes  | Listar sincronizações                                       |
| POST   | /api/sync/importar           | Receber dados de sincronização ( <i>chamado pela loja</i> ) |

## 8.2 backend-loja (porta 8081)

| Método | Endpoint                       | Descrição              |
|--------|--------------------------------|------------------------|
| POST   | /api/auth/login                | Autenticação           |
| GET    | /api/produtos                  | Listar produtos locais |
| GET    | /api/produtos/{id}             | Obter produto          |
| GET    | /api/stock                     | Listar stock           |
| GET    | /api/stock/produto/{produtoId} | Stock de um produto    |

| Método | Endpoint                   | Descrição                       |
|--------|----------------------------|---------------------------------|
| POST   | /api/stock/entrada         | Registrar entrada de stock      |
| POST   | /api/stock/ajuste          | Ajustar stock                   |
| GET    | /api/clientes              | Listar clientes                 |
| GET    | /api/clientes/{id}         | Obter cliente                   |
| POST   | /api/clientes              | Criar cliente                   |
| PUT    | /api/clientes/{id}         | Atualizar cliente               |
| GET    | /api/usuarios              | Listar funcionários             |
| GET    | /api/usuarios/{id}         | Obter funcionário               |
| POST   | /api/usuarios              | Criar funcionário               |
| PUT    | /api/usuarios/{id}         | Atualizar funcionário           |
| POST   | /api/vendas                | Registrar venda                 |
| POST   | /api/vendas/{id}/fatura    | Emitir fatura                   |
| POST   | /api/vendas/{id}/devolucao | Registrar devolução             |
| GET    | /api/vendas/historico      | Histórico de vendas             |
| POST   | /api/vendas/sync           | Forçar sincronização com cadeia |

## 9. Sincronização Cadeia ↔ Loja

O backend-loja envia automaticamente um resumo das vendas do dia para o backend-cadeia.

### Sincronização Automática

- **Horário:** Todos os dias às 23:55
- **Dados enviados:** total de vendas, número de transações, produtos vendidos

### Sincronização Manual

Via API:

POST `http://localhost:8081/api/vendas/sync`

Via interface web:

- Autenticar como ADM na loja → **Sync com Cadeia**



### O que é sincronizado

- Totais financeiros das vendas do dia
- Número de transações
- Os dados detalhados (linhas de venda, clientes) **não** são enviados — ficam apenas na loja

## 10. Base de Dados

### Localização dos ficheiros

| Ficheiro  | Localização                  | Conteúdo                 |
|-----------|------------------------------|--------------------------|
| cadeia.db | backend-cadeia/<br>cadeia.db | Dados centrais da cadeia |
| loja.db   | backend-loja/loja.db         | Dados locais da loja     |

### Tabelas principais — cadeia.db

| Tabela             | Descrição                           |
|--------------------|-------------------------------------|
| cadeia             | Informação da cadeia                |
| loja               | Lojas registadas                    |
| produto            | Catálogo de produtos central        |
| categoria          | Categorias de produtos              |
| fornecedor         | Fornecedores                        |
| funcionario        | Funcionários da cadeia              |
| relatorio          | Relatórios gerados                  |
| sincronizacao_loja | Registos de sincronização recebidos |

### Tabelas principais — loja.db

| Tabela         | Descrição                    |
|----------------|------------------------------|
| produto        | Produtos disponíveis na loja |
| stock          | Níveis de stock por produto  |
| cliente        | Clientes registados          |
| funcionario    | Funcionários da loja         |
| venda          | Vendas realizadas            |
| linha_de_venda | Linhas de cada venda         |
| fatura         | Faturas emitidas             |
| devolucao      | Devoluções registadas        |

| Tabela              | Descrição             |
|---------------------|-----------------------|
| historico_de_vendas | Histórico consolidado |

## Acesso direto à base de dados (SQLite)

```
# Instalar o cliente SQLite
# Ubuntu/Debian: sudo apt install sqlite3
# macOS: brew install sqlite

# Consultar a base de dados da cadeia
sqlite3 backend-cadeia/cadeia.db

# Exemplos de queries
.tables -- listar tabelas
SELECT * FROM funcionario; -- ver todos os funcionários
SELECT * FROM sincronizacao_loja ORDER BY data DESC LIMIT 10; -- últimas sync
.quit
```

## 11. Manutenção

### 11.1 Logs

Os logs são apresentados no terminal onde cada processo foi iniciado.

**Nível de log por defeito:** DEBUG para com.li4.\*

Para reduzir a verbosidade em produção, alterar no `application.yml`:

```
logging:
  level:
    com.li4: INFO
```

Para guardar logs em ficheiro:

```
# Redirecionar output para ficheiro
java -Xmx256m -jar backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar \
  > logs/cadeia-$(date +%Y%m%d).log 2>&1 &

java -Xmx256m -jar backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar \
  > logs/loja-$(date +%Y%m%d).log 2>&1 &
```

### 11.2 Backup

**Backup manual das bases de dados:**

```
# Criar diretório de backups
mkdir -p backups

# Backup da cadeia
cp backend-cadeia/cadeia.db backups/cadeia-$(date +%Y%m%d-%H%M%S).db

# Backup da loja
cp backend-loja/loja.db backups/loja-$(date +%Y%m%d-%H%M%S).db
```

Fazer o backup com os backends **parados** ou usando o comando SQLite `.backup`:

```
sqlite3 backend-cadeia/cadeia.db ".backup backups/cadeia-backup.db"
```

**Recomendação:** Agendar backups diários (ex.: via `CRON` no Linux):

```
# Editar crontab: crontab -e
# Backup todos os dias às 02:00
0 2 * * * cp /caminho/para/LI4/backend-cadeia/cadeia.db /backups/cadeia-$(date +\%Y\%m\%d).db
0 2 * * * cp /caminho/para/LI4/backend-loja/loja.db /backups/loja-$(date +\%Y\%m\%d).db
```

### 11.3 Reinício dos Serviços

**Parar os serviços:**

```
# Encontrar os PIDs
ps aux | grep "backend-cadeia\|backend-loja"

# Terminar os processos
kill <PID-cadeia>
kill <PID-loja>
```

**Reiniciar:**

```
# Reiniciar pela ordem correta: cadeia → loja → frontend
java -Xmx256m -jar backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar &
sleep 10
java -Xmx256m -jar backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar &
cd frontend && npm run start &
```

### 11.4 Atualização do Sistema

1. Parar todos os serviços
2. Fazer backup das bases de dados (ver 11.2)
3. Atualizar o código:

```
git pull origin main
```

4. Recompilar os backends:

```
./gradlew :backend-cadeia:bootJar :backend-loja:bootJar
```

5. Atualizar dependências do frontend:

```
cd frontend && npm install
npm run build
```

6. Alterar `ddl-auto` para `update` nos ficheiros `application.yml` de ambos os backends (para preservar dados existentes)
  7. Reiniciar os serviços (ver 11.3)
-

## 12. Resolução de Problemas (Troubleshooting)

### O backend não inicia — Port 8080 already in use

```
# Identificar o processo que usa a porta
lsof -i :8080      # Linux/macOS
netstat -ano | findstr :8080  # Windows

# Terminar o processo
kill -9 <PID>
```

### Erro de ligação entre loja e cadeia

Verificar:

1. O backend-cadeia está em execução (<http://localhost:8080/swagger-ui.html> acessível)
2. `cadeia.url` em `backend-loja/application.yml` aponta para o endereço correto
3. Firewall não bloqueia a porta 8080

### Base de dados apagada a cada reinício

O `ddl-auto: create` **apaga e recria** o schema a cada arranque. Para preservar dados:

```
# Em application.yml (ambos os backends)
jpa:
  hibernate:
    ddl-auto: update  # ← alterar de "create" para "update"
```

### Frontend não consegue comunicar com os backends

Verificar `frontend/next.config.ts`:

```
destination: "http://localhost:8081/api/:path*" // IP/porta corretos?
destination: "http://localhost:8080/api/:path*" // IP/porta corretos?
```

Após alterar o ficheiro, recompilar o frontend: `npm run build && npm run start`

### Erro de autenticação — 401 Unauthorized

- Verificar que o número e senha estão corretos (ver secção 6)
- Confirmar que o backend correto está a ser usado (cadeia vs. loja)
- Os dados são reiniciados se `ddl-auto: create` estiver activo — as credenciais por defeito são restauradas

### Sincronização falha

1. Verificar que ambos os backends estão em execução
2. Verificar os logs do `backend-loja` para mensagens de erro
3. Forçar sync manual: `POST http://localhost:8081/api/vendas/sync`
4. Verificar em `backend-cadeia`: `GET`

`http://localhost:8080/api/central/sincronizacoes`

### Memória insuficiente ao correr ambos os backends

Cada backend é iniciado com `-Xmx256m` (256 MB máximo). Para reduzir o consumo:

```
java -Xmx128m -jar backend-cadeia/build/libs/backend-cadeia-0.0.1-SNAPSHOT.jar
java -Xmx128m -jar backend-loja/build/libs/backend-loja-0.0.1-SNAPSHOT.jar
```

---

## 13. Paragem do Sistema

Para parar o sistema de forma limpa:

```
# 1. Parar o frontend (Ctrl+C no terminal, ou:)
kill $(lsof -ti :3000)

# 2. Parar o backend-loja (aguardar que a sync pendente termine)
kill $(lsof -ti :8081)

# 3. Parar o backend-cadeia
kill $(lsof -ti :8080)
```

A paragem do backend-loja antes das 23:55 num dia com vendas **não** perde dados

—  
as vendas ficam guardadas na base de dados local e a sync pode ser forçada  
manualmente  
após reiniciar.

---

## Contactos e Suporte

**Projeto:** LI4 — Grupo 9

**Instituição:** Universidade do Minho

**Ano letivo:** 2025/2026

---

*Documento gerado em 2026-05-27*

## **Anexo 2: Ata da Reunião de Kick-off**

### **Ata da Reunião de Kickoff com a Direção Geral**

A reunião de kickoff realizou-se no dia 27 de fevereiro de 2026 e contou com a presença de membros da Direção Geral da empresa e da equipa de desenvolvimento responsável pelo projeto. O principal objetivo deste encontro foi apresentar a visão global do sistema a desenvolver, alinhar expectativas entre as partes envolvidas e estabelecer os princípios orientadores que irão guiar o desenvolvimento da solução ao longo do semestre.

Durante a reunião, a Direção Geral destacou a necessidade de implementar um sistema de gestão integrada que permita às diferentes lojas da rede operar de forma autónoma no seu funcionamento diário, sem dependência constante de um sistema central. No entanto, foi também salientada a importância de garantir a existência de um mecanismo de consolidação automática dos dados gerados por cada loja, de forma a permitir que a informação relevante seja agregada e sincronizada com o sistema central no final de cada dia de operação.

Outro ponto considerado essencial foi o cumprimento rigoroso das obrigações legais associadas ao processo de faturação. A Direção Geral reforçou que o sistema deverá suportar faturação certificada, em conformidade com a legislação portuguesa em vigor, assegurando que todas as transações comerciais realizadas nas lojas possam ser registadas, armazenadas e auditadas de forma adequada.

Além disso, foi manifestado um forte interesse na capacidade de o sistema gerar relatórios analíticos detalhados que permitam à gestão central acompanhar o desempenho das diferentes lojas da rede. Estes relatórios deverão possibilitar análises comparativas entre lojas, identificação de tendências de vendas e avaliação do desempenho de categorias de produtos, constituindo assim uma ferramenta de apoio para processos de tomada de decisão estratégica.

No final da reunião, ficou acordado que a equipa de desenvolvimento deverá trabalhar numa primeira versão funcional do sistema que inclua as funcionalidades essenciais identificadas nesta fase inicial. Esta versão deverá estar disponível até ao final do semestre, permitindo assim a sua demonstração e avaliação por parte da Direção Geral e dos restantes stakeholders envolvidos no projeto.

## **Anexo 3: Ata de Workshop de Requisitos**

### **Ata do Workshop com Gestores de Loja**

O workshop com os gestores de loja teve lugar no dia 3 de março de 2026 e contou com a participação de três gestores responsáveis por diferentes lojas da rede, bem como com os membros da equipa de desenvolvimento do projeto. O principal objetivo desta sessão foi recolher informações detalhadas sobre as necessidades operacionais das lojas, identificar requisitos funcionais relevantes e compreender os principais desafios enfrentados no dia a dia da gestão comercial.

Durante a discussão, os gestores destacaram a importância de o sistema incluir mecanismos automáticos de monitorização do stock disponível em cada loja. Em particular, foi referida a necessidade de implementar alertas automáticos que notifiquem os utilizadores sempre que a quantidade de determinado produto atinja um nível mínimo previamente definido. Este tipo de funcionalidade é considerado essencial para evitar rupturas de stock e garantir a disponibilidade contínua dos produtos mais procurados pelos clientes.

Outro aspecto amplamente discutido foi a eficiência da interface utilizada no processo de venda. Os gestores enfatizaram que o sistema deverá proporcionar uma experiência de utilização simples, rápida e intuitiva para os operadores de caixa. Idealmente, cada transação de venda deverá poder ser concluída num período máximo de aproximadamente trinta segundos, de forma a evitar filas prolongadas e melhorar o atendimento ao cliente.

No que diz respeito à gestão de produtos, foi sugerido que o sistema permite organizar os artigos comercializados de acordo com diferentes categorias, facilitando assim a sua pesquisa e identificação durante o processo de venda. Para além disso, foi considerada importante a possibilidade de associar cada produto a um fornecedor específico, permitindo uma gestão mais eficiente dos processos de reposição e encomendas.

Os gestores manifestaram ainda interesse na disponibilização de relatórios de vendas diários, que permitam acompanhar o desempenho da loja ao longo do dia e identificar rapidamente padrões de consumo. Estes relatórios deverão incluir informações como volume de vendas, produtos mais vendidos, métodos de pagamento utilizados e períodos de maior movimento.

Por fim, foi discutida a necessidade de o sistema suportar múltiplas formas de pagamento, incluindo pagamentos em numerário, cartão bancário e outros métodos digitais. Esta funcionalidade é considerada fundamental para garantir flexibilidade no processo de venda e responder às diferentes preferências dos clientes.

#### Anexo 4: Logo da Universidade do Minho

